

Microservice-Based Neural Network Inference Across Architectural Splits, Cloud Deployment, and Security Hardening

A Staged Empirical Evaluation of ResNet-18 on Azure Kubernetes Service

Nick Glas

Microservice-Based Neural Network Inference Across Architectural Splits, Cloud Deployment, and Security Hardening

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

Nick Glas

under the supervision of
Dr. Ana Maria Oprescu (CCI, UvA)

May 2026



Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands

Complex Cyber Infrastructure
Informatics Institute, University of
Amsterdam
Science Park 904
1098 XH Amsterdam
The Netherlands

Thesis Committee

Examiner (chair):	Dr. Ana Maria Oprescu (CCI, UvA)
Second Reviewer:	Dr. Thomas L. van Binsbergen (CCI, UvA)
Daily Supervisor:	Dr. Damian Frölich (CCI, UvA)
External Supervisor:	Dennis Bijlsma (SIG)

Abstract

Neural network inference is increasingly deployed inside cloud-native software systems that must balance latency, deployability, scalability, and isolation. Splitting a model across microservices can improve modularity and make security boundaries more explicit, but it also forces intermediate activations to cross service interfaces. Each boundary adds serialization, transport, scheduling, and coordination cost. The central problem studied in this thesis is therefore not only where to split a neural network, but whether a decomposition that appears acceptable in a controlled local setting remains credible once it is moved into Kubernetes, Azure, and security-hardened deployment contexts.

This thesis addresses that problem through a staged experimental evaluation of ResNet-18 inference. A proof-of-concept gRPC-based microservice framework is used to compare monolithic inference, two-part model partitions, and synchronously chained multi-service deployments. The evaluation first screens coarse architectural split points under controlled local execution, then refines the selected region and explains the observed behaviour using activation-transfer size and compute distribution. The same decomposition family is then evaluated in local Kubernetes and Azure Kubernetes Service, after which the selected Azure deployment is hardened with service identity and mutual TLS. Confidential-execution mechanisms are treated as a later incremental protection layer on top of the secured Azure path.

The results show that monolithic execution remains the fastest baseline, but that not all microservice decompositions are equally costly. Early splits suffer from large activation transfers, while very late splits can become compute-degenerate. Mid-network boundaries provide the most credible two-part trade-off. Fine-grained refinement within this region does not materially improve latency, supporting the use of coarse architectural boundaries for later stages. In Kubernetes and Azure, latency increases with service count, but the increase is bounded and non-linear: later boundaries add less cost when the transferred activations are smaller. Azure preserves the condition ordering observed in local Kubernetes, indicating directional transfer despite different absolute latencies. Adding mutual TLS and service identity introduces measurable but bounded overhead, alongside clear operational complexity from sidecars, policies, and readiness delays.

Overall, the thesis concludes that microservice-based inference is viable only when architectural partitioning, deployment context, and security techniques are evaluated

together. The main engineering trade-off is not between monolithic and distributed inference in isolation, but between deployability and stronger isolation on one side and cumulative boundary-related overhead on the other.

Contents

Abstract	vii
Contents	ix
List of Figures	xiii
List of Tables	xvi
Acknowledgements	xix
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Approach	3
1.5 Outline	4
2 Background	5
2.1 ResNet-18 as the Inference Workload	5
2.2 Split Inference and Partition Boundaries	6
2.3 Coarse Split Points in ResNet-18	6
2.4 Block-Level Structure and Fine-Grained Splits	7
2.5 Service Boundaries and RPC Cost	7
2.6 Kubernetes Service Chains	8
2.7 Communication and Runtime Protection Boundaries	9
3 Related Work	11
3.1 Split Computing and DNN Partitioning	11
3.2 Granularity and Internal Model Structure	12
3.3 Kubernetes and Cloud-Native Inference	12
3.4 Security Hardening	12
3.5 Positioning of This Thesis	13

4	Methods	15
4.1	Overall Research Design	15
4.2	System Under Study	16
4.2.1	Model	16
4.2.2	Microservice Framework	17
4.3	Shared Benchmark Contract	17
4.4	CPU Stabilisation	18
4.5	Measurement and Statistical Analysis	18
4.5.1	Primary Metrics	18
4.5.2	Cross-Round Consistency	19
4.5.3	Effect Size and Significance Tests	20
4.6	Carry-Forward and Selection Logic	20
4.7	Stage-Specific Methods	20
4.7.1	RQ1.1 – Coarse Architectural Boundary Selection	20
4.7.2	RQ1.2 – Fine-Grained Refinement	21
4.7.3	RQ1.3 – Explanatory Synthesis	21
4.7.4	RQ1.4 – Multi-Microservice Kubernetes Chain	22
4.7.5	RQ1.4b – kind CNI Sensitivity (appendix-only)	22
4.7.6	RQ1.5 – AKS Transfer Validation	23
4.7.7	RQ1.5b – AKS Multi-Node Sensitivity	23
4.7.8	RQ2.1 – Service Identity and Mutual TLS Hardening	24
4.7.9	RQ2.1 (Ablation) – Decomposition of the Hardening Bundle	25
4.7.10	RQ2.1 (Split Sensitivity) – Single-Hop mTLS vs Activation Size	26
4.7.11	RQ2.1b – Multi-Node Sensitivity of mTLS Overhead	26
4.7.12	RQ2.2 – Confidential VM Execution Hardening	26
4.7.13	RQ2.3 – Security-Hardening Trade-off Synthesis	27
4.8	Artifact Freezing and Reproducibility	28
4.9	Limitations and Threats to Validity	28
4.10	Ethical Considerations	29
5	Experiments & Results	31
5.1	RQ1.1 Coarse Architectural Boundary Selection	32
5.1.1	Research Question	32
5.1.2	Experimental Setup	32
5.1.3	Benchmark Design	33
5.1.4	Partition Conditions	33
5.1.5	Results	34
5.2	RQ1.2 Fine-Grained Refinement	38
5.2.1	Research Question	38
5.2.2	Experimental Setup	38
5.2.3	Benchmark Design	38
5.2.4	Refined Partition Conditions	39
5.2.5	Results	40
5.3	RQ1.3 Explanatory Synthesis: Activation-Transfer Size and Compute Distribution	42
5.3.1	Research Question	42

5.3.2	Analytical Setup	42
5.3.3	Explanatory Findings	43
5.4	RQ1.4 Multi-Microservice Kubernetes Inference Chain	48
5.4.1	Research Question	48
5.4.2	Experimental Setup	48
5.4.3	Conditions	48
5.4.4	Results	50
5.5	RQ1.5 Azure Kubernetes Service Transfer Validation	53
5.5.1	Research Question	53
5.5.2	Experimental Setup	53
5.5.3	Conditions	54
5.5.4	Results	54
5.6	RQ1.5b AKS Multi-Node Sensitivity	55
5.6.1	Research Question	55
5.6.2	Experimental Setup	56
5.6.3	Conditions	57
5.6.4	Results	57
5.7	RQ2.1 Service Identity and Mutual TLS Hardening	62
5.7.1	Research Question	62
5.7.2	Experimental Setup	62
5.7.3	Results	63
5.8	RQ2.2 Confidential VM Execution Hardening	66
5.8.1	Research Question	66
5.8.2	Experimental Setup	67
5.8.3	Results	68
5.9	RQ2.3 Security-Hardening Trade-off Synthesis	68
5.9.1	Research Question	68
5.9.2	Synthesis Basis	68
5.9.3	Trade-off Summary	69
6	Analysis	71
6.1	Architectural Split Choice	72
6.1.1	RQ1.1: Coarse Boundary Selection	72
6.1.2	RQ1.2: Fine-Grained Refinement	74
6.1.3	RQ1.3: Explanatory Synthesis	76
6.1.4	Synthesis: Architectural Split Choice	79
6.2	Deployment Context	81
6.2.1	RQ1.4: Local Kubernetes Chain	81
6.2.2	RQ1.4b: kind Multi-Worker CNI Sensitivity	84
6.2.3	RQ1.5: AKS Transfer Validation	87
6.2.4	RQ1.5b: AKS Multi-Node Sensitivity	90
6.2.5	Synthesis: Deployment Context	93
6.3	Security Hardening	94
6.3.1	RQ2.1: Service Identity and Mutual TLS	94
6.3.2	RQ2.2: Confidential VM Execution	95
6.3.3	RQ2.3: Trade-off Synthesis	96

6.3.4	Synthesis: Security Hardening	97
6.4	Validation	98
6.5	Evaluation	100
6.6	Threats to Validity	101
7	Conclusion	103
7.1	Summary & Findings	103
7.2	Contributions	103
7.3	Limitations & Threats to Validity	104
7.4	Future Work	104
A	Appendix	105
A.1	RQ1.4b: kind Multi-Worker CNI Sensitivity Data	105
	References	107

List of Figures

2.1	Top-level ResNet-18 inference pipeline used in this thesis. The model is treated as a stem, four residual stages, and a classification head.	5
2.2	Example two-segment decomposition produced by a split after <code>layer2</code> . The first segment executes the stem through <code>layer2</code> ; the second segment receives the intermediate activation tensor and executes <code>layer3</code> through the classification head.	6
2.3	Coarse ResNet-18 partition boundaries evaluated in RQ1.1. The stem and classification head are collapsed into single blocks, while the four residual stages are shown as the candidate split locations.	7
2.4	Simplified ResNet-18 block structure. The stem and classification head are shown as single components, while each residual stage is shown as two basic blocks.	7
2.5	Detailed ResNet-18 structure with BasicBlock containers. Each residual stage contains two BasicBlocks, and each BasicBlock contains two convolution operations. The first block of layers 2–4 includes a projection shortcut for downsampling.	7
2.6	Block-level two-segment decomposition used in RQ1.2. The refined split point is placed inside <code>layer3</code> , after the first residual block. Segment 1 executes the stem through <code>layer3.0</code> ; Segment 2 receives the intermediate activation tensor and executes <code>layer3.1</code> through the classification head.	8
2.7	Runtime cost components introduced by a two-service inference boundary. In addition to the model computation on each side of the split, the intermediate activation must be serialized, transferred through gRPC, deserialized by the downstream service, and followed by response serialization and reconstruction. The boundary-crossing interval therefore includes serialization, RPC transport, remote execution, and response reconstruction.	8

2.8	Kubernetes pod and service placement for the RQ1.4 five-service chain. The benchmark client and all inference pods are placed on the same Kubernetes node, while each model segment is exposed through a Kubernetes Service and communicates using synchronous gRPC forwarding. The labels above the arrows show the activation-transfer size at each coarse ResNet-18 boundary.	9
2.9	RQ1.5b AKS multi-node placement of <code>chain_5svc</code> . Each chain segment lands on a distinct cluster node; the benchmark client pod lands on a sixth dedicated node. Every gRPC hop therefore crosses a node boundary, in contrast to the same-node layout used for RQ1.4 and RQ1.5 (fig. 2.8). . . .	9
2.10	Protection boundaries in the secured two-service AKS deployment. The service mesh provides communication-level protection between sidecar proxies using mTLS and workload identity. AMD SEV-SNP confidential nodes add a VM-level host-protection boundary around the downstream service's Kubernetes node. The figure is schematic: plaintext activations still exist inside the endpoint after mTLS termination, and the VM-level boundary does not remove all trusted software or residual attack surfaces.	10
5.1	RQ1.1 coarse split latency results under controlled local execution. Bar heights are the mean end-to-end latency per condition for the frozen run <code>rq1_1_20260515_111428</code> ; bar colours encode the role assigned by the carry-forward rule (baseline, selected_main, selected_reference, rejected, degenerate). <code>split_after_layer3</code> is the raw-fastest split in this run and is also non-degenerate; <code>split_after_layer4</code> is excluded by the compute-degeneracy rule despite its small intermediate activation. The carry-forward decision (see <code>carry_forward.json</code> for the same run) selects <code>split_after_layer3</code> as the main candidate and <code>split_after_layer2</code> as the reference candidate. The figure is generated deterministically from the frozen <code>condition_summaries.csv</code> and <code>carry_forward.json</code> by <code>scripts/render_rq11_figure.py</code>	36
5.2	Stage-level compute distribution from the RQ1.3 internal timing analysis. The final classification head contributes less than 1% of total forward-pass compute, while <code>layer4</code> accounts for the largest stage-level share. This explains why <code>split_after_layer4</code> is raw-fast but compute-degenerate: almost all meaningful model computation remains before the split.	45
5.3	Mean end-to-end latency for the RQ1.4 local Kubernetes chain family (<code>rq1_4_fully_controlled_20260515</code>). Latency increases as the predefined service-chain configurations move from one to five services, but the adjacent differences are descriptive comparisons across this predefined family rather than isolated causal estimates for a single added boundary.	51
5.4	RQ1.5 normalised overhead transfer comparison between local Kubernetes and AKS. Overheads are computed relative to each environment's own monolithic Kubernetes baseline, so the figure visualises directional transfer of the service-count trend rather than raw latency equivalence.	55

5.5	Same chain, two topologies. (a) RQ1.4 / RQ1.5 single-node placement: all six pods on one node, every gRPC hop is loopback. (b) RQ1.4b / RQ1.5b multi-node placement: each chain segment lands on a distinct cluster node and the client lands on a sixth dedicated node, so every gRPC hop crosses the Azure VNet. The chain itself is identical between the two panels; only the pod-to-node assignment changes.	56
5.6	RQ1.5b multi-node penalty per condition. Same SKU (Standard_D8s_v3), same region (swedencentral); the intentional experimental contrast is single-node colocation versus multi_node_anti_affinity placement. The figure is manually rendered from the same frozen summary artefacts as table 5.19. Per-condition deltas range from 2.131 ms (chain_3svc) to 4.086 ms (chain_5svc), and the monolithic-baseline penalty is 2.305 ms.	58
5.7	RQ2.1 mTLS/AuthZ latency overhead for the primary and stress AKS chains. Mean and p95 deltas are computed within the paired plain/mTLS design. The stress topology contains four protected inter-service boundaries, so the larger overhead visualises the observed stress-case sensitivity of sidecar-mediated communication hardening.	65

List of Tables

5.1	RQ1.1 benchmark configuration	33
5.2	RQ1.1 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, and overhead vs. monolithic. Source: <code>condition_summaries.csv</code> in <code>rq1_1_20260515_111428</code> . Confidence intervals are reported from the same artifact's <code>ci95_lower_ms / ci95_upper_ms</code> columns and summarise per-iteration variation within this run.	35
5.3	RQ1.2 benchmark configuration	39
5.4	RQ1.2 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, overhead, and activation-transfer burden. Source files: <code>condition_summaries.csv</code> , <code>cross_condition.csv</code> , and <code>round_consistency.json</code> in the frozen artifact <code>rq1_2_20260515_112636</code> . Confidence intervals are reported from the <code>ci95_lower_ms / ci95_upper_ms</code> columns of <code>condition_summaries.csv</code> and summarise per-iteration variation within this run.	40
5.5	RQ1.3 supporting internal timing configuration. Source: <code>config.yaml</code> and <code>instrumentation_overhead.json</code> in <code>internal_timing_resnet18_20260515_113614</code> . 43	
5.6	RQ1.3 supporting internal timing summary used in the explanatory analysis. Source: <code>summary.csv</code> in <code>internal_timing_resnet18_20260515_113614</code> . Classification head is the sum of <code>avgpool</code> and <code>fc</code> . The convolution and batch-norm rows are aggregated over all matching units at level L3 in the same summary file.	44
5.7	RQ1.3 heaviest internal compute units from the supporting timing analysis. Source: <code>summary.csv</code> in <code>internal_timing_resnet18_20260515_113614</code> , sorted by <code>mean_ms</code> at level L3.	44
5.8	RQ1.4 benchmark configuration	49
5.9	RQ1.4 summary of mean latency and overhead vs. Kubernetes monolithic baseline. Source: <code>merged_results/condition_summaries.csv</code> in <code>rq1_4_fully_controlled_20260515_113614</code>	50

- 5.10 RQ1.4 overhead and non-compute/platform cost breakdown. Recorded tensor-byte totals are taken from `total_activation_bytes_mean` in `merged_results/condition_summaries.csv` and converted to KiB (1024 bytes/KiB); the metric includes the input tensor. The platform column is `non_compute_overhead_ms_mean`. 50
- 5.11 RQ1.4 effect sizes relative to monolithic baseline. Source: `merged_results/effect_sizes.csv` in `rq1_4_fully_controlled_20260515_141036`. 51
- 5.12 RQ1.4 cross-round consistency. Source: `merged_results/round_consistency.json` in `rq1_4_fully_controlled_20260515_141036`. 52
- 5.13 RQ1.5 AKS benchmark configuration. Source: `merged_results/config.yaml` and `deployment_metadata.json` in `rq1_5_fully_controlled_20260515_145954`. 59
- 5.14 RQ1.5 AKS summary of mean latency and overhead vs. AKS monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954`. 60
- 5.15 RQ1.5 AKS overhead and inferred non-compute/platform cost breakdown. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954`; the platform column is `non_compute_overhead_ms_mean`. 60
- 5.16 RQ1.5 transfer comparison between local Kubernetes and AKS. Source: `merged_results/condition_summaries.csv` in `rq1_4_fully_controlled_20260515_141036` and `rq1_5_fully_controlled_20260515_145954`. 60
- 5.17 RQ1.5 AKS cross-round consistency. Source: frozen RQ1.5 `merged_results/round_consistency.json`. 60
- 5.18 RQ1.5b AKS multi-node summary of mean latency and overhead vs. AKS multi-node monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_5b_multinode_20260515_152628`. 61
- 5.19 RQ1.5b multi-node penalty, condition-by-condition vs. frozen RQ1.5 single-node AKS. Source: `merged_results/condition_summaries.csv` in `rq1_5_fully_controlled_20260515_145954` and `rq1_5b_multinode_20260515_152628`; overhead points are with respect to each environment's own monolithic baseline (tables 5.14 and 5.18). 61
- 5.20 RQ1.5b AKS multi-node cross-round consistency. Source: `merged_results/round_consistency.json` in `rq1_5b_multinode_20260515_152628`. 61
- 5.21 RQ2.1 paired AKS security benchmark configuration. Sources: `merged/paired_summary.md` in the two artifacts named below. 63
- 5.22 RQ2.1 latency results for plain and mTLS AKS chains. Sources: `merged/aggregated_results.md` in `rq2_1_paired_20260515_160012(chain_2svc)` and `rq2_1_paired_20260517_114303(chain_5svc)`. 64
- 5.23 RQ2.1 paired-pass latency deltas. Sources: `merged/aggregated_results.md` (per-pass tables) in `rq2_1_paired_20260515_160012` and `rq2_1_paired_20260517_114303`. The pass-delta std is computed from the five per-pass deltas. 64
- 5.24 RQ2.1 `chain_2svc` ablation of the communication-hardening bundle. Source: `merged/rq2_1_ablation_summary.md` in `rq2_1_ablation_20260516_112348`. As documented in the evidence manifest, ablation deltas are internally comparable only and must not be spliced into the frozen two-condition paired result. 65
- 5.25 RQ2.1 resource and operational overheads. Sources: `merged/paired_summary.md` in `rq2_1_paired_20260515_160012(chain_2svc)` and `rq2_1_paired_20260517_114303(chain_5svc)`. 66
- 5.26 RQ2.2 benchmark configuration. Sources: `rq22_rolling_summary.json`, `rq22_terraform.tfvars.json`, and `image_reference.txt` in `rq2_2_confidential_20260516_135726`. 67

5.27	RQ2.2 service2-only confidential execution results. Source: per-pass benchmark/raw_iterations.csv in rq2_2_confidential_20260516_135726/conditions/, aggregated across the five standard and five confidential passes (1,000 iterations per condition).	68
5.28	RQ2.3 synthesis of evaluated security-hardening levels. mTLS overheads are taken from table 5.22; the selective TEE overhead is taken from table 5.27 relative to its own paired standard-node mTLS baseline.	70
A.1	RQ1.4b condition summary on the six-worker kind cluster with multi_node_anti_affinity placement. Source: merged_results/condition_summaries.csv in rq1_4b_20260515_163322. Overheads are computed against the RQ1.4b monolithic baseline.	105
A.2	RQ1.4b cross-round consistency. Source: merged_results/round_consistency.json in rq1_4b_20260515_163322. Round CVs are materially inflated relative to the single-node RQ1.4 baseline (table 5.12); the highest CVs do not track chain depth, consistent with kind-CNI variability and shared-host effects on the single physical machine that hosts all six kind worker containers rather than with a chain-specific noise mechanism.	106
A.3	RQ1.4b kind-CNI per-condition penalty, condition-by-condition vs. the frozen single-node RQ1.4 baseline. Source: merged_results/condition_summaries.csv in rq1_4_fully_controlled_20260515_141036 and rq1_4b_20260515_163322. The per-condition delta is computed as $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$, as declared in section 4.7.5. Following the cross-stage host-control caveat in section 4.7.5, the comparison is interpreted at the per-condition delta level only: RQ1.4b differs from RQ1.4 in host control state (governor/turbo/nice) as well as in topology. The monolithic delta is negative on average but is dominated by a first-round shift in RQ1.4b (round means {62.000, 68.553, 68.677, 69.107, 69.119} ms; table A.2) and a heavily skewed per-iteration distribution (mean 67.491 ms vs. median 61.917 ms; table A.1), and should not be read as evidence that the kind multi-worker topology is faster than single-node placement.	106

Acknowledgements

This work was carried out under the supervision of Dr. Ana Maria Oprescu

Introduction

1.1 Context

Deep neural network inference is increasingly embedded in software systems that must satisfy not only predictive requirements, but also operational ones such as latency, deployability, maintainability, scalability, and isolation. Edge-intelligence and split-computing surveys describe this shift as part of a broader move from purely centralized inference toward device, edge, and cloud collaboration [22, 35]. In many practical deployments, inference still runs as a monolithic component inside a single process. That architecture minimizes boundary-crossing overhead, but it also couples model execution into one deployment unit and limits architectural flexibility. A microservice-oriented design offers a different trade-off: the inference pipeline can be decomposed across service boundaries, allowing clearer separation of responsibilities, independent deployment, and more explicit control over resource placement.

For deep learning systems, however, decomposition is not only a software architecture decision. It is also a systems problem shaped by the structure of the neural network itself and by the environment in which the resulting services run. When a model is split across an execution boundary, intermediate activations must be serialized, transferred, and reconstructed before downstream computation can continue. Prior work on split computing and distributed DNN inference shows that these costs are highly sensitive to where the boundary is placed. DNNSplit demonstrates that split points should be selected systematically with respect to latency and cost rather than by arbitrary layer choice [18]. DeeperThings and survey work on DNN partitioning likewise show that early layers often produce large feature maps that are expensive to transmit, while later boundaries may reduce transfer volume but shift the compute balance too far toward one side of the partition [31, 33].

The deployment environment introduces an additional dimension to this problem. A split that is acceptable in a controlled local benchmark may behave differently once it is moved into containerized, orchestrated, or cloud-hosted execution, because container platforms, managed Kubernetes services, and cloud infrastructure add their own scheduling, networking, and virtualization effects [8, 9, 16]. Beyond performance, there is also a security dimension. As inference services move into shared or remote infrastructure,

communication-level protections such as service identity and mutual TLS become relevant because they strengthen inter-service trust boundaries, while service-mesh studies show that this protection can add measurable runtime and resource cost [5, 36]. Mutual TLS terminates at the pod boundary, however, so intermediate activations are still exposed to the runtime environment of the downstream service; prior work shows that an untrusted remote partition holding only those activations can reconstruct the input [12], motivating an additional layer of protection. Confidential-computing techniques such as trusted execution environments may add such a layer, but their overhead and protection boundary are workload- and platform-dependent [10, 24], and recent work shows that confidential VMs still have residual attack surfaces beyond memory encryption, including malicious-host interrupt injection [29]. This makes microservice-based inference a broader software engineering problem involving architectural partitioning, deployment infrastructure, and security mechanisms rather than only local latency optimisation.

1.2 Problem Statement

The central problem addressed in this thesis is how to design and evaluate deep neural network inference as a microservice system without losing control over performance. A poor split can make a distributed design clearly inferior to monolithic execution. Prior partitioning systems show that if the boundary is placed too early, the system may incur excessive latency because large activation tensors must cross the service interface; if the boundary is placed too late, communication cost decreases, but the downstream service may be left with too little computation to justify the split [15, 18, 31]. In other words, minimizing activation size alone is not enough; the chosen boundary must also preserve a meaningful compute distribution across the resulting services.

That architectural problem is only the first layer of the thesis. After a defensible split region has been identified locally, the same decomposition logic must also be evaluated under progressively more realistic deployment conditions. Container orchestration and cloud deployment introduce additional scheduling, networking, and platform overhead that may alter the trade-offs seen in a local benchmark [6, 8]. Finally, if the selected deployment path is security-hardened, the system may incur further overhead in exchange for stronger inter-service authentication, encrypted communication, and confidential execution for selected services [34, 36]. The thesis therefore treats the problem as a staged evaluation pipeline: first identify and explain suitable boundaries under controlled local conditions, then study how a coarse-stage service-chain family behaves in Kubernetes and Azure deployments, and finally measure the impact of adding communication-level and VM-level security hardening to the selected Azure deployment path.

1.3 Research Questions

This thesis is organised around two broader research strands. RQ1 studies how architectural split choice and deployment context affect microservice-based inference. RQ2 studies the performance, operational, and protection-scope trade-offs introduced when the selected Azure deployment path is security-hardened. The research questions are:

- RQ1.1** Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execution, and how do their latency and boundary-crossing costs compare to monolithic inference?
- RQ1.2** How does finer-grained refinement within the late-stage coarse region carried forward from RQ1.1 affect latency and communication-related overhead for two-part microservice inference?
- RQ1.3** How do activation-transfer size and compute distribution explain the latency and boundary-crossing behaviour observed for the decomposition choices evaluated in RQ1.1 and RQ1.2?
- RQ1.4** How does increasing the number of microservices affect end-to-end inference cost when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution?
- RQ1.5** Does the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain transfer to Azure Kubernetes Service, and how do the absolute and relative costs change under managed cloud execution?
- RQ1.5b** When every chain hop is forced across a node boundary, how much additional cost is introduced relative to the same-node RQ1.5 baseline?
- RQ2.1** What performance and operational overhead is introduced when the selected AKS microservice inference path is hardened with service identity, mutual TLS, and inter-service authorization policy?
- RQ2.2** What additional performance and operational overhead is introduced when VM-level confidential execution is added to the already communication-secured AKS inference chain, when the downstream protected service is placed on AMD SEV-SNP confidential nodes?
- RQ2.3** Which security-hardening configuration provides the most appropriate trade-off between performance cost, operational complexity, and protection scope for the selected AKS microservice inference deployment?

RQ1.1 and RQ1.2 form the local split-selection stages. RQ1.3 is an explanatory synthesis that uses activation-transfer size and internal compute distribution to interpret those local results. RQ1.4 and RQ1.5 then evaluate the same coarse-stage chain family in local Kubernetes and AKS, while RQ1.5b acts as a sensitivity stage for cross-node placement. RQ2.1 adds communication-level hardening through service identity, mutual TLS, and authorization policy. RQ2.2 adds selective AMD SEV-SNP VM-level confidential execution for the downstream protected service. RQ2.3 synthesises those security stages against the plain AKS baseline.

1.4 Approach

The thesis uses a staged experimental methodology centred on ResNet-18, a residual architecture whose stage structure provides natural coarse split candidates [11]. The initial benchmark compares one monolithic baseline against four two-part split configurations placed after `layer1`, `layer2`, `layer3`, and `layer4`. These boundaries correspond to major architectural stages in the network and therefore provide a principled coarse sweep from early to late splits. This first stage is executed locally with CPU-only inference, FP32

precision, gRPC over localhost, fixed warmup and measurement windows, and functional parity checks between monolithic and split execution so that boundary effects can be isolated under controlled conditions, following systems-benchmarking guidance that emphasises controlled comparisons and explicit reporting of measurement context [13].

The local selection stages are then refined and explained before the deployment context is changed. A block-level split inside the carried-forward region tests whether finer granularity changes the conclusion, while an internal timing study of the monolithic model supports the interpretation of activation-transfer and compute-distribution effects. The same model and benchmark contract are then carried into a predefined Kubernetes service-chain family, from a one-service monolithic deployment to a five-service coarse-stage chain, and this family is evaluated first in local Kubernetes and then on Azure Kubernetes Service.

The final stages add security hardening to the selected Azure path. RQ2.1 compares the plain AKS chain against a managed-Istio configuration with service identity, mutual TLS, and inter-service authorization policy. RQ2.2 keeps that communication-security contract fixed and moves the downstream protected service onto AMD SEV-SNP confidential nodes to measure the incremental cost of VM-level confidential execution. RQ2.3 then compares the plain, communication-secured, and selectively confidential configurations. This design follows the broader literature showing that DNN partitioning must be evaluated as a trade-off between communication and computation rather than as a purely architectural choice [19, 25, 27]. In this thesis, that trade-off is examined across multiple deployment and protection layers rather than being treated as a purely local benchmarking problem.

1.5 Outline

The remainder of this thesis is organised as follows. Chapter 2 introduces the technical background required for understanding deep neural network inference, ResNet-style architectures, microservice-based deployment, and the broader systems context in which distributed inference is evaluated. Chapter 3 reviews prior literature on split computing, DNN partitioning, distributed inference, and the trade-offs that motivate this thesis. Chapter 4 describes the research methodology and the staged evaluation design used to move from local boundary screening to broader deployment and security contexts. Chapter 5 presents the empirical evaluation stages, beginning with RQ1.1 and continuing through the later deployment-oriented and security-hardening analyses. Chapter 6 interprets those findings in relation to the research questions and discusses their implications and limitations. Finally, Chapter 7 summarises the thesis and outlines directions for future work.

Background

This chapter introduces the technical concepts needed to interpret the staged experiments. It explains the ResNet-18 architecture used as the inference workload, the meaning of a split boundary in distributed DNN inference, the difference between coarse and block-level split granularity, and the service-chain execution model used in the Kubernetes and AKS stages. It also introduces the protection boundaries used in the security-hardening stages. The chapter refers to literature where it clarifies a technique or concept; the more comparative discussion of prior work appears in chapter 3.

2.1 ResNet-18 as the Inference Workload

ResNet-18 can be viewed as three major parts for the purposes of this thesis: an initial stem, four residual stages, and a classification head. The stem performs the first feature extraction and spatial downsampling through convolution, normalisation, activation, and max pooling. The four residual stages, denoted layer 1 through layer 4, contain the main residual-block computation of the network. The classification head then reduces the final feature map through average pooling and applies the final fully connected classifier. This stage-based view follows the residual-network design introduced by He et al., where successive stages reduce spatial resolution while increasing channel depth [11].

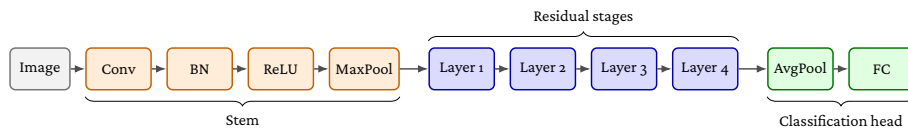


Figure 2.1: Top-level ResNet-18 inference pipeline used in this thesis. The model is treated as a stem, four residual stages, and a classification head.

2.2 Split Inference and Partition Boundaries

In split inference, a neural network forward pass is divided into execution segments. An upstream segment computes an intermediate activation tensor, transfers that tensor to another execution environment, and a downstream segment completes the forward pass. Prior split-inference systems use this same basic abstraction when partitioning neural network inference across devices, edge nodes, or cloud resources [7, 15].

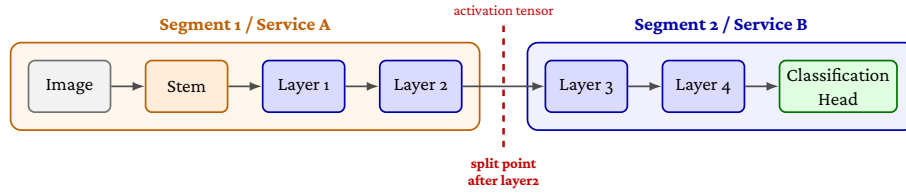


Figure 2.2: Example two-segment decomposition produced by a split after layer2. The first segment executes the stem through layer2; the second segment receives the intermediate activation tensor and executes layer3 through the classification head.

A split boundary is therefore both an architectural position in the model and a runtime interface between two execution segments. For example, `split_after_layer2` creates the two-segment decomposition shown in fig. 2.2: the first segment executes the model up to the boundary, and the second segment resumes computation from the transferred activation tensor.

2.3 Coarse Split Points in ResNet-18

The stage structure of ResNet-18 motivates the coarse split choices used later in RQ1.1. The thesis does not treat every primitive operation as an equally meaningful split candidate. Instead, the initial coarse sweep uses the four residual-stage boundaries after layer1, layer2, layer3, and layer4. These boundaries preserve the architectural stage structure of ResNet-18 and provide a controlled progression from early, high-activation regions to later, lower-activation regions.

Splits inside the stem are excluded from the primary coarse sweep because they would place boundaries between tightly coupled primitive operations before substantial activation-size reduction. Splits inside the classification head are also excluded because the head contains only the final pooling and linear classification operations, leaving one side of the partition with negligible computation. This framing follows partitioning literature that treats split selection as a trade-off among activation-transfer cost, compute placement, and deployment context rather than as a search over depth alone [18, 31, 33].

As shown in fig. 2.3, RQ1.1 evaluates split points after the four major residual stages rather than inside the stem or classification head.

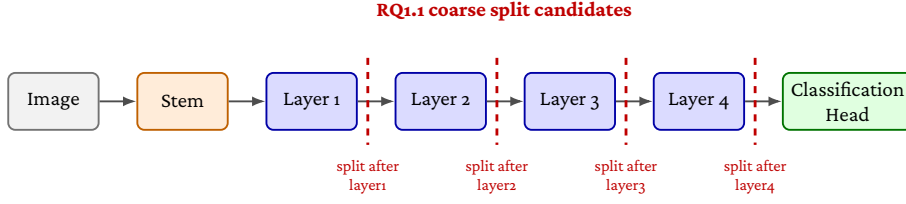


Figure 2.3: Coarse ResNet-18 partition boundaries evaluated in RQ1.1. The stem and classification head are collapsed into single blocks, while the four residual stages are shown as the candidate split locations.

2.4 Block-Level Structure and Fine-Grained Splits

Each residual stage in ResNet-18 contains two BasicBlock containers, and each BasicBlock contains two convolution operations. The first block of layers 2–4 also uses a projection shortcut for downsampling. This internal structure is hidden in the coarse stage-level view, but it matters when reasoning about finer split boundaries [11].

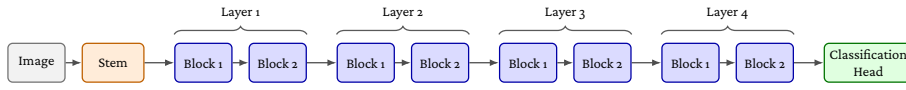


Figure 2.4: Simplified ResNet-18 block structure. The stem and classification head are shown as single components, while each residual stage is shown as two basic blocks.

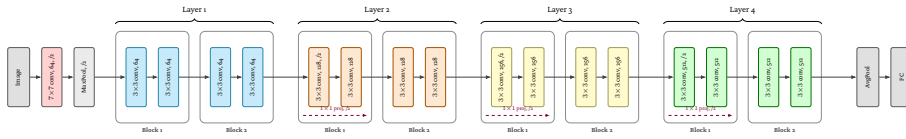


Figure 2.5: Detailed ResNet-18 structure with BasicBlock containers. Each residual stage contains two BasicBlocks, and each BasicBlock contains two convolution operations. The first block of layers 2–4 includes a projection shortcut for downsampling.

The block-level view matters because two boundaries with the same activation tensor can still assign different amounts of computation to the upstream and downstream services. The refined boundary inside Layer 3 used later in the thesis should therefore be read as a compute-placement distinction, not only as an activation-size distinction. Prior work on fine-grained and hierarchical DNN partitioning similarly argues that useful split selection requires awareness of the model’s internal computational structure [20, 27, 32].

2.5 Service Boundaries and RPC Cost

A split boundary becomes a service boundary when the execution segments are packaged as independently addressable services. In that setting, the boundary-crossing interval

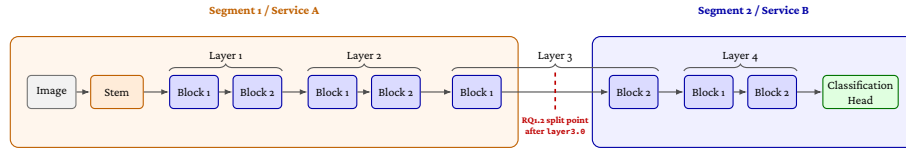


Figure 2.6: Block-level two-segment decomposition used in RQ1.2. The refined split point is placed inside layer 3, after the first residual block. Segment 1 executes the stem through layer 3.0; Segment 2 receives the intermediate activation tensor and executes layer 3.1 through the classification head.

used later in the thesis should be read as a compound systems cost rather than as network transfer alone. It includes tensor serialization, RPC handling, transport through the service interface, downstream deserialization, remote computation after the boundary, and response handling.

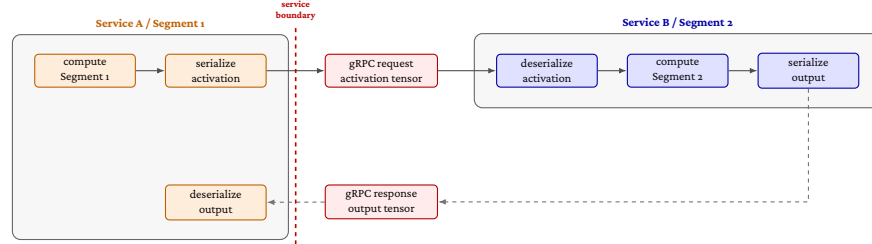


Figure 2.7: Runtime cost components introduced by a two-service inference boundary. In addition to the model computation on each side of the split, the intermediate activation must be serialized, transferred through gRPC, deserialized by the downstream service, and followed by response serialization and reconstruction. The boundary-crossing interval therefore includes serialization, RPC transport, remote execution, and response reconstruction.

Recent microservice systems work shows that RPC paths can accumulate overhead through protocol processing, memory copies, socket operations, and network-stack traversal, even before the application-specific computation is considered [2, 37].

2.6 Kubernetes Service Chains

In the Kubernetes and AKS stages, each model segment is packaged as a separate service and communicates through synchronous gRPC forwarding. This makes the inference pipeline closer to a cloud-native microservice deployment, but it also introduces platform effects from container execution, service networking, CNI behavior, and managed-cluster infrastructure. The single-node and multi-node placements used later are shown in figs. 2.8 and 2.9.

Prior Kubernetes and cloud-performance studies show that container execution, service networking, CNI behavior, and managed infrastructure can affect measured

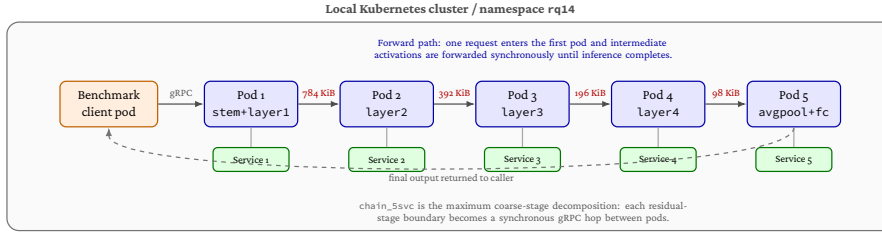


Figure 2.8: Kubernetes pod and service placement for the RQ1.4 five-service chain. The benchmark client and all inference pods are placed on the same Kubernetes node, while each model segment is exposed through a Kubernetes Service and communicates using synchronous gRPC forwarding. The labels above the arrows show the activation-transfer size at each coarse ResNet-18 boundary.

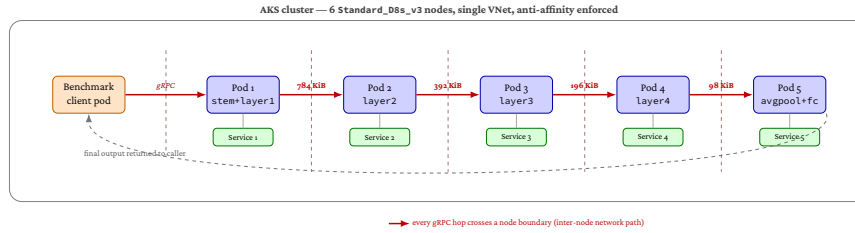


Figure 2.9: RQ1.5b AKS multi-node placement of chain_5svc. Each chain segment lands on a distinct cluster node; the benchmark client pod lands on a sixth dedicated node. Every gRPC hop therefore crosses a node boundary, in contrast to the same-node layout used for RQ1.4 and RQ1.5 (fig. 2.8).

latency and variability. This is why the terminology in this thesis distinguishes local process benchmarks, local Kubernetes benchmarks, and managed AKS benchmarks as separate execution contexts [6, 8, 16].

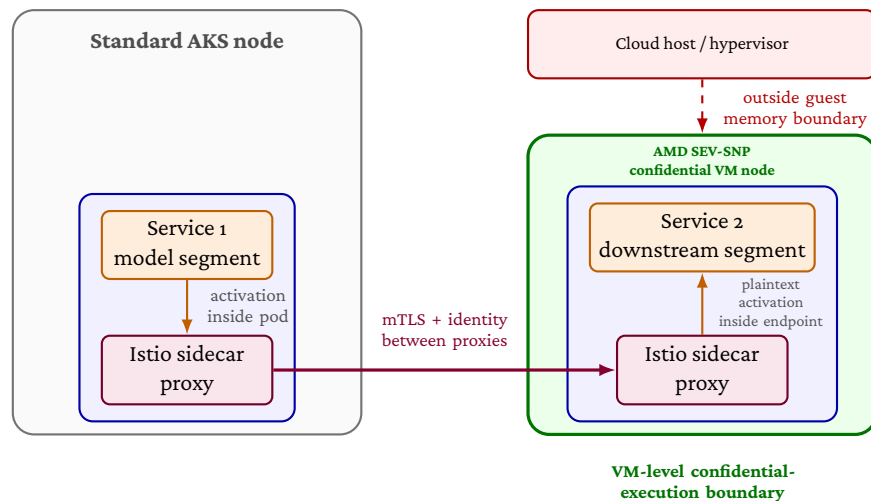
2.7 Communication and Runtime Protection Boundaries

The security-hardening stages use two distinct protection layers. The first layer is communication-level hardening. In a service-mesh deployment, sidecar proxies can attach identity to workloads, establish mutually authenticated TLS channels between services, and enforce authorization policy based on those identities. This strengthens the service-to-service path because a downstream service can reject traffic that does not come from the expected peer identity, while the traffic between proxies is encrypted and mutually authenticated [4, 5].

This protection boundary is still a communication boundary. Once a request has reached the downstream endpoint and the activation tensor has been deserialized, the plaintext activation exists inside the receiving service’s runtime environment. Communication hardening therefore protects the inter-service path, but it does not by itself protect the downstream service from the infrastructure on which it runs or from software

inside the receiving runtime.

The second layer used later in the thesis is VM-level confidential execution. On Azure, confidential VM nodes backed by AMD SEV-SNP protect the guest VM's memory and execution state against parts of the host platform and hypervisor, shifting the relevant boundary from the network path to the VM boundary [3]. In this thesis, this mechanism is applied selectively to the downstream service in the secured two-service chain. The term *VM-level confidential execution* is used deliberately: the protected unit is the VM that hosts the Kubernetes node, not an application-level enclave around only the model code or activation tensor.



mTLS protects traffic on the service-to-service path. SEV-SNP shifts the downstream service into a confidential VM boundary, but plaintext still exists inside the trusted endpoint after decryption.

Figure 2.10: Protection boundaries in the secured two-service AKS deployment. The service mesh provides communication-level protection between sidecar proxies using mTLS and workload identity. AMD SEV-SNP confidential nodes add a VM-level host-protection boundary around the downstream service's Kubernetes node. The figure is schematic: plaintext activations still exist inside the endpoint after mTLS termination, and the VM-level boundary does not remove all trusted software or residual attack surfaces.

This distinction, shown in fig. 2.10, is important for interpreting the security results. Service-mesh mTLS and authorization answer whether the selected AKS inference path can be protected at the communication level and what that costs. SEV-SNP-backed confidential nodes answer a different question: what extra cost is paid when the downstream service is also placed inside a VM-level host-protection boundary. Neither layer removes every trusted component or eliminates all residual attack surfaces; confidential-VM systems still have bounded threat models and known residual risks [10, 29].

Related Work

This chapter positions the thesis within prior work on split neural-network inference, cloud-native deployment overhead, and security hardening for microservice systems. The central distinction is that most prior partitioning work focuses on selecting or optimising split points, while this thesis follows one workload through a staged software-engineering evaluation: local split screening, Kubernetes deployment, managed AKS transfer validation, and security hardening.

3.1 Split Computing and DNN Partitioning

Early split-computing systems established that neural-network inference can be divided between execution environments, but that the split point determines whether the distributed design is useful. Neurosurgeon models the latency trade-off between mobile and cloud execution and shows that different layers have different computation and data transfer profiles [15]. JointDNN frames partitioning as an optimisation problem in which upload and download costs depend on the intermediate tensor at the chosen boundary [7]. DeeperThings extends this line of work to fully distributed CNN inference on constrained edge devices and reinforces the observation that early feature maps can be expensive to move because they remain spatially large [31].

More recent work broadens the partitioning objective beyond a single latency value. DNNSplit selects split points by jointly considering latency and cost in multi-tier settings [18]. Genetic-algorithm and delay-aware approaches likewise treat the split point as dependent on device capability, bandwidth, throughput, and parallelism constraints [19, 25]. Survey work on DNN partitioning and split computing summarizes this literature as a multi-objective problem involving activation size, compute allocation, deployment tier, and runtime conditions [33]. This thesis adopts that view but keeps the optimisation scope deliberately narrow: it studies a fixed ResNet-18 workload and evaluates how a small number of architecturally meaningful split choices behave across deployment layers.

3.2 Granularity and Internal Model Structure

Several studies show that coarse layer depth is not enough to explain split performance. Fine-grained elastic partitioning demonstrates that more detailed placement decisions can improve distributed DNN inference when the placement is aware of both computation and communication conditions [27]. Hierarchical partitioning similarly combines global partitioning with local refinement, showing that staged decisions can account for heterogeneity at both cluster and node levels [32]. Pareto-front analysis of DNN partitioning further shows that execution time can vary across blocks inside the same model, so boundaries with similar activation sizes may still differ in compute distribution [20]. BottleFit and BottleNet++ approach the problem from the feature-compression side, showing that intermediate representations can be compressed or shaped to reduce communication cost, but that the position of the boundary remains central to the latency and accuracy trade-off [21, 30].

The experiments in this thesis use these findings to justify a coarse-to-fine design. RQ1.1 evaluates residual-stage boundaries in ResNet-18, and RQ1.2 adds a block-level boundary inside the carried-forward region. The goal is not to claim that the selected boundaries are globally optimal for all DNNs. Instead, the goal is to show how activation-transfer size and compute placement interact in a reproducible microservice setting.

3.3 Kubernetes and Cloud-Native Inference

Moving from local split inference to Kubernetes changes the systems context. A microservice boundary is not only a model partition; it is also an RPC path with serialization, protocol handling, socket operations, scheduling effects, and service networking. Recent RPC and microservice studies show that communication overhead can come from layered protocol processing and network-stack traversal, not only from the application payload itself [2, 37]. Service-mesh work reaches a similar conclusion for proxied microservices: sidecars and data-plane components add workload-dependent overhead through additional critical-path processing [36].

Kubernetes and cloud studies also show why local and managed-cluster results should be compared carefully. Managed Kubernetes performance varies across services and configurations [8]. Container and VM layers can differ from bare-metal execution [9]. CNI plugins and edge-oriented Kubernetes networking can affect communication cost [16]. Cloud environments also exhibit network-performance variability that can change application scalability and measurement stability [6]. For this reason, this thesis does not treat the local Kubernetes and AKS stages as numerical replicas. Instead, RQ1.5 asks whether the ordering and relative overhead trends from local Kubernetes transfer directionally to managed AKS.

3.4 Security Hardening

The security stages build on prior work in service identity, mTLS, service meshes, and confidential computing. Policy-driven Kubernetes security work motivates combining encrypted communication with explicit authorization policy rather than treating TLS

alone as the complete protection mechanism [4]. Service mesh studies show that sidecar-based mTLS can improve service-to-service trust boundaries, but also that it can add latency and CPU/resource overhead, with the cost depending on mesh configuration and workload [5, 36]. Work on service-mesh control-plane trust further cautions that mesh CAs and control planes remain security-critical components [26]. RQ2.1 therefore treats mTLS and authorization as a bounded communication-hardening layer, not as a complete security solution.

Confidential-computing work motivates the second hardening layer. The motivation itself comes from work on activation-level privacy attacks: He et al. [12] demonstrate that a remote partition holding only intermediate feature maps can reconstruct the edge client’s input, and that simple noise-based defenses fail. This shows that in-transit mTLS alone does not protect activations once they reach the downstream segment, although their threat model is malicious-remote-service rather than the infrastructure-level adversary that confidential VMs address.

AMD SEV-SNP provides VM-level memory encryption and integrity protection against parts of the host platform [3, 17]. Comparative studies of virtualization-based confidential-computing platforms emphasize that these systems have specific attacker models, attestation mechanisms, and residual limitations [10, 23]. Performance studies show that confidential VM overhead is workload-dependent, with CPU-bound, memory-intensive, and I/O-heavy workloads affected differently [1, 23]. Machine-learning confidential computing surveys identify protected model execution and data privacy as important motivations, while also emphasizing deployment and performance trade-offs [24]. Recent work additionally shows that the hypervisor remains a meaningful adversary even under SEV-SNP: HECKLER [29] uses malicious-interrupt injection to bypass OpenSSH and sudo authentication on production SEV-SNP and TDX, and additionally breaks execution integrity of analytical workloads on SEV-SNP, an attack surface that hardware memory encryption alone does not cover. RQ2.2 follows this bounded framing by measuring VM-level confidential execution on AKS for the downstream service only, rather than claiming application-level enclave isolation or evaluating those residual attack surfaces.

3.5 Positioning of This Thesis

The related literature establishes the individual concerns that motivate this thesis: split points affect activation-transfer and compute costs, microservice and Kubernetes deployment add platform overhead, and security hardening strengthens protection while introducing additional cost. The contribution of this thesis is to connect those concerns in one staged empirical pipeline. Instead of evaluating partitioning, orchestration, and hardening as separate problems, the thesis tracks the same ResNet-18 microservice inference workload from local split selection through Kubernetes, AKS, mTLS/AuthZ, and confidential VM execution.

Methods

This chapter describes the research design used to answer the research questions defined in Chapter 1. It explains which methods and procedures are used at each stage, why those methods are appropriate, and what the known limitations and pitfalls of the chosen approach are. The thesis addresses a problem that spans software architecture, containerised deployment, cloud infrastructure, and communication-level security. Studying these layers in a single undifferentiated experiment would confound the effects of partition boundary placement, orchestration overhead, and security mechanisms into one mixed result. The methodology therefore follows a deliberate *staged experimental pipeline* in which each stage changes one layer of the system at a time, preserves the model and benchmark contract, and compares against the nearest relevant baseline.

4.1 Overall Research Design

The thesis is organised around two research strands and a staged experimental pipeline. The primary benchmark stages are RQ1.1, RQ1.2, RQ1.4, RQ1.4b, RQ1.5, RQ1.5b, RQ2.1, and RQ2.2. RQ1.3 is an explanatory synthesis supported by a separate monolithic timing experiment, and RQ2.3 is the final security trade-off synthesis. RQ1 studies how architectural split choice and deployment context affect microservice-based neural network inference. RQ2 studies the performance and operational cost of communication-level and runtime-level security hardening applied to the AKS deployment path selected from RQ1.

The stages unfold in the following sequence. RQ1.1 evaluates four coarse ResNet-18 stage boundaries under tightly controlled local CPU-only execution in order to identify which two-part decompositions are credible enough to carry forward before the thesis moves to orchestrated or cloud deployment. RQ1.2 tests one block-level split inside the carried-forward late-stage region to determine whether finer granularity reveals meaningful differences beyond the coarse level; it is a targeted refinement, not a new blind search. RQ1.3 is a hybrid analytical stage that explains the RQ1.1 and RQ1.2 latency patterns using two structural properties of the partition – activation-transfer size and compute distribution – drawing on the prior benchmark summaries and on a separately collected internal timing experiment on the monolithic model, without conducting a

new split benchmark. RQ1.4 moves a predefined family of one-to-five service chains into a local single-node kind Kubernetes cluster to measure how end-to-end latency accumulates as synchronous service boundaries are added under in-cluster orchestration. RQ1.5 re-runs the same predefined chain family on Azure Kubernetes Service (AKS) to test whether the local Kubernetes ordering and overhead pattern transfer directionally to managed cloud execution. RQ1.5b is an AKS multi-node sensitivity stage that re-runs the same chain family on a six-node cluster with pod anti-affinity, so every chain hop is forced across a node boundary; it quantifies the inter-node penalty that the single-node RQ1.5 design intentionally hides. RQ2.1 adds managed-Istio mTLS, service accounts, and authorisation policy to the selected AKS deployment path, measuring the resulting latency, resource, and operational overhead using a paired execution design. RQ2.2 adds AMD SEV-SNP VM-level confidential execution to the already communication-secured chain from RQ2.1, measuring the incremental cost of selective downstream-service confidential-node placement (`service2_only`); extending the confidential boundary to both services is out of the thesis-facing scope and is treated as future work. RQ2.3 synthesises the plain AKS baseline from RQ1.5, the mTLS/AuthZ results from RQ2.1, and the selective TEE result from RQ2.2 into a trade-off comparison across cost, operational complexity, and protection scope, without collecting any new benchmark data.

The common design principle across all stages is to change one layer of the system at a time. This is a local design choice for the staged pipeline, motivated by general benchmarking guidance on reproducibility, interpretability, and the careful description of experimental conditions in systems performance work [13], rather than a rule prescribed verbatim by that source.

4.2 System Under Study

4.2.1 Model

The neural network architecture used in all stages is ResNet-18 [11]. The implementation loads the Torchvision default pretrained ImageNet weights and keeps the model fixed across all conditions. ResNet-18 is organised into four residual stages (`layer1` through `layer4`). In this implementation the feature maps after these stages have shapes $64 \times 56 \times 56$, $128 \times 28 \times 28$, $256 \times 14 \times 14$, and $512 \times 7 \times 7$ respectively. Thus, after `layer1`, later stage boundaries halve spatial resolution and double channel count. This structure provides architecturally motivated split candidates with different activation sizes and computational loads.

ResNet-18 is chosen for three reasons. First, its stage structure supports a principled coarse-to-fine screening methodology. Second, its size enables CPU-only inference at latencies realistic enough for measurement while avoiding the confounds of GPU scheduling variance in the early controlled stages. Third, as a standard reference architecture, its behaviour under partition is directly relatable to prior split-computing and DNN-partitioning work [15, 31, 33].

All experiments use FP32 precision with a fixed input shape of $1 \times 3 \times 224 \times 224$ and a batch size of one. Batch size one reflects the latency-oriented framing of the research questions: the thesis measures per-request end-to-end inference latency rather than throughput.

4.2.2 Microservice Framework

A proof-of-concept gRPC-based microservice framework was developed for this thesis. Each service runs as an independent process – or, in the Kubernetes stages, as an independent pod – that receives an input tensor over gRPC, executes its assigned segment of the ResNet-18 model, and transmits the resulting intermediate activation to the next service in the chain. The final service returns the classification output to a benchmark client.

gRPC is chosen as the inter-service protocol because it provides a standard RPC stack over typed Protocol Buffer messages. This keeps the benchmark close to cloud-native microservice practice while still making the transport path explicit; prior microservice-systems work treats RPC communication and Protocol Buffer serialisation as central sources of latency overhead in such systems [37]. The maximum gRPC message size is set to 16 MiB in all stages, which accommodates the largest intermediate activation tensor in the study (approximately 784 KiB for a split after layer 1 in FP32).

The framework supports variable chain depths: the same code path handles two-service chains used in RQ1.1, RQ1.2, and RQ2, and chains of up to five services used in RQ1.4 and RQ1.5. The independent variable in each stage is therefore the choice of split point or the number of chained services, not the communication mechanism.

4.3 Shared Benchmark Contract

To support direct comparison across stages, the thesis defines a shared benchmark contract that is preserved unless a stage explicitly states otherwise. The fixed subject under test is ResNet-18 with pretrained weights, executed on CPU in FP32 with input shape $1 \times 3 \times 224 \times 224$, communicating over gRPC on port 50051 with a maximum message size of 16 MiB.

The shared timing protocol consists of 50 warmup iterations followed by 200 measured iterations per condition execution, with a five-second cooldown between successive conditions, a base random seed of 42 from which a per-round permutation seed $\text{seed} + \text{round_index}$ is derived so that condition order is independently permuted in each round, and a warmup calibration check using a trailing window of 10 iterations and a coefficient-of-variation (CV) threshold of 0.02. Thesis-facing stages aggregate five rounds or five paired passes, yielding 1,000 measured iterations per condition.

The warmup calibration is implemented as a minimum-plus-stability rule rather than a fixed 50-iteration ceiling. The runner always executes at least the configured 50 warmup iterations, then continues until the trailing-window CV falls below the 0.02 threshold. In the source configs, `max_extra_iterations = -1` denotes no experiment-level extra-warmup cap; the implementation still applies a safety cap of 10 000 additional iterations to prevent non-terminating runs. Each condition’s actual `total_iterations` and `extra_iterations_used` are recorded in the frozen `warmup_calibration.json` artefact for that run.

The shared functional validation contract requires local segment validation before timing and live gRPC chain validation for any split or Kubernetes deployment, using an absolute tolerance of $\text{atol} = 1 \times 10^{-5}$ and five validation inputs unless a stage explicitly records a stricter tolerance. A condition is not benchmarked unless both checks pass.

In practice, all thesis-facing frozen runs achieve a maximum absolute difference of 0.0 against the monolithic reference, confirming that no numerical drift is introduced by the partition or transport layer; the RQ1.3 internal timing run records the same outcome under the stricter tolerance $\text{atol} = 1 \times 10^{-6}$.

This shared contract is the invariant across the staged pipeline: what changes between stages is the infrastructure and security context, not the model, the measurement discipline, or the validation requirements.

4.4 CPU Stabilisation

Systematic CPU stabilisation is applied in all local and Kubernetes stages to reduce variance attributable to hardware noise rather than to the partition boundary. The following controls are requested in every thesis-facing local run. Thread counts for PyTorch intra-op and inter-op threads, and for OpenMP, MKL, and OpenBLAS, are all set to one, ensuring single-threaded model execution. CPU affinity is pinned to one physical core with simultaneous multithreading (SMT) excluded, preventing the benchmark process from migrating between cores or sharing a physical core with a sibling hyperthread. Process scheduling priority is raised using `nice -5` where the runtime permits. The CPU frequency governor is set to `performance` and turbo boost is disabled to prevent frequency scaling from affecting measurements.

These controls are applied symmetrically across all conditions within a benchmark so that differences in measured latency can be attributed to the independent variable rather than to hardware variability.

In containerised and cloud stages, some controls are partially unavailable. In the local `k8s` Kubernetes environment (RQ1.4), `governor` and `turbo-disable` writes to `/sys` are blocked by the read-only `sysfs` exposed inside the container; however, the detected governor is already `performance` and turbo is already disabled, so the intended state is achieved through the host configuration. In AKS (RQ1.5 and RQ2), the `nice -5` request is denied due to insufficient privileges and the benchmark runs at `nice = 0`; governor and turbo controls are unavailable through the managed node interface. These constraints are documented per stage and treated as realistic limitations of cloud-native deployment rather than as experimental flaws, because they affect all conditions within a stage symmetrically.

4.5 Measurement and Statistical Analysis

4.5.1 Primary Metrics

End-to-end latency is the primary measurement in all stages, defined as the wall-clock time for one full inference request from the moment the benchmark client dispatches the input to the moment it receives the final classification output.

Results are summarised using the arithmetic mean, median, and standard deviation of per-iteration latencies. The median provides a robust central tendency estimate in the presence of tail spikes. The mean is used as the primary comparison value because the thesis is concerned with expected per-request cost. The 95th-percentile (p95) latency

is reported in the deployment and security stages to characterise tail behaviour under more realistic conditions.

Overhead is reported both in absolute milliseconds and as a percentage of the relevant baseline mean latency. In stages that compare conditions across environments (RQ1.5), within-environment normalised overhead is the primary comparison metric rather than raw absolute latency, because absolute values are environment-specific and do not transfer across different host CPUs, kernels, and virtualisation layers.

Several derived metrics are used in specific stages, and three of them deserve an explicit caveat because they are *not* directly measured quantities even though they appear as numbers in result tables.

The *boundary-crossing interval* is a composite that bundles serialisation, transport, and the work executed on the remote side after the boundary. It is not the cost of “crossing the wire” alone – the thesis does not instrument the gRPC stack internally – but the time observed at the client between dispatching the request and receiving the response, minus the local pre-boundary compute. Any interpretation as a “transport cost” alone is therefore unsafe; it is a from-here-to-there interval, not a pure transport measurement.

The *activation-transfer burden* is derived from the model architecture as the intermediate tensor size at a split boundary (or the cumulative tensor size across all boundaries in a chain). It is an architectural quantity, not a measured one: no network profiler is used to confirm the on-wire byte count. The 16 MiB gRPC message limit and the parity validation against the monolithic model together ensure that the architectural size is what is actually transmitted, but the figure itself is computed from model shapes rather than from packet capture.

The *inferred non-compute / platform cost* is an analytical residual:

$$\widehat{\text{non-compute}} = \widehat{\text{end-to-end}} - \widehat{\text{compute}},$$

where the compute estimate comes from per-segment local timing. It is used in the Kubernetes, AKS, and mTLS analyses to separate platform and communication overhead from model computation. As a residual it inherits the error of the compute estimate; it cannot be falsified against a directly measured ground truth and should be interpreted as an upper bound on the platform + communication share rather than as a direct measurement of it.

In RQ2.1, resource and operational overhead metrics are also reported, including side-car CPU and memory, total pod resource deltas, schedule-to-ready time, and the count of added mesh and security objects. In RQ2.2, sequential throughput is derived from mean latency as requests per second to express the throughput impact of confidential-node placement; this is an algebraic restatement of mean latency at batch size one, not an independent throughput measurement under load.

4.5.2 Cross-Round Consistency

Cross-round consistency is assessed using the round CV, computed as the standard deviation of per-round condition means divided by the grand mean. A low round CV indicates stable execution across independent rounds and supports treating the reported condition means as representative rather than as artefacts of a single favourable or

unfavourable run. Cross-round stability and parity validation are treated as the primary indicators of methodological validity.

4.5.3 Effect Size and Significance Tests

In stages with 1,000 per-condition measured iterations, Cohen’s d is computed from pooled per-iteration data and the Mann-Whitney U test is reported for completeness. These statistics are descriptive diagnostics rather than the main decision rule. The thesis bases methodological carry-forward decisions on the predeclared eligibility and near-best-window rules in section 4.6; statistical-test outputs are used to contextualise the size and consistency of observed differences.

4.6 Carry-Forward and Selection Logic

The staged pipeline uses a carry-forward mechanism for the local split-selection stages (RQ1.1 and RQ1.2). After each of these stages, one or two candidates are selected to enter the next stage according to two predeclared rules.

The first rule concerns compute degeneracy. A split condition is classified as compute-degenerate if the smaller of the two services contributes less than 10% of the total split compute, i.e. $\min(t_a, t_b)/(t_a + t_b) < 0.10$, where t_a and t_b are the measured per-segment compute times. Compute-degenerate conditions are excluded from main-candidate eligibility because they produce structurally unbalanced decompositions that do not constitute meaningful microservice partitions, even if their raw latency is low.

The second rule selects two candidates from the eligible (non-degenerate) set inside the 5% near-best window: the condition with the lowest mean end-to-end latency is selected as the main carry-forward candidate, and the next-lowest is retained as the reference candidate. Ties on mean latency are broken in favour of the smaller activation-transfer size, so that activation size acts as a structural tie-breaker rather than as a primary selection criterion.

Carry-forward is not applicable in RQ1.4 and RQ1.5, which benchmark a predefined chain family rather than selecting among candidates. It is also not applicable in the RQ2 stages, which fix the deployment topology established by RQ1 and vary only the security configuration.

4.7 Stage-Specific Methods

4.7.1 RQ1.1 – Coarse Architectural Boundary Selection

RQ1.1 screens the four coarse stage boundaries of ResNet-18 against a monolithic baseline under the maximally controlled local setup described in Sections 4.4 and 4.3. The four split conditions place the boundary after `layer1`, `layer2`, `layer3`, and `layer4`, producing intermediate activation tensors of approximately 784 KiB, 392 KiB, 196 KiB, and 98 KiB in FP32 respectively. Both split services run as separate OS processes communicating via gRPC over `127.0.0.1:50051`.

The justification for screening at the coarse architectural level before conducting any finer search is that moving directly into an exhaustive layer-by-layer search would risk

over-fitting the selection to incidental local-execution noise. Prior DNN-partitioning systems commonly reason about split placement at layer or candidate-boundary granularity [15, 18], while later work motivates reducing the candidate set when exhaustive layer-wise search becomes impractical [18, 20, 33]. This thesis therefore uses stage boundaries for the first screening pass as a deliberate search-space reduction and only refines inside the carried-forward region in RQ1.2.

The output of this stage is one main carry-forward candidate and one reference candidate, selected using the degeneracy and near-best-window rules defined in Section 4.6.

4.7.2 RQ1.2 – Fine-Grained Refinement

RQ1.2 takes the carry-forward outcome of RQ1.1 as its starting point and introduces one block-level split point between the two coarse carry-forward anchors. The new condition, `split_after_layer3_block0`, bisects the `layer3` stage at its internal residual-block boundary.

A key structural property motivates this particular design choice. `split_after_layer3_block0` and `split_after_layer3` both produce intermediate activation tensors of approximately 196 KiB. The design therefore holds activation-transfer size constant while varying compute placement, creating a controlled comparison that can isolate the role of compute distribution independently of communication burden. This comparison is grounded in the DNN-partitioning literature, which treats compute distribution as a first-class criterion alongside activation-transfer cost [18, 27].

The experimental setup and measurement protocol are identical to RQ1.1, ensuring that the two stages remain directly comparable.

4.7.3 RQ1.3 – Explanatory Synthesis

RQ1.3 is not a split benchmark. It is a hybrid analytical stage that explains the patterns observed in RQ1.1 and RQ1.2 using two structural properties: activation-transfer size and compute distribution. The explanatory analysis draws on three evidence sources: the split benchmark summaries from RQ1.1, the split benchmark summaries from RQ1.2, and a separately conducted internal timing experiment on the monolithic ResNet-18 model.

The internal timing experiment instruments the monolithic forward pass at the stage, block, and selected operation level. It uses the same single-threaded CPU-stabilised environment as the split benchmarks, with 10 rounds, 50 warmup iterations per round, and 200 measured iterations per round. Functional equivalence against the uninstrumented model is verified (`max_abs_diff = 0.0`), and instrumentation overhead is measured explicitly at approximately 0.71 ms (0.94% of total forward-pass time), confirming that profiling does not materially distort the timing distribution.

The justification for this supporting experiment is that split-benchmark boundary-crossing intervals reflect a composite of serialisation, transport, and remote-side computation. Disentangling those components – particularly distinguishing activation-transfer cost from compute-placement effects – requires a direct measurement of local per-block execution cost that the split benchmark alone cannot provide.

4.7.4 RQ1.4 – Multi-Microservice Kubernetes Chain

RQ1.4 moves from local process-based communication to real Kubernetes in-cluster services. The environment is a single-node `kind` cluster running on Ubuntu/Linux, with all service pods colocated on the same node. This colocation rule is applied deliberately so that inter-service gRPC traffic remains in-cluster rather than traversing an external network, meaning that any observed overhead reflects orchestration and pod-to-pod communication rather than wide-area latency.

The stage benchmarks a predefined family of five conditions from `monolithic_k8s_1svc` to `chain_5svc`, corresponding to one through five synchronously chained ResNet-18 service segments. The chain family is predefined rather than selected by carry-forward because the purpose of this stage is not to choose among candidates but to characterise the full cost curve as a function of service count. The split points used are: `layer2` for `chain_2svc`; `layer1`, `layer3` for `chain_3svc`; `layer1`, `layer2`, `layer3` for `chain_4svc`; and `layer1`, `layer2`, `layer3`, `layer4` for `chain_5svc`. This family deliberately spans the full range of RQ1.1 activation sizes from 784 KiB (`layer1`) down to 98 KiB (`layer4`), and consequently re-introduces boundaries that RQ1.1 rejected (`layer1` as too transfer-heavy and `layer4` as compute-degenerate). The resulting cost curve should therefore be read as an *upper-bound envelope* on the cost of adding service boundaries – a purely best-boundary chain at each depth would lie below this envelope – not as a generic per-hop cost. Service pods are allocated one CPU core and 512 MiB of memory with Guaranteed QoS to fix the resource envelope and reduce scheduler and eviction variability during measurement windows.

The use of a single-node `kind` cluster is an acknowledged limitation: it eliminates inter-node network hops and does not capture the scheduling and networking variability of a multi-node or managed cluster. This limitation is addressed in three stages: RQ1.4b adds a `kind` CNI-bridge sensitivity probe (still single-host, so it does not measure real inter-node networking), RQ1.5 re-runs the same chain family on managed AKS to validate ordering, and RQ1.5b adds a genuine multi-node sensitivity stage on a six-VM AKS cluster so the real inter-node penalty becomes a measured, bounded quantity.

4.7.5 RQ1.4b – kind CNI Sensitivity (appendix-only)

RQ1.4b is a supporting sensitivity stage that is reported as appendix-level evidence only, not as a headline number.

The stage re-runs the same five-condition chain family from RQ1.4 on a six-worker `kind` cluster with pod anti-affinity enforced so that every chain segment of a given condition lands on a distinct `kind` worker node and the benchmark client lands on a sixth dedicated worker. The shared benchmark contract is otherwise unchanged (same model, same Guaranteed-QoS pod profile, same five-round / 200-measured-iteration / 50-warmup discipline, same image tag).

A methodological caveat governs how this stage should be interpreted. `kind`'s "nodes" are Docker containers that share a kernel on a single host machine and communicate over `kind`'s CNI bridge; they are not separate physical or virtual hosts. RQ1.4b therefore measures the cost of routing inter-service gRPC through the `kind` CNI bridge under anti-affinity, rather than the cost of real cross-host networking. The genuine multi-

host inter-node penalty is measured separately in RQ1.5b on a multi-node AKS cluster (section 4.7.7). Round-level cross-condition consistency (round CV) is also materially higher in RQ1.4b than in single-node RQ1.4, reflecting kind-CNI variability rather than a property of the chain family. Comparisons against RQ1.4 use the per-condition penalty $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$, which is read as a kind-CNI sensitivity bound rather than as a deployment-grade multi-node result.

A further cross-stage caveat must be made explicit before the per-condition delta Δ_c is interpreted. RQ1.4b is not a clean topology-only swap from RQ1.4: it also runs with a weaker host-control state. Writes to `/sys` fail in the containerised environment, so the CPU governor is observed at `schedutil` (rather than `performance`) and `turbo` is observed as enabled (rather than disabled); the `nice -5` request also fails for permission reasons. This is recorded in the `frozen merged_results/environment.json` for the RQ1.4b run and is one plausible co-contributor to the inflated RQ1.4b round CVs. For this reason, comparisons against RQ1.4 are interpreted at the *per-condition* Δ_c level only and are not lifted to a cross-stage means comparison; the two stages differ in host-control state in addition to placement, and Δ_c should be read as an upper sensitivity bound that combines the kind-CNI-bridge cost with this degraded host-control state on the same host.

4.7.6 RQ1.5 – AKS Transfer Validation

RQ1.5 re-runs the same predefined chain family from RQ1.4 on a single-node AKS cluster (`Standard_D8s_v3`, region `swedencentral`, `kubenet` networking). The stage does not aim to replicate the local Kubernetes absolute latency values, which are expected to differ because the two environments use different host CPUs, kernels, virtualisation layers, container runtimes, and networking paths.

The transfer claim is therefore directional rather than numerical: if the same architectural chain ordering is preserved in AKS, that constitutes evidence that the ordering reflects the architecture of the model partition rather than incidental local-cluster noise. This emphasis on within-environment comparison is consistent with benchmarking guidance that absolute performance numbers do not transfer across hardware and software stacks without careful environment characterisation [13]. Within-environment normalised overhead is used as the primary comparison metric for this reason.

All service pods and the benchmark client are colocated on the same AKS node and run with `Guaranteed` QoS, applying the same single-node placement discipline as RQ1.4. Pod resource requests differ between the two stages: RQ1.4 service pods request 1 CPU and 512 MiB of memory, whereas RQ1.5 service pods request 1 CPU and 1 GiB, matching the larger memory headroom available on `Standard_D8s_v3`. This difference is documented per artefact and does not affect placement or QoS class.

4.7.7 RQ1.5b – AKS Multi-Node Sensitivity

RQ1.5b is a sensitivity stage that quantifies the inter-node network cost the single-node RQ1.5 design intentionally hides. The stage re-runs the same predefined chain family on a six-node AKS cluster (`Standard_D8s_v3` $\times$ 6, region `swedencentral`, single node pool, single VNet) and enforces *multi-node* placement: each chain segment of a

given condition lands on a distinct cluster node, and the benchmark client lands on a sixth dedicated node. Placement is enforced by Kubernetes pod anti-affinity keyed on `kubernetes.io/hostname`, with two terms per service deployment: a `service-vs-service` term using the `experiment-condition` and `workload-role: service` labels, and a `service-vs-client` term using the `workload-role: client` label. The benchmark client carries a mirror anti-affinity rule against `workload-role: service`.

Compliance is verified after deployment by reading observed `nodeName` values from the Kubernetes API. A condition is admitted to the rolling merge only if its service pods are pairwise disjoint by node and the client node is distinct from every service node; a single collision aborts the run. The shared benchmark contract is otherwise unchanged from RQ1.5: same model, same chain family, same Guaranteed-QoS pod profile, same five-round / 200-iteration / 50-warmup discipline, and the same image used in the same-day RQ1.5 single-node run. The headline sensitivity metric is the per-condition multi-node penalty $\Delta_c = \text{mean}_c^{\text{multi}} - \text{mean}_c^{\text{single}}$, computed against the same-day frozen RQ1.5 single-node baseline so that the inter-node delta is not contaminated by region-time drift.

4.7.8 RQ2.1 – Service Identity and Mutual TLS Hardening

RQ2.1 introduces a paired execution design. Each paired pass executes both the plain AKS condition and the mTLS/AuthZ condition in alternating order, with the order determined by a seeded plan in which three passes run mTLS first and two passes run plain first. This alternation reduces the risk that the observed delta is explained by slow performance drift over the course of the run rather than by the hardening mechanism itself.

The plain condition uses standard Kubernetes service-to-service communication without mesh injection. The mTLS condition adds one Istio sidecar proxy per inference service, service accounts, workload-level peer-authentication objects, and authorisation-policy objects. These mechanisms correspond to the common Istio pattern of using sidecar proxies for service-to-service mTLS and policy enforcement in Kubernetes microservices [4]. The benchmark client remains outside the mesh; the stage therefore measures hardening of the inter-service path inside the inference chain, not external ingress hardening.

The managed AKS Istio add-on (`asm-1-29`) is used as the mesh implementation. This choice reflects the thesis’s focus on realistic cloud-native deployment: managed add-ons represent the configuration path most likely to be used by practitioners adopting mTLS in AKS and their behaviour reflects actual cloud-operator defaults rather than custom tuning.

Security validation is performed for each paired run using four methods: static manifest validation of peer-authentication and authorisation-policy objects; runtime validation of policy object counts; a full-chain positive probe confirming that valid upstream-to- downstream requests succeed; and a direct non-mesh denial probe confirming that unauthenticated access to protected downstream services is blocked.

The primary topology is `chain_2svc` (split after `layer2`). It is the lowest-overhead non-monolithic chain in the preceding RQ1.4 / RQ1.5 stages. The split point used is `layer2`, which corresponds to the RQ1.1 *reference* carry-forward candidate rather than the main carry-forward (`layer3`); this is a deliberate choice because `chain_2svc` in the

RQ1.4 / RQ1.5 chain family is built on layer2, so reusing the same boundary preserves continuity with the already-frozen chain-cost numbers.

A `chain_5svc` stress topology is included to determine how mTLS overhead grows when more service boundaries are protected. `chain_5svc` reuses the same RQ1.4 / RQ1.5 split family (layer1, layer2, layer3, layer4), which deliberately includes the RQ1.1-rejected boundaries to span the full activation range. The `chain_5svc` overhead figure should therefore be read against the same upper-bound-envelope caveat introduced in section 4.7.4.

A known limitation of sidecar-based mTLS is that the measured protection boundary is the proxy-mediated inter-service data path, not all data handling inside the pod or application process. MeshInsight shows that sidecar meshes add app-to-sidecar and sidecar-to-sidecar data-path segments whose latency and resource costs depend on proxy configuration and workload details [36]. A second limitation is that the mesh control plane and certificate authority remain trust-critical components even when the data-plane is fully mTLS-protected [26]. Both limitations are acknowledged explicitly: RQ2.1 is claimed as a communication-level hardening layer, not as a complete runtime-secrecy mechanism.

4.7.9 RQ2.1 (Ablation) – Decomposition of the Hardening Bundle

RQ2.1's main paired design treats the mTLS configuration as a single bundle: sidecar injection, strict peer-authentication, and authorisation-policy enforcement are all applied together. An additional ablation campaign is run on `chain_2svc` to decompose this bundle into its incremental contributions.

The ablation introduces four conditions on the same AKS cluster, same Istio revision (asm-1-29), and same strict same-node placement as the primary RQ2.1 design: (c0) plain non-mesh; (c1) mesh-default sidecar with automatic mTLS but no AuthorizationPolicy; (c2) strict PeerAuthentication mTLS with no AuthorizationPolicy; and (c3) strict mTLS with AuthorizationPolicy. The conditions are run as a seeded interleaved campaign with five passes per condition (5 passes $\times$ 4 conditions $\times$ 200 measured iterations = 4,000 measured rows).

The adjacent deltas decompose the hardened path as follows. $c3 - c0$ is the *full hardened-path cost* and $c3 - c2$ is the *clean AuthorizationPolicy cost*; both are cleanly attributable. $c1 - c0$ and $c2 - c1$ require a methodological caveat: Istio's mesh-default behaviour is automatic mTLS in PERMISSIVE mode, so c1 already encrypts sidecar-to-sidecar traffic. The delta $c1 - c0$ therefore measures the combined cost of *sidecar process + IPC + auto-mTLS data plane* over plain non-mesh, and cannot be attributed to "sidecar process overhead" alone. The delta $c2 - c1$ measures the cost of upgrading the policy from PERMISSIVE to STRICT with auto-mTLS already in effect, not the cost of "strict mTLS encryption" relative to plaintext sidecar traffic. A clean separation of sidecar process cost from mTLS cryptography cost would require a fifth condition with the sidecar present but `PeerAuthentication: DISABLE`; this thesis does not include that condition and treats the disentanglement as future work.

This campaign is treated as supporting, not headline. The ablation deltas are internally comparable across c0–c3 within the same campaign run, but the absolute numbers

must not be spliced into the frozen two-condition paired `chain_2svc` and `chain_5svc` results.

4.7.10 RQ2.1 (Split Sensitivity) – Single-Hop mTLS vs Activation Size

A second supporting campaign measures whether the mTLS overhead at a single protected hop scales with the size of the activation crossing that hop. The independent variable in this campaign is the split point that produces hop 2's activation, varying it across `layer1`, `layer2`, `layer3`, and `layer4` (activation sizes 802 816, 401 408, 200 704, and 100 352 bytes respectively, matching the RQ1.1 sizes).

For each split point, a plain condition and an mTLS condition are run as a paired pair, yielding eight conditions in total. Only the hop between service 1 and service 2 is protected by the mTLS configuration; the same strict same-node placement, Istio revision, pod profile, and image digest as the primary RQ2.1 design are otherwise preserved. The campaign runs five seeded-interleaved passes across the eight conditions for a total of 8,000 measured rows. The per-split deltas (mean and p95 mTLS overhead, per-split operational overhead, and per-split resource summary) are the headline outputs.

This campaign is treated as supporting, not headline, for two reasons. First, the `split_after_layer4` condition remains compute-degenerate per the RQ1.1 degeneracy rule and is retained only as a low-payload diagnostic endpoint, not as a candidate deployment. Second, the per-split mTLS deltas are campaign-internal: they may not be spliced into the frozen `chain_2svc` or `chain_5svc` paired numbers.

4.7.11 RQ2.1b – Multi-Node Sensitivity of mTLS Overhead

RQ2.1b is a paired sensitivity stage that quantifies how the `chain_2svc` mTLS overhead changes when the two inference services are forced onto distinct AKS nodes. It mirrors the RQ1.5b design at the security layer: the `chain_2svc` plain and mTLS conditions are deployed on a dedicated AKS node pool with `multi_node_anti_affinity` placement and a dedicated benchmark-client node. The same paired alternation, five-pass budget (5 paired passes $\times$ 2 conditions $\times$ 200 measured iterations = 2,000 rows), seeded order, and security-validation contract as the primary RQ2.1 `chain2` run are reused.

Compliance with the multi-node placement contract is verified after deployment by reading observed `nodeName` values from the Kubernetes API and rejecting any pass where the two service pods share a node or where the client node coincides with either service node. The headline output is the multi-node paired mTLS delta (mean, p95, and per-pass spread) together with the placement-validation status of every pass.

4.7.12 RQ2.2 – Confidential VM Execution Hardening

RQ2.2 keeps the communication-security contract from RQ2.1 fixed and changes the execution environment of the downstream protected service. The confidential condition places `service2` on an AMD SEV-SNP confidential node pool (`Standard_DC8as_v5`); the standard condition uses matched non-confidential AMD nodes (`Standard_D8as_v5`) in the same AKS cluster.

The RQ2.2 cluster is deployed in Azure region westeurope rather than the swedencentral region used by the RQ1.5, RQ1.5b, and RQ2.1 clusters, because at the time of the experiments the AMD SEV-SNP DC-series SKUs (Standard_DC8as_v5) were the constraining resource and were available with the required quota in westeurope. This region change is the reason a fresh paired standard baseline is collected on the RQ2.2 cluster rather than reusing the RQ1.5 or RQ2.1 mTLS baseline: absolute latency between the two regions is not directly comparable, so only the within-cluster paired delta between the standard and confidential conditions is reported as the incremental TEE overhead. This paired design reduces region- and session-level confounding, although the delta still reflects the practical platform change from the standard AMD node pool to the confidential AMD SEV-SNP node pool.

The thesis-facing TEE scope is `service2_only`. This selective placement isolates the cost of protecting the downstream service that receives intermediate activations, motivated by activation-privacy work [12] showing that those activations are recoverable when held by an untrusted remote partition. Extending the confidential-execution boundary to both services (full two-service TEE) is out of the thesis-facing scope of RQ2.2 per the evidence manifest and is treated as future work rather than measured here.

The protection boundary of AMD SEV-SNP must be stated precisely. SEV-SNP provides VM-level memory encryption and integrity protection against an untrusted host platform [3, 17], but the guest operating system, application process, PyTorch runtime, model weights, and Istio sidecar all remain inside the same VM trust boundary. Recent work additionally shows residual attack surfaces beyond memory encryption, including malicious-interrupt injection from the untrusted hypervisor [29]; this thesis does not experimentally evaluate such attacks. The motivation for protecting the downstream segment at infrastructure level is supplied by activation-privacy work [12] showing that intermediate feature maps can be inverted to recover client inputs, although that threat model differs from the host-level adversary SEV-SNP defends against. RQ2.2 therefore uses the term *VM-level confidential execution* throughout rather than claiming application-level enclave isolation.

4.7.13 RQ2.3 – Security-Hardening Trade-off Synthesis

RQ2.3 is a synthesis stage: no new benchmark data is collected. The stage combines the plain AKS baseline from RQ1.5, the mTLS/AuthZ results from RQ2.1, and the selective `service2_only` TEE result from RQ2.2 to compare the three thesis-facing hardening configurations across measured cost, operational complexity, and protection scope. Full two-service TEE is treated as future work rather than as a measured trade-off point.

The synthesis is structured around the principle that the preferred hardening configuration is threat-model-dependent rather than universally optimal. Applying every available security mechanism to every service boundary is not always the best engineering choice, because each additional protection layer adds measurable latency, resource, and operational cost. RQ2.3 therefore frames its output as a structured trade-off comparison rather than a single global ranking, allowing a practitioner to match a security configuration to a stated threat model and performance budget.

4.8 Artifact Freezing and Reproducibility

Thesis-facing runs are exported and frozen as artifact sets before analysis is performed. This means that the reported results are derived from a fixed, immutable snapshot of the raw measurement data rather than from ad hoc terminal output or re-runnable live scripts. Each frozen artifact set records the full execution environment: operating system, Python and PyTorch version, Kubernetes version and cluster name, node type, namespace, container image reference, condition ordering, and parity validation outcomes. For the AKS stages (RQ1.5, RQ1.5b, RQ2.1, RQ2.1b, RQ2.2) the image reference is a pinned sha256 digest from the project's Azure Container Registry. For the local kind stages (RQ1.4, RQ1.4b) the image reference is a mutable tag (`thesis-inference:latest`) preloaded into the cluster with `IfNotPresent`; reproduction in those stages therefore depends on the locally built image alongside the recorded runtime metadata.

The separation between the frozen artifact and the analysis code ensures that the reported numbers remain stable even if benchmark scripts are modified for subsequent exploratory runs. The repository contains both thesis-facing configs and older exploratory or smoke-test configs; the artifact freeze identifies which run is the thesis-facing primary result.

4.9 Limitations and Threats to Validity

The methodological scope of this study has known limitations covering five themes, each of which constrains how the results may be generalised.

Single-node deployment. The primary RQ1.4 and RQ1.5 designs colocate all chain services on one node so that the measured cost is attributable to orchestration and gRPC rather than to wide-area or inter-node networking. RQ1.4b and RQ1.5b are explicit multi-node sensitivity stages that bound the cost the single-node design hides; the RQ1.5/RQ2.1 primary numbers should not be read as representative of arbitrary multi-node deployments.

CPU-only, single-model, single-workload scope. All experiments use ResNet-18 in FP32 on CPU at batch size one. Results may not transfer quantitatively to GPU inference, mixed precision, larger or smaller models, transformer architectures, or batched throughput-oriented workloads, although the staged methodology itself is model-agnostic.

Controlled-versus-production variability. Strict CPU stabilisation is enforceable on the local stages but only partially enforceable on AKS (governor and turbo are not exposed; nice is denied). The measurement environment is therefore quieter than a typical production AKS workload, and absolute numbers may shift under contended noisy-neighbour conditions.

Bounded RQ2.1 threat model. RQ2.1 measures communication-level hardening only. The sidecar design protects the proxy-mediated inter-service path, while data handling inside each pod and application process remains outside the measured mesh boundary; the mesh control plane plus CA also remain trust-critical. RQ2.1 must therefore be interpreted as a confidentiality and authenticity boundary on the inter-service path, not as a complete runtime-secrecy mechanism.

Bounded RQ2.2 threat model. RQ2.2 measures the incremental cost of AMD SEV-SNP at VM granularity. The guest OS, application, runtime, model weights, and sidecar remain inside the VM trust boundary, and residual attack surfaces such as malicious-interrupt injection from the untrusted hypervisor are out of experimental scope. RQ2.2 is therefore an infrastructure-level confidentiality control against a host-platform adversary, not an application-level enclave guarantee.

In addition to the five themes above, several measurement scopes are deliberately out of this thesis and should be named explicitly so that the reported numbers are not over-generalised. *Concurrency.* All measurements use batch size one and sequential request issuance; throughput under concurrent load is not measured, and the cost of mTLS, sidecar IPC, and SEV-SNP memory encryption may scale non-linearly when the sidecar or the CVM memory controller is contended. *Tail behaviour beyond p95.* With 1,000 measured iterations per condition the p95 estimate is well populated, but only ten samples sit beyond the p99 cut and the data do not support reliable claims about p99 or deeper tail percentiles. *Cold start and steady state.* Pod startup, sidecar bootstrap, CVM boot, and SEV-SNP attestation latency are not part of the measured end-to-end interval; the headline numbers describe steady-state behaviour only. *Cloud-dollar cost.* The thesis reports resource overhead in CPU-time and memory units rather than in Azure billing units; converting to dollar-equivalents at the SKU prices used (Standard_D8s_v3, Standard_D8as_v5, Standard_DC8as_v5) is left to the synthesis discussion in RQ2.3. *Sidecar capacity headroom.* The Istio sidecar limits are taken from the managed-mesh defaults; whether the sidecar approaches its CPU limit at the measured load is checked via the resource sampling captured in the RQ2.1 artefacts and is reported alongside the overhead numbers, but no load-stress condition was run to find the sidecar’s saturation point.

The consolidated discussion of these limitations – including their implications for the synthesis in RQ2.3 and for external validity – is in section 6.6.

4.10 Ethical Considerations

This thesis does not involve human participants, personal data, or biological material. The study is a software systems benchmark using a publicly available pretrained image classification model and standard synthetic inputs; no classification decisions derived from this model are used to affect any real-world outcomes, and no personal images or sensitive datasets are processed.

ResNet-18 with pretrained ImageNet weights inherits any biases or limitations of the ImageNet training process. However, since the thesis does not evaluate model accuracy and does not deploy the model in a decision-making context, these concerns are not directly relevant to the research questions addressed here.

Cloud compute resources are used responsibly: experiments are scoped to the minimum infrastructure required to answer each research question, and resources are deallocated after each benchmark stage. All measurement artifacts contain only timing data and system metadata; no sensitive data is stored on cloud infrastructure.

Experiments & Results

5.1 RQ1.1 Coarse Architectural Boundary Selection

5.1.1 Research Question

This subsection addresses the following sub-research question:

Which coarse architectural stage boundaries provide the most suitable two-part microservice decompositions of ResNet-18 under controlled local execution, and how do their latency and boundary-crossing costs compare to monolithic inference?

RQ1.1 is designed as a coarse-boundary screening stage rather than an exhaustive exploration of all possible split points. Its purpose is to identify architecturally meaningful two-part decompositions that can be compared against monolithic inference under controlled local execution, and to determine which coarse boundaries provide the strongest carry-forward basis for later refinement. In this stage, suitability is not treated as a question of raw latency alone. It is evaluated in relation to the joint trade-off between end-to-end latency, activation-transfer burden, boundary-crossing cost, and the structural balance of computation across the two resulting services.

The literature framing for coarse architectural boundary selection is discussed in sections 3.1 and 3.2.

5.1.2 Experimental Setup

The RQ1.1 benchmark used the fully controlled local CPU-only configuration in order to minimize variance and isolate the effect of the partition boundary itself. The source configuration file is `configs/rq1/1.1/rq1_1_fully_controlled.yaml`; an exact byte-for-byte snapshot is preserved alongside the frozen results as `config.yaml` in the artifact directory. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. The benchmark compared one monolithic baseline against four two-part split configurations, each corresponding to a coarse architectural stage boundary in ResNet-18.

Activation-tensor sizes in this section are quoted in binary kibibytes (KiB, 1024 bytes) for consistency with the gRPC message-size limit, which is also a binary quantity. Three further configuration files are present under `configs/rq1/1.1/` but do not contribute to the local-controlled RQ1.1 thesis-facing result reported here: `rq1_1_experimental.yaml` (a smoke test with minimal iterations, used only for pipeline validation), `rq1_1_threaded.yaml` (a four-core ancillary profile retained as a historical comparison point), and `rq1_1_azure_fully_controlled.yaml` (an AKS-backed coarse-split variant driven by `scripts/run_rq1_1_azure_fully_controlled.py`, retained for completeness but not part of the local single-host result reported in this section).

The benchmark was configured to use a single-threaded PyTorch execution model pinned to one physical CPU core with SMT avoided. Thread counts for PyTorch, OpenMP, MKL, and OpenBLAS were all fixed to one. In addition, process priority was increased, the CPU governor was set to performance, and turbo boost was disabled in order to reduce frequency-related variance. These controls were applied symmetrically across all conditions so that the comparison remained fair and reproducible.

Table 5.1: RQ1.1 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer1, layer2, layer3, layer4
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (127.0.0.1:50051)
Maximum message size	16 MiB (16777216 bytes)
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

5.1.3 Benchmark Design

The experiment followed a repeated-measures design with five conditions: a monolithic baseline and four split configurations. The split points were selected after the major architectural stages of ResNet-18, namely after layer1, layer2, layer3, and layer4. These boundaries are meaningful because they preserve the stage structure of the network while allowing the evaluation of progressively later partition points.

The benchmark protocol consisted of 5 rounds. For each round and condition, 50 warmup iterations were executed before 200 measured iterations were recorded. A 5-second cooldown separated successive conditions. The ordering of conditions was randomized using deterministic per-round seeds derived from the configured seed and round index to reduce ordering bias. Functional equivalence between monolithic and split executions was enforced as a mandatory precondition through both local and gRPC parity checks using five inputs and an absolute tolerance of 1.0×10^{-5} .

A warmup calibration procedure was used to monitor whether latency had stabilized. This procedure used a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02. The implementation could extend warmup adaptively beyond the configured minimum if the trailing window remained unstable; in this frozen run, all condition-round pairs stabilized within the configured 50 iterations, so no extra warmup iterations were used.

5.1.4 Partition Conditions

The four split conditions corresponded to the following coarse stage boundaries in ResNet-18:

- **split_after_layer1:** The first service executes the stem and layer1, and the second service executes layer2 through the classifier head. This split transfers the largest intermediate tensor, of exact shape $1 \times 64 \times 56 \times 56$, corresponding to 802,816 bytes (784 KiB) in FP32.
- **split_after_layer2:** The first service executes the stem and layer1--layer2, and the second service executes layer3 through the classifier head. The intermediate tensor is of exact shape $1 \times 128 \times 28 \times 28$, 401,408 bytes (392 KiB).
- **split_after_layer3:** The first service executes the stem and layer1--layer3, and the second service executes layer4 plus pooling and classification. The intermediate tensor is of exact shape $1 \times 256 \times 14 \times 14$, 200,704 bytes (196 KiB).
- **split_after_layer4:** The first service executes the stem and layer1--layer4, and the second service executes only average pooling and the final linear layer. The intermediate tensor is of exact shape $1 \times 512 \times 7 \times 7$, 100,352 bytes (98 KiB).

These stage boundaries provide a coarse sweep across the network from earlier to later partition points. This design is consistent with partitioning research that treats split placement as a search-space and granularity decision shaped by computation, communication, and deployment constraints, rather than as an unconstrained exhaustive sweep over every internal operation [18, 20, 33].

Carry-forward rule. The decision of which coarse boundary to promote to the next stage is governed by a predeclared two-part rule applied to the per-condition mean end-to-end latencies and the per-service compute measurements (see section 4.6 for the full statement). In short: (i) the *near-best window* comprises every split whose mean latency lies within 5% of the raw-fastest split; (ii) a split is treated as *compute-degenerate* when the smaller of the two per-service mean compute times contributes less than 10% of the total split compute, i.e. $\min(A, B)/(A + B) < 0.10$ for service-A compute A and service-B compute B . The main carry-forward candidate is the non-degenerate split with the lowest mean latency in the near-best window; the next-best non-degenerate split is retained as the reference candidate. The frozen rule outputs for this run are recorded in `carry_forward.json`.

Boundary-crossing interval. The *boundary-crossing interval* reported for each split condition is the client-observed interval that begins immediately before the intermediate-tensor serialisation and ends immediately after the response deserialisation. It is a composite of activation serialisation, gRPC transport, the remote service-B compute, and response deserialisation, and is implemented by the timer block in `src/client/split_client.py` (see section 4.3 for the full measurement contract). It is not a pure “wire” cost.

5.1.5 Results

The thesis-facing RQ1.1 benchmark used the frozen artifact `rq1_1_20260515_111428` (source: `results/frozen_new/rq1_1_20260515_111428/condition_summaries.csv`). Monolithic execution achieved the lowest mean end-to-end latency at 77.24 ms. The four split conditions yielded mean latencies of 80.88 ms for `split_after_layer1`, 80.12 ms for `split_after_layer2`, 79.08 ms for `split_after_layer3`, and 79.58 ms

for `split_after_layer4`. This corresponds to relative overheads of approximately 4.7%, 3.7%, 2.4%, and 3.0%, respectively, compared to monolithic execution.

Table 5.2: RQ1.1 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, and overhead vs. monolithic. Source: `condition_summaries.csv` in `rq1_1_20260515_111428`. Confidence intervals are reported from the same artifact’s `ci95_lower_ms/ci95_upper_ms` columns and summarise per-iteration variation within this run.

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Within-run 95% CI (ms)	Overhead vs. Monolithic
Monolithic	77.24	75.29	3.59	[77.01, 77.46]	—
Split after layer1	80.88	80.24	2.27	[80.74, 81.02]	4.7%
Split after layer2	80.12	79.60	2.40	[79.97, 80.27]	3.7%
Split after layer3	79.08	78.38	2.31	[78.94, 79.23]	2.4%
Split after layer4	79.58	78.85	2.43	[79.43, 79.73]	3.0%

Figure 5.1 visualises the latency pattern reported in table 5.2. In this frozen run the raw-fastest split is the mid-to-late boundary after layer3, not the latest boundary, and the carry-forward decision still depends on both latency and structural compute balance rather than latency alone.

All split configurations therefore introduced positive latency overhead relative to monolithic inference. However, the magnitude and character of this overhead varied substantially with split location. The earliest split, after layer1, performed worst because it crossed the largest intermediate activation and exhibited the highest boundary-crossing cost. The latest split, after layer4, minimized activation-transfer size but left only a minimal residual computation segment on the second service (well under 1 ms of remote compute according to the per-service compute means in `condition_summaries.csv`), making it structurally compute-degenerate despite its low transfer burden. The mid-to-late splits, after layer2 and layer3, formed the strongest non-degenerate candidates in the coarse sweep, with `split_after_layer3` also yielding the lowest raw split mean in this frozen run.

The boundary-crossing measurements further support this interpretation. The split after layer1 incurred the largest boundary-crossing interval at 53.81 ms, followed by layer2 at 40.85 ms and layer3 at 26.06 ms, while layer4 was much smaller at 2.22 ms. This descending pattern is consistent with the joint reduction in activation-transfer burden and downstream service-B compute as the boundary moves later, from 784 KiB at layer1 to 98 KiB at layer4. At the same time, the late layer4 split shows that minimizing transfer burden alone does not determine overall suitability as a microservice decomposition, because the downstream service becomes too small to represent a meaningful balanced distribution.

Cross-round consistency was strong across all configurations, indicating stable execution behavior. Per `round_consistency.json` for the same artifact, the monolithic condition had a round coefficient of variation (CV) of 0.002, while `split_after_layer1`, `split_after_layer2`, `split_after_layer3`, and `split_after_layer4` had round CVs of 0.005, 0.011, 0.003, and 0.005, respectively.

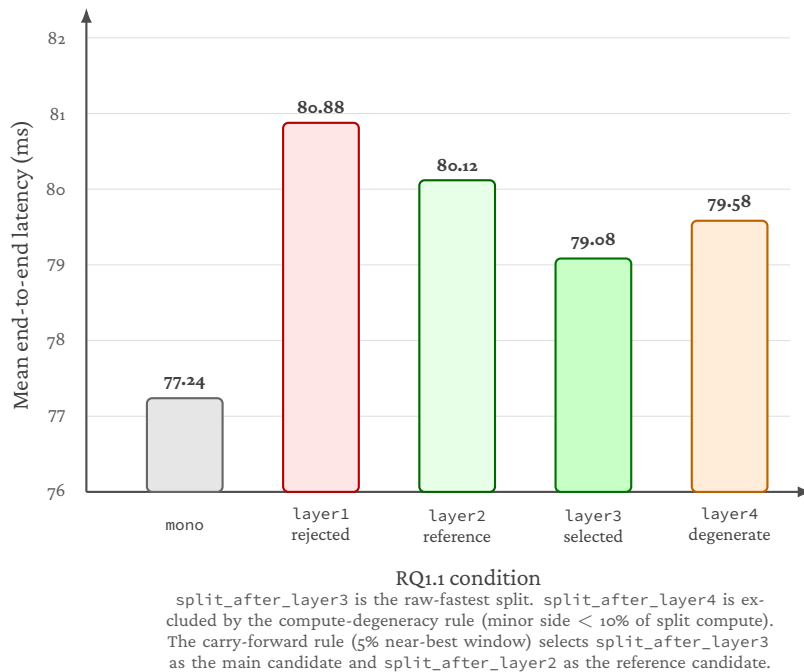


Figure 5.1: RQ1.1 coarse split latency results under controlled local execution. Bar heights are the mean end-to-end latency per condition for the frozen run rq1_1_20260515_111428; bar colours encode the role assigned by the carry-forward rule (baseline, selected_main, selected_reference, rejected, degenerate). split_after_layer3 is the raw-fastest split in this run and is also non-degenerate; split_after_layer4 is excluded by the compute-degeneracy rule despite its small intermediate activation. The carry-forward decision (see carry_forward.json for the same run) selects split_after_layer3 as the main candidate and split_after_layer2 as the reference candidate. The figure is generated deterministically from the frozen condition_summaries.csv and carry_forward.json by scripts/render_rq11_figure.py.

Scope caveat. The numbers reported in this section are derived from a single frozen artifact (rq1_1_20260515_111428) generated by a single benchmark run with the fixed seed 42. The five rounds within that run randomise condition order but do not vary the input seed, the host, or the build, and cross-host or cross-seed replication is not in scope for RQ1.1. The mean-latency differences between the non-degenerate split conditions are small relative to their per-iteration standard deviation. Although the descriptive within-run per-iteration 95% confidence intervals in table 5.2 do not overlap between `split_after_layer3` and `split_after_layer2`, those intervals should not be read as host-independent uncertainty estimates. The absolute ranking within the non-degenerate set should therefore be read as “the strongest carry-forward candidate observed in this frozen run” rather than as a general ordering. The carry-forward decision is stated in terms of the predeclared rule (section 4.6) applied to this artifact, and the ranking is carried forward for refinement in RQ1.2 rather than treated as a final answer.

The interpretation of these results and the answer to RQ1.1 are presented in section 6.1.1.

5.2 RQ1.2 Fine-Grained Refinement

5.2.1 Research Question

This subsection addresses the following sub-research question:

How does finer-grained refinement within the late-stage coarse region carried forward from RQ1.1 affect latency and communication-related overhead for two-part microservice inference?

RQ1.2 is designed as a targeted refinement stage, not as a new blind search across the network. It takes the carry-forward outcome of RQ1.1 as its starting point. In RQ1.1, the coarse architectural boundary screening identified the mid-to-late region around layer2 and layer3 as the strongest balanced carry-forward zone, with `split_after_layer3` selected as the main candidate and `split_after_layer2` retained as the reference candidate. RQ1.2 therefore operates within that carried-forward comparison space and introduces one meaningful finer-grained split point between the two coarse anchors. The purpose is to determine whether a block-level boundary within this region reveals latency or communication-related differences that were not visible at the coarse stage level.

The literature framing for finer-grained partition refinement is discussed in section 3.2.

5.2.2 Experimental Setup

The RQ1.2 benchmark reused the same fully controlled local CPU-only configuration as RQ1.1 in order to ensure that the two stages remain directly comparable. The evaluated model was ResNet-18 with pretrained weights, executed on CPU in FP32 precision with an input shape of $1 \times 3 \times 224 \times 224$. Communication between the two split services used gRPC over localhost (127.0.0.1:50051) with a maximum message size of 16 MiB (16,777,216 bytes).

As in RQ1.1, activation-transfer sizes in this section are reported in binary kibibytes (KiB, 1024 bytes).

All CPU stabilisation controls from RQ1.1 were applied identically: single-threaded PyTorch execution pinned to one physical core with SMT avoided, thread counts for PyTorch, OpenMP, MKL, and OpenBLAS all fixed to one, process priority elevated, the CPU governor set to performance, and turbo boost disabled. These controls were applied symmetrically across all conditions so that environmental variation was reduced as a competing explanation for observed boundary-placement differences.

5.2.3 Benchmark Design

The experiment followed the same repeated-measures protocol as RQ1.1: 5 rounds of 200 measured iterations per condition, preceded by 50 warmup iterations and separated by 5-second cooldowns. Condition ordering was randomised within each round using a fixed seed. Functional equivalence between monolithic and split executions was enforced through both local and gRPC parity checks using five inputs and an absolute tolerance of

Table 5.3: RQ1.2 benchmark configuration

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Conditions	Monolithic + split after layer2, layer3.0, layer3
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC over localhost (127.0.0.1:50051)
Maximum message size	16 MiB (16,777,216 bytes)
Parity tolerance	1.0×10^{-5}
Parity inputs	5
Warmup window	10 iterations
Warmup CV threshold	0.02

1.0×10^{-5} . A warmup calibration procedure using a trailing window of 10 iterations and a coefficient-of-variation threshold of 0.02 was applied to verify that latency had stabilised before measurement.

The key difference from RQ1.1 is the composition of the condition set. Rather than sweeping across all coarse stage boundaries, RQ1.2 evaluates a targeted set of three refined split conditions anchored by the RQ1.1 carry-forward candidates, with one new block-level split point inserted between them [27].

5.2.4 Refined Partition Conditions

The RQ1.2 condition set consists of one monolithic baseline and three split configurations:

- **split_after_layer2:** The first service executes the stem and layer1–layer2, and the second service executes layer3 through the classifier head. This is the RQ1.1 reference carry-forward candidate. The intermediate activation tensor is of exact shape $1 \times 128 \times 28 \times 28$, 401,408 bytes (392 KiB) in FP32.
- **split_after_layer3_block0:** The first service executes the stem and layer1–layer3.0 (i.e., the first residual block of layer3), and the second service executes from layer3.1 onward through the classifier head. This is the single new finer-grained split point introduced in RQ1.2. It bisects the coarse layer3 stage at its internal block boundary. The intermediate activation tensor is of exact shape $1 \times 256 \times 14 \times 14$, 200,704 bytes (196 KiB) in FP32 — the same size as the tensor produced at the end of the full layer3 stage.
- **split_after_layer3:** The first service executes the stem and layer1–layer3, and the second service executes layer4 plus pooling and classification. This is the

Table 5.4: RQ1.2 summary of mean latency, dispersion, descriptive within-run per-iteration 95% confidence interval on the mean, overhead, and activation-transfer burden. Source files: `condition_summaries.csv`, `cross_condition.csv`, and `round_consistency.json` in the frozen artifact `rq1_2_20260515_112636`. Confidence intervals are reported from the `ci95_lower_ms` / `ci95_upper_ms` columns of `condition_summaries.csv` and summarise per-iteration variation within this run.

Condition	Mean (ms)	Median (ms)	Std. Dev. (ms)	Within-run 95% CI (ms)	Overhead
Monolithic	77.565	75.641	3.663	[77.338, 77.792]	
Split after layer2	80.348	79.570	2.510	[80.192, 80.504]	3.3%
Split after layer3_block0	80.157	79.379	2.564	[79.998, 80.316]	3.6%
Split after layer3	80.392	79.687	2.475	[80.238, 80.545]	3.3%

RQ1.1 main carry-forward candidate. The intermediate activation tensor is of exact shape $1 \times 256 \times 14 \times 14$, 200,704 bytes (196 KiB) in FP32.

The monolithic baseline is retained for reference so that all overhead values remain anchored to the same absolute comparison. Together, these four conditions form a targeted refinement set rather than an exhaustive within-stage search: `split_after_layer2` and `split_after_layer3` anchor the carried-forward coarse space, while `split_after_layer3_block0` probes the single meaningful block-level internal boundary between them.

An important structural observation motivates this design. The coarse boundary after `layer3` and the block-level boundary after `layer3.0` produce intermediate activation tensors of the same exact size (both 200,704 B / 196 KiB). This creates a controlled comparison in which the activation-transfer burden is held constant while the distribution of computation across the two services differs. If the two conditions yield different end-to-end latencies or boundary-crossing intervals, the comparison points to compute placement as a plausible explanatory factor rather than to activation size alone [20]; it does not by itself establish compute placement as the only source of the difference.

5.2.5 Results

The thesis-facing RQ1.2 benchmark used the tracked frozen artifact `rq1_2_20260515_112636`. Mean and table values use `condition_summaries.csv`; boundary and cross-round values use `cross_condition.csv` and `round_consistency.json` in the same artifact. The measured results are summarised in table 5.4. Monolithic execution achieved the lowest mean end-to-end latency at 77.565 ms. Among the three refined split conditions, `split_after_layer3_block0` achieved the lowest mean split latency at 80.157 ms, followed by `split_after_layer2` at 80.348 ms and `split_after_layer3` at 80.392 ms. This corresponds to overhead relative to monolithic execution of approximately 3.3%, 3.6%, and 3.6%, respectively. The three split means are tightly clustered within a 0.235 ms band in this run, so this ordering should be read as a narrow within-run ordering rather than as a materially separated ranking.

The boundary-crossing interval measurements reveal a notable difference between the two conditions that share the same activation size. `split_after_layer3_block0`

exhibited a boundary-crossing interval of 33.97 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 26.16 ms. Despite producing activation tensors of the same exact size (200,704 B / 196 KiB), the two conditions differ by approximately 7.8 ms in boundary-crossing cost. As restated from the measurement contract in section 4.3, the boundary-crossing interval is a composite that includes the remote service-B compute alongside activation serialisation, gRPC transport, and response deserialisation. In this run, the per-service compute means in `cross_condition.csv` show service-B compute of 31.87 ms for `split_after_layer3_block0` and 24.06 ms for `split_after_layer3` (a 7.81 ms gap), while the non-compute residual is approximately 2.1 ms for both conditions. The 7.8 ms boundary-crossing difference is therefore dominated by the service-B compute component rather than by a transport or serialisation difference, which is consistent with treating compute placement (not activation size or wire cost) as the proximate explanatory factor for this pair. The earlier split (`layer2`) had the largest boundary-crossing interval at 40.86 ms, consistent with its larger activation tensor of 392 KiB.

Within-run cross-round consistency was strong across all conditions, per `round_consistency.json` in the same artifact. The monolithic condition had a round coefficient of variation (CV) of 0.0017, while the split conditions had round CVs of 0.0046 (`split_after_layer2`), 0.0033 (`split_after_layer3`), and 0.0070 (`split_after_layer3_block0`). The reported means can therefore be treated as representative of this frozen run rather than as artefacts of a single favourable round. This remains a single-run local result with fixed seed, host, and build; cross-seed or cross-host replication is outside the scope of RQ1.2.

The interpretation of these results and the answer to RQ1.2 are presented in section 6.1.2.

5.3 RQ1.3 Explanatory Synthesis: Activation-Transfer Size and Compute Distribution

5.3.1 Research Question

This subsection addresses the following sub-research question:

How do activation-transfer size and compute distribution explain the latency and boundary-crossing behavior observed for the decomposition choices evaluated in RQ1.1 and RQ1.2?

RQ1.3 is not an additional split benchmark. It is an explanatory synthesis stage that interprets the local findings already reported in RQ1.1 and RQ1.2 using two structural properties of the partition: the size of the intermediate activation tensor transferred at the boundary, and the distribution of computation across the two resulting services. To support this interpretation, RQ1.3 draws on the split benchmark measurements from RQ1.1 and RQ1.2 as primary evidence and uses an independently conducted internal timing experiment on the monolithic ResNet-18 model as supporting evidence for the compute-distribution explanation.

The literature framing for activation-transfer size, compute distribution, and the ResNet-18 stage structure used as the basis for this synthesis is discussed in sections 3.1 and 3.2.

5.3.2 Analytical Setup

RQ1.3 draws on three bodies of evidence:

1. **Split benchmark measurements from RQ1.1.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the five coarse conditions (monolithic plus four stage-level splits). The relevant data are summarised in table 5.2 and the overhead analysis in section 5.1.5.
2. **Split benchmark measurements from RQ1.2.** These include the mean end-to-end latencies, boundary-crossing intervals, and activation-transfer sizes for the four refined conditions (monolithic plus three refined splits within the carried-forward region). The relevant data are summarised in table 5.4 and the overhead analysis in section 5.2.5.
3. **Internal timing experiment on monolithic ResNet-18.** A separate instrumented forward-pass profiling experiment was conducted on the same monolithic model under a closely matched single-threaded CPU-stabilised environment, frozen as `internal_timing_resnet18_20260515_113614`. This experiment measured per-stage, per-block, and selected internal operation timings for a single-threaded CPU forward pass, averaged over 200 measured iterations preceded by 50 warmup iterations, using 10 rounds and the same randomisation seed. Functional equivalence was verified (maximum absolute difference of 0.0 against the uninstrumented model in `validation.json`), and instrumentation overhead was measured at 0.708 ms (0.94% of total forward-pass time, per `instrumentation_overhead.json`), confirming that the profiling did not meaningfully distort the timing distribution.

Table 5.5: RQ1.3 supporting internal timing configuration. Source: config.yaml and instrumentation_overhead.json in internal_timing_resnet18_20260515_113614.

Setting	Value
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Profiling mode	Monolithic internal timing (supporting evidence)
Timing scope	Stage-, block-, and selected operation-level timing
Rounds	10
Warmup iterations	50
Measured iterations	200
Validation inputs	5
Validation tolerance	1.0×10^{-6}
CPU stabilisation	Single-threaded, single-core pinned execution
Instrumentation overhead	0.708 ms (0.94%)

This experiment is used as supporting evidence to characterise the internal compute distribution of ResNet-18; it is not a split benchmark and does not introduce new split conditions.

No new split benchmark was conducted for RQ1.3. The explanatory analysis is based entirely on measurements already reported in RQ1.1 and RQ1.2, supplemented by the internal timing experiment described above. To keep this supporting evidence auditable without turning RQ1.3 into a separate benchmark stream, tables 5.5 to 5.7 and fig. 5.2 provide a compact summary of the internal timing configuration and the specific internal timing results used in the explanatory analysis.

5.3.3 Explanatory Findings

The findings are organised around four explanatory observations, each building on the previous one.

A. Activation-transfer burden explains much of the coarse latency pattern. In RQ1.1, the boundary-crossing interval decreased monotonically from the earliest split to the latest: 53.81 ms after layer1 (784 KiB activation), 40.85 ms after layer2 (392 KiB), 26.06 ms after layer3 (196 KiB), and 2.22 ms after layer4 (98 KiB). This pattern closely tracks the reduction in activation-transfer size as the split point moves deeper into the network. Larger intermediate tensors require more serialisation, transmission, and deserialisation time over the gRPC boundary, and this cost is directly reflected in the boundary-crossing measurements.

This observation is consistent with the broader partitioning literature, which identifies activation size as a primary driver of communication overhead in distributed inference [7, 15, 30, 31, 33]. At the coarse level, the monotonic relationship between activation

Table 5.6: RQ1.3 supporting internal timing summary used in the explanatory analysis. Source: `summary.csv` in `internal_timing_resnet18_20260515_113614`. Classification head is the sum of `avgpool` and `fc`. The convolution and batch-norm rows are aggregated over all matching units at level L3 in the same summary file.

Unit / Metric	Mean (ms)	% of model
Model total	75.377	100.00%
Stem	12.172	16.15%
Layer1	13.132	17.42%
Layer2	12.172	16.15%
Layer3	13.961	18.52%
Layer4	23.576	31.28%
Layer3.1 block	7.475	9.92%
Classification head	0.330	0.44%
Convolution ops (L3 agg.)	61.830	82.0%
BatchNorm ops (L3 agg.)	2.580	3.4%

Table 5.7: RQ1.3 heaviest internal compute units from the supporting timing analysis. Source: `summary.csv` in `internal_timing_resnet18_20260515_113614`, sorted by `mean_ms` at level L3.

Unit	Mean (ms)	% of model
<code>stem.maxpool</code>	7.267	9.64%
<code>layer4.1.conv1</code>	6.157	8.17%
<code>layer4.0.conv2</code>	6.153	8.16%
<code>layer4.1.conv2</code>	6.103	8.10%
<code>layer4.0.conv1</code>	4.002	5.31%
<code>layer3.0.conv2</code>	3.669	4.87%
<code>layer3.1.conv1</code>	3.625	4.81%
<code>stem.conv1</code>	3.615	4.80%

size and boundary-crossing interval shows that transfer burden is a dominant explanatory factor for the boundary-crossing cost gradient observed across the four RQ1.1 split points.

B. Activation size alone does not determine suitability. Although `split_after_layer4` achieved the lowest boundary-crossing interval in RQ1.1 and one of the lowest raw split mean latencies (79.58 ms), it was classified as compute-degenerate because the second service executed only average pooling and the final linear layer — a residual computation segment well under 1 ms. This means that minimising activation-transfer size to its smallest value does not, by itself, produce a suitable microservice decomposition. A boundary that pushes nearly all computation to one side leaves the second service structurally empty, which defeats the purpose of decomposition.

The internal timing experiment provides quantitative support for this observation.

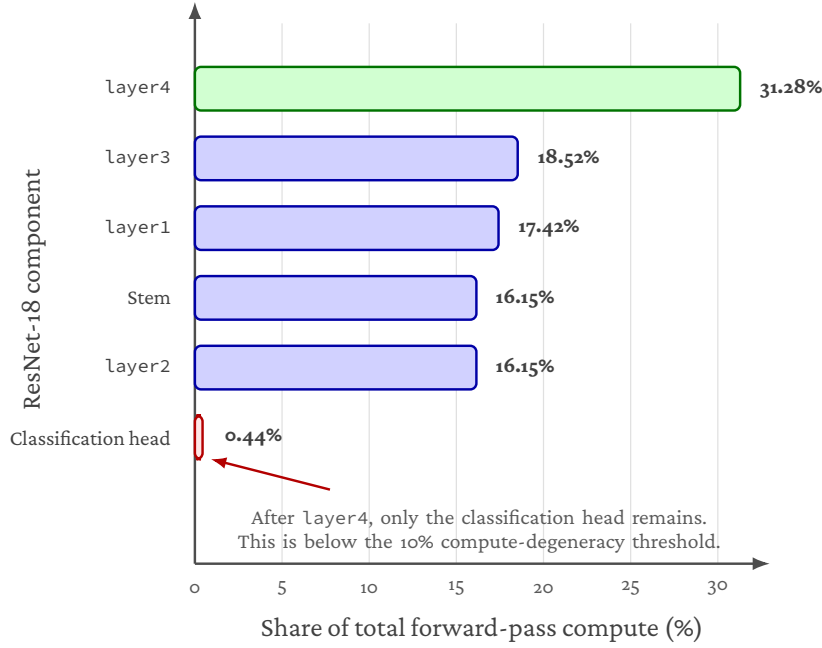


Figure 5.2: Stage-level compute distribution from the RQ1.3 internal timing analysis. The final classification head contributes less than 1% of total forward-pass compute, while layer4 accounts for the largest stage-level share. This explains why `split_after_layer4` is raw-fast but compute-degenerate: almost all meaningful model computation remains before the split.

As shown in fig. 5.2, the monolithic forward pass took approximately 75.38 ms on average, of which layer4 alone accounted for 31.28% (approximately 23.58 ms). More importantly, the computation remaining after a layer4 split — average pooling plus the final linear layer — accounts for only about 0.33 ms in total, or roughly 0.44% of the full forward pass. This means that a split after layer4 places approximately 99.6% of the total computation on the first service and leaves only about 0.4% on the second. By contrast, a split after layer3 places approximately 68.3% of compute on the first service and 31.7% on the second when layer4, pooling, and classification are counted together, producing a substantially more balanced distribution.

The supporting timing analysis also shows that compute is concentrated in a small number of heavy internal units rather than being spread uniformly. As shown in table 5.7, the heaviest units are dominated by convolutional operations in later network regions, especially layer4. This makes a very late split structurally problematic: it transfers a small activation tensor, but it also leaves the remote side with almost no substantial computation.

This distinction between raw-fastest and structurally suitable is consistent with literature that treats compute distribution as a first-class partition criterion alongside communication cost [14, 18, 20, 21].

C. The RQ1.2 controlled comparison clarifies the role of compute placement.

The most precise supporting evidence for the role of compute distribution comes from the RQ1.2 comparison between `split_after_layer3_block0` and `split_after_layer3`. Both conditions transfer an intermediate tensor of approximately 196 KiB. If activation-transfer size were the sole determinant of boundary-crossing cost, these two conditions should exhibit similar boundary-crossing intervals and similar end-to-end latencies. They do not.

`split_after_layer3_block0` exhibited a boundary-crossing interval of 33.97 ms, while `split_after_layer3` exhibited a boundary-crossing interval of 26.16 ms — a difference of approximately 7.8 ms despite identical activation size. RQ1.2 already attributed this gap to compute placement rather than to a transport or serialisation difference: as reported in section 5.2.5, the per-service compute means in `cross_condition.csv` show service-B compute of 31.87 ms and 24.06 ms for the two conditions (a 7.81 ms gap), while the non-compute residual is identical at approximately 2.10 ms for both. The internal timing data sharpens this picture by identifying the specific block responsible for the service-B compute gap. The `layer3.1` block (the second residual block of `layer3`) accounts for approximately 9.92% of total monolithic compute, or 7.48 ms. When the split is placed after `layer3.0`, Service B executes `layer3.1` in addition to `layer4`, pooling, and classification; when the split is placed after the full `layer3`, that computation remains on Service A. The 7.48 ms monolithic cost of `layer3.1` is within roughly 4% of the 7.81 ms service-B compute gap measured in RQ1.2, which is the expected order of agreement given that the two values come from separate runs (instrumented monolithic vs split benchmark). Taken together with the identical non-compute residual, this provides strong supporting evidence that the boundary-crossing difference is associated with compute placement on the remote side — specifically, with whether `layer3.1` is executed by Service A or Service B — rather than with activation-transfer size, which is identical in both cases.

This comparison also shows why a purely tensor-size-based explanation is incomplete. The two refined conditions share the same activation size, but the internal work performed after the boundary differs materially. In that sense, RQ1.2 holds activation size constant while making compute placement the main observed difference within the carried-forward region.

This observation is consistent with the principle established in the partitioning literature that compute distribution must be considered alongside communication when interpreting end-to-end split behavior and boundary-crossing measurements [14, 18, 27].

D. Internal timing confirms that compute is unevenly distributed within ResNet-

18. The internal timing experiment provides a structural explanation for why different boundaries within the same coarse region yield different performance. As visualised in fig. 5.2, the per-stage compute shares are approximately: stem 16.15%, `layer1` 17.42%, `layer2` 16.15%, `layer3` 18.52%, and `layer4` 31.28%. Aggregating across the level-3 operation rows in table 5.6, convolution operations account for approximately 82% of total compute, while batch normalisation contributes approximately 3.4% (as reported in table 5.6) and individual ReLU or residual-addition operations are negligible.

The operation-level support data adds a more concrete view of this distribution. As shown in table 5.7, the heaviest internal units are predominantly convolutional operations, with the largest shares concentrated in `layer4` and the late `layer3` region. By contrast, batch normalisation, residual addition, and ReLU operations are individually small and do not serve as meaningful compute anchors by themselves. This confirms that ResNet-18 is not only uneven at the stage level, but also internally dominated by a relatively small number of heavy convolutional units.

Two aspects of this distribution are relevant to the RQ1.1 and RQ1.2 findings. First, `layer4` is by far the most compute-intensive stage, which explains why a split after `layer4` is structurally degenerate: it assigns the dominant compute block entirely to the first service. Second, the per-block decomposition within `layer3` reveals that `layer3.0` and `layer3.1` do not contribute equally; the difference in their execution times contributes to the boundary-crossing asymmetry observed between `split_after_layer3_block0` and `split_after_layer3`. This structural unevenness is consistent with the design of ResNet-18, where each successive stage doubles the channel count and halves the spatial resolution, producing progressively heavier convolution operations in later stages [11].

It should be noted that the internal timing experiment was conducted on the monolithic model and therefore measures local per-layer execution cost without the serialisation and communication overhead present in the split benchmarks. The timing values are used here to explain the compute-distribution dimension; the actual boundary-crossing cost is a composite of both communication and compute-related factors as measured in the split benchmarks themselves.

The interpretation of these findings and the answer to RQ1.3 are presented in section 6.1.3.

5.4 RQ1.4 Multi-Microservice Kubernetes Inference Chain

5.4.1 Research Question

How does increasing the number of microservices affect end-to-end inference cost when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution?

RQ1.4 moves from the local split-selection and explanatory stages of RQ1.1–RQ1.3 to the final thesis-facing local Kubernetes benchmark for the coarse-stage ResNet-18 chain family. Rather than selecting a boundary through carry-forward, this stage benchmarks a predefined set of five in-cluster conditions from `monolithic_k8s_1svc` to `chain_5svc` to measure how end-to-end latency changes as synchronous service boundaries are added under controlled local Kubernetes execution.

The literature framing for multi-boundary chain cost and Kubernetes-level deployment overhead is discussed in sections 3.1 and 3.3.

5.4.2 Experimental Setup

The thesis-facing RQ1.4 benchmark used the frozen controlled local Kubernetes run `rq1_4_fully_controlled_20260515_141036`. The `environmentsnapshot` (`merged_results/environment.` records an Ubuntu/Linux runtime (`Linux-6.17.0-23-generic-x86_64-with-glibc2.41`), Python 3.10.20, and PyTorch 2.11.0+cpu. Deployment metadata (`merged_results/deployment_metadata.json`) shows that the benchmark ran in namespace `rq14` on a single-node kind cluster (`kind-thesis-rq14`); for every condition, all service pods were scheduled on the same node, `thesis-rq14-control-plane`. The container image is the mutable local tag `thesis-inference:latest` with pull policy `IfNotPresent`, as documented in the manifest’s reproducibility note (section 4.8). The evaluated model was pretrained ResNet-18 with CPU-only FP32 execution and input shape $1 \times 3 \times 224 \times 224$.

CPU stabilisation controls were requested for the benchmark processes and applied where the runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were all requested at one thread and observed at one thread. CPU affinity to one physical core with SMT excluded was requested and applied; the observed affinity set was core 0. Process priority `nice -5` was requested and applied. Governor and turbo controls were also requested, but writes to `/sys` failed because the containerised environment exposed those paths as read-only. The detected governor was already `performance`, and turbo was already disabled. ASLR remained at the detected value 2 and was not modified.

Methodological validity was established before timing. For all five conditions, local segment validation against the monolithic model and live gRPC chain validation against the deployed services both passed with `max_abs_diff = 0.0`, `atol =  $1 \times 10^{-5}$` , and `num_inputs = 5`.

5.4.3 Conditions

Five conditions were evaluated in a rolling orchestration sweep:

Table 5.8: RQ1.4 benchmark configuration

Setting	Value
Runtime	Local Ubuntu/Linux Kubernetes benchmark
Cluster	Single-node kind cluster (kind-thesis-rq14)
Namespace	rq14
Conditions	Five predefined configurations from monolithic_k8s_1svc to chain_5svc
Execution mode	Local in-cluster rolling orchestration over the predefined condition set
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MiB (16,777,216 bytes)
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	512 MiB / 512 MiB
Client CPU request / limit	1 core / 1 core
Client memory request / limit	1 GiB / 1 GiB
Image pull policy	IfNotPresent
Carry-forward	Not applicable; RQ1.4 benchmarks a predefined configuration set
Parity validation	Local segment validation and live gRPC chain validation; num_inputs = 5, atol = 1×10^{-5}
Warmup calibration	Trailing window = 10, CV threshold = 0.02

The byte totals reported below are the frozen runner's `total_activation_bytes_mean` values converted with 1024 bytes/KiB. This runner metric sums the input tensor and the tensors recorded across the service calls, so it is a per-request recorded tensor-byte total rather than the boundary-only activation-transfer size used in the local split-selection sections.

- **monolithic_k8s_1svc**: Full ResNet-18 inference in a single Kubernetes service. This is the Kubernetes-native monolithic baseline.

- **chain_2svc**: Two services with a split after layer2. Recorded tensor bytes per request: 980 KiB.
- **chain_3svc**: Three services with splits after layer1 and layer3. Recorded tensor bytes per request: 1568 KiB.
- **chain_4svc**: Four services with splits after layer1, layer2, and layer3. Recorded tensor bytes per request: 1960 KiB.
- **chain_5svc**: Five services with splits after layer1, layer2, layer3, and layer4. Recorded tensor bytes per request: 2058 KiB.

Carry-forward is not applicable in RQ1.4. The frozen `carry_forward.json` states that carry-forward is defined only for the two-service split-selection experiments (RQ1.1 and RQ1.2), whereas RQ1.4 benchmarks this predefined Kubernetes chain set.

5.4.4 Results

Table 5.9: RQ1.4 summary of mean latency and overhead vs. Kubernetes monolithic baseline. Source: `merged_results/condition_summaries.csv` in `rq1_4_fully_controlled_20260515_141036`.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	81.066	79.506	4.017	88.250	—
chain_2svc	83.387	81.803	3.525	90.189	+2.321 / +2.9%
chain_3svc	87.728	85.775	4.233	95.269	+6.662 / +8.2%
chain_4svc	90.572	88.675	3.929	97.713	+9.506 / +11.7%
chain_5svc	92.535	90.456	3.965	99.491	+11.469 / +14.1%

Figure 5.3 visualises the same latency trend reported in table 5.9. By `chain_4svc`, the mean-latency overhead is 9.506 ms out of the final `chain_5svc` overhead of 11.469 ms, so most of the measured increase has already occurred before the five-service configuration.

Table 5.10: RQ1.4 overhead and non-compute/platform cost breakdown. Recorded tensor-byte totals are taken from `total_activation_bytes_mean` in `merged_results/condition_summaries.csv` and converted to KiB (1024 bytes/KiB); the metric includes the input tensor. The platform column is `non_compute_overhead_ms_mean`.

Condition	Overhead (ms)	Overhead (%)	Tensor Bytes (KiB)	Non-Compute / Platform (ms)
chain_2svc	2.321	2.9	980	6.215
chain_3svc	6.662	8.2	1568	9.435
chain_4svc	9.506	11.7	1960	12.088
chain_5svc	11.469	14.1	2058	13.633

Methodological note. Effect sizes are computed from pooled per-iteration data ($N = 1,000$ iterations per condition). With large N , Mann-Whitney p -values are near-zero for any non-trivial difference and should not be over-interpreted as evidence of strong practical significance. Cohen’s d provides a more informative measure of effect magnitude. Cross-round consistency and parity validation are the primary indicators of result stability and methodological validity.

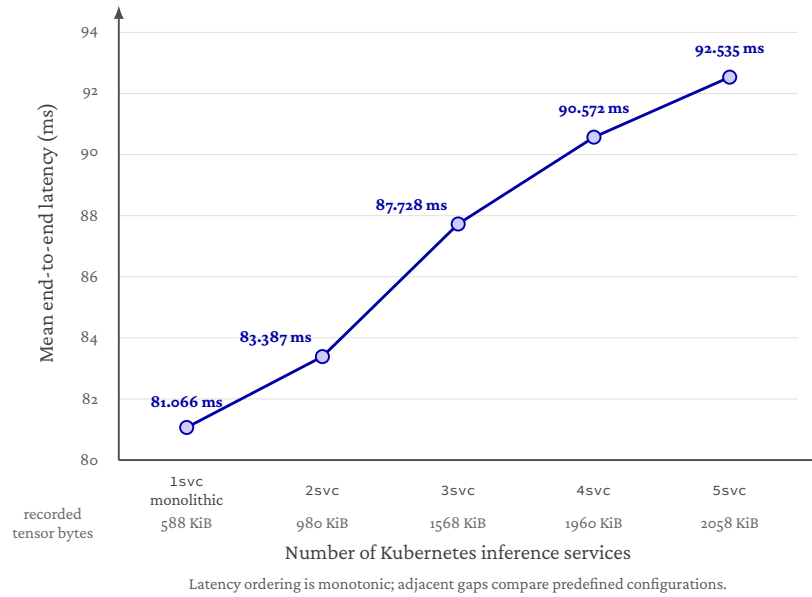


Figure 5.3: Mean end-to-end latency for the RQ1.4 local Kubernetes chain family (rq1_4_fully_controlled_20260515_141036). Latency increases as the predefined service-chain configurations move from one to five services, but the adjacent differences are descriptive comparisons across this predefined family rather than isolated causal estimates for a single added boundary.

Table 5.11: RQ1.4 effect sizes relative to monolithic baseline. Source: merged_results/effect_sizes.csv in rq1_4_fully_controlled_20260515_141036.

Condition	Cohen's <i>d</i>	Interpretation	Mann-Whitney <i>p</i>
chain_2svc	0.614	medium	7.66×10^{-77}
chain_3svc	1.614	large	4.02×10^{-181}
chain_4svc	2.392	large	3.23×10^{-276}
chain_5svc	2.874	large	1.75×10^{-301}

The frozen controlled local Kubernetes run was stable across rounds. Round CVs remained low for all five conditions: 0.0085 for monolithic_k8s_1svc, 0.0099 for chain_2svc, 0.0161 for chain_3svc, 0.0097 for chain_4svc, and 0.0074 for chain_5svc. Together with the parity checks above, this supports treating the reported condition means as representative of the local in-cluster benchmark.

The interpretation of these results and the answer to RQ1.4 are presented in section 6.2.1.

Table 5.12: RQ1.4 cross-round consistency. Source: merged_results/round_consistency.json in rq1_4_fully_controlled_20260515_141036.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	81.066	0.6864	1.8009	0.0085
chain_2svc	83.387	0.8288	2.0200	0.0099
chain_3svc	87.728	1.4109	3.8162	0.0161
chain_4svc	90.572	0.8826	2.0073	0.0097
chain_5svc	92.535	0.6820	1.6288	0.0074

5.5 RQ1.5 Azure Kubernetes Service Transfer Validation

5.5.1 Research Question

Does the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain transfer to Azure Kubernetes Service, and how do the absolute and relative costs change under managed cloud execution?

RQ1.5 moves the predefined RQ1.4 chain family from local kind Kubernetes to Azure Kubernetes Service (AKS). The purpose is not to select a new decomposition, but to test whether the architectural ordering observed in local Kubernetes remains credible when the same synchronously chained inference workload is executed on managed cloud infrastructure. The stage therefore compares absolute latency, relative overhead, and condition ordering between the frozen RQ1.4 local Kubernetes run and the frozen RQ1.5 AKS run.

The literature framing for transfer-validation between local Kubernetes and managed cloud execution is discussed in section 3.3: managed Kubernetes, CNI, and cloud-network studies motivate treating local-to-cloud transfer as a directional comparison rather than a millisecond-identical reproduction [6, 8, 16], while benchmarking guidance motivates reporting ranking, tails, and cross-round stability alongside means [13].

5.5.2 Experimental Setup

The thesis-facing RQ1.5 benchmark used the frozen controlled AKS transfer-validation benchmark `rq1_5_fully_controlled_20260515_145954`. The deployment ran in namespace `rq15` on a single-node AKS cluster, with `Standard_D8s_v3` nodes (per `deployment_metadata.json`, `node_instance_type`) in the `swedencentral` Azure region. Deployment metadata confirms that pod colocation was enforced: for every condition, the benchmark client and all service pods were scheduled on the same AKS node (`aks-rq15pool-42653082-vmss000000`). All AKS conditions used the same pinned container image digest: `sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b`.

The service-pod memory request and limit were raised from 512 MiB in RQ1.4 to 1 GiB in RQ1.5. This is a within-experimenter configuration change rather than a managed-cloud constraint: it affects pod admission and the cgroup memory ceiling but not the single-threaded ResNet-18 CPU inference path itself. The client memory profile (1 GiB request / limit) is unchanged from RQ1.4, and the Guaranteed QoS class, CPU request/limit, image, and pull policy are otherwise identical to the RQ1.4 carry-forward setup.

Threading, affinity, and priority controls followed the same stabilisation intent as the preceding stages and were applied where the managed AKS runtime allowed. Threading controls for PyTorch, OpenMP, MKL, and OpenBLAS were requested and observed at one thread. CPU affinity was applied to one physical core with SMT excluded; the observed affinity set was core 0. Process priority `nice -5` was requested but could not be applied due to insufficient privileges, so the benchmark ran at `nice=0`. CPU governor and turbo controls were deliberately disabled in the AKS configuration because the relevant sysfs interfaces were unavailable. These constraints are treated as realistic

cloud-environment limitations of managed or virtualised execution, not as architectural effects of the microservice decomposition.

Before timing, both local chain-segment validation and live gRPC validation passed for every condition with $\text{max_abs_diff} = 0.0$ and $\text{atol} = 1 \times 10^{-5}$. This supports functional equivalence across the monolithic and chained AKS deployments.

5.5.3 Conditions

RQ1.5 reused the same predefined chain family as RQ1.4:

- **monolithic_k8s_1svc**: Full ResNet-18 inference in a single Kubernetes service.
- **chain_2svc**: Two services with a split after layer2. Recorded tensor bytes per request: 980 KiB.
- **chain_3svc**: Three services with splits after layer1 and layer3. Recorded tensor bytes per request: 1568 KiB.
- **chain_4svc**: Four services with splits after layer1, layer2, and layer3. Recorded tensor bytes per request: 1960 KiB.
- **chain_5svc**: Five services with splits after layer1, layer2, layer3, and layer4. Recorded tensor bytes per request: 2058 KiB.

Carry-forward is not applicable in this stage. The experiment evaluates a fixed deployment family in AKS rather than choosing a new split point. As in RQ1.4, the recorded tensor-byte totals are taken from the runner metadata and include the input tensor as well as hop activations; they are therefore used as a consistent input-inclusive traffic proxy, not as a boundary-only activation-transfer size.

5.5.4 Results

The transfer comparison was derived directly from the frozen RQ1.4 and RQ1.5 summary CSV artifacts, because the generated RQ1.5 report does not itself embed the cross-stage comparison.

Figure 5.4 visualises the transfer comparison in table 5.16. The overhead trend is similar across environments even though the absolute millisecond values differ.

The rank ordering is identical between the frozen local Kubernetes and AKS runs: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`. Absolute latency is lower in the AKS run for every condition, but the within-environment ordering and the direction of the overhead trend are preserved.

The AKS run is stable across measured rounds for all chained conditions, with round CVs of 0.0019–0.0061. The monolithic baseline shows a slightly higher round CV of 0.0117. This pattern is compatible with managed-cloud variability on the `Standard_D8s_v3` VM, but the run does not isolate a single cause; in this frozen measurement, the chained conditions show lower round-level CVs than the monolithic baseline. Warmup calibration indicated that each run reached a stable trailing-window CV at least once before measurement, while the cross-round CVs of the measured iterations provide the stronger stability evidence for the final reported means. As in the previous stage, effect sizes and Mann-Whitney tests were computed from pooled per-iteration data and are treated as

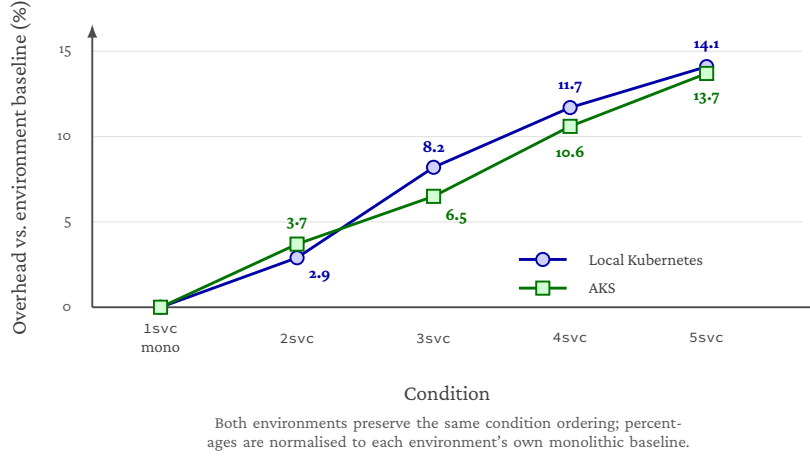


Figure 5.4: RQ1.5 normalised overhead transfer comparison between local Kubernetes and AKS. Overheads are computed relative to each environment's own monolithic Kubernetes baseline, so the figure visualises directional transfer of the service-count trend rather than raw latency equivalence.

supplementary because $N = 1,000$ per condition makes very small differences statistically significant. Cross-round stability, parity validation, and preserved ordering are the primary evidence for this stage.

The interpretation of these results and the answer to RQ1.5 are presented in section 6.2.3.

5.6 RQ1.5b AKS Multi-Node Sensitivity

5.6.1 Research Question

When every chain hop is forced across a node boundary, how much additional end-to-end latency does the inter-node network path add, relative to the same-node RQ1.5 baseline?

RQ1.5b is a sensitivity stage. It does not replace RQ1.5; the frozen single-node AKS results in section 5.5 remain the thesis-facing primary numbers. The role of RQ1.5b is to quantify, in milliseconds, the inter-node penalty that the single-node design intentionally hides, so that the single-node deployment threat to validity becomes a bounded, measured cost rather than an unspecified one. Public-cloud networks exhibit performance variability that can shift absolute latency without changing the application [6], which is precisely why this sensitivity stage measures the multi-node penalty directly rather than assuming it.

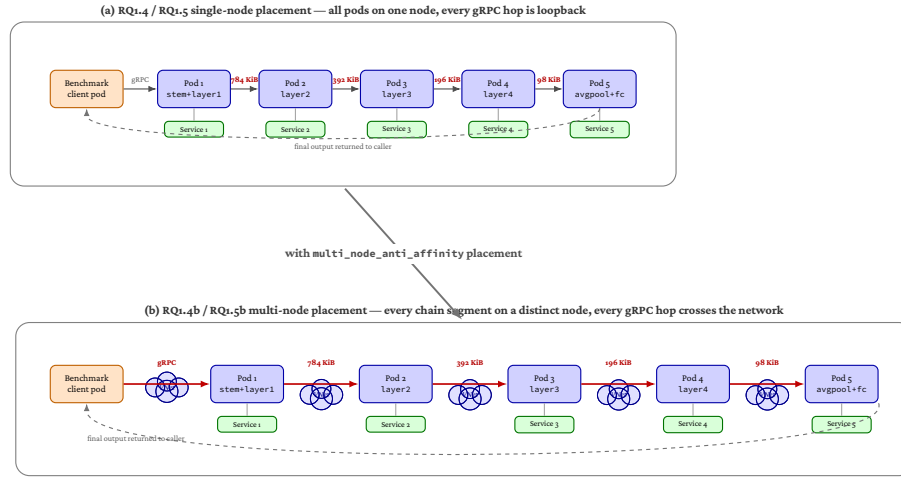


Figure 5.5: Same chain, two topologies. (a) RQ1.4 / RQ1.5 single-node placement: all six pods on one node, every gRPC hop is loopback. (b) RQ1.4b / RQ1.5b multi-node placement: each chain segment lands on a distinct cluster node and the client lands on a sixth dedicated node, so every gRPC hop crosses the Azure VNet. The chain itself is identical between the two panels; only the pod-to-node assignment changes.

5.6.2 Experimental Setup

The thesis-facing RQ1.5b benchmark used the frozen multi-node sensitivity run `rq1_5b_multinode_20260515_152`. The deployment ran in namespace `rq15b` on a multi-node AKS cluster in the `swedencentral` Azure region. All worker nodes were `Standard_D8s_v3` (per `node_instance_type` in `deployment_metadata.json`) and shared one node pool, so the only intentional experimental difference from RQ1.5 is the pod-to-node assignment.

Multi-node placement was enforced by Kubernetes pod anti-affinity. Each service pod of a given condition repels every other service pod of that condition keyed on `kubernetes.io/hostname`, and the benchmark client repels every pod with `workload-role: service`. The combined rule guarantees that for the `chain_5svc` condition all five service segments land on distinct nodes and the client lands on a sixth dedicated node, so every gRPC hop in the inference path crosses the Azure VNet rather than staying loopback. Compliance was verified after deployment by reading observed `nodeName` values from the Kubernetes API for each pod and by checking the runner’s aggregated multi-node validation flag. For every condition, the recorded service nodes were pairwise disjoint and the client node was distinct from every service node, as preserved in `merged_results/deployment_metadata.json`; the aggregated `multi_node_validation` entry in `merged_results/environment.json` records `passed=True` for the run as a whole. Functional parity was independently verified: `max_abs_diff = 0.0` at `atol = 1 × 10-5` for all five conditions in both the local segment validation and the live gRPC chain validation (`num_inputs = 5`).

The shared benchmark contract is otherwise unchanged from RQ1.5: ResNet-18 with pretrained weights, CPU FP32 inference, the same five chain conditions, the same five-

round / 200-iteration / 50-warmup discipline, the same Guaranteed-QoS pod profile (1 vCPU + 1 GiB per pod), and the same image used in RQ1.5 (sha256:d292aef407ee8a15da2bd2daa680de1cc24b0975e7de81d27e1fc3a7d128b8736b).

5.6.3 Conditions

RQ1.5b reuses the same predefined chain family as RQ1.4 and RQ1.5 (`monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, `chain_5svc`). The full family is included rather than a single representative because the per-condition delta is the headline metric: a single point cannot show whether the inter-node penalty grows with hop count, dominates one specific boundary, or stays roughly flat. Five points support the direct comparison shown in table 5.19.

5.6.4 Results

The within-environment ordering is preserved (`monolithic_k8s_1svc` < `chain_2svc` < `chain_3svc` < `chain_4svc` < `chain_5svc`) and the per-hop addition does not keep growing: the `chain_4svc` → `chain_5svc` increment is 2.502 ms, smaller than the preceding 4.562 ms step, mirroring the saturating per-hop behaviour observed in RQ1.4 (table 5.9). The same pattern is visible in the single-node RQ1.5 run, where the `chain_4svc` → `chain_5svc` increment is 2.225 ms versus a preceding 2.884 ms step (table 5.14), so the saturating last step is consistent across RQ1.4, RQ1.5, and RQ1.5b. Cross-round CVs are 0.0036–0.0127 across all chained conditions. Because the placement metadata confirms distinct AKS nodes for the service pods, these CVs show that the distributed placement did not introduce large round-to-round drift in this frozen run; they do not isolate a single causal source of stability.

Effect-size note. Per-iteration Cohen’s *d* and Mann–Whitney statistics are emitted alongside the frozen artefact in `merged_results/effect_sizes.csv` for completeness, but RQ1.5b is framed as a sensitivity stage on the single-node RQ1.5 primary. The headline numbers in this section are therefore the per-condition multi-node penalty (table 5.19) and the cross-round CVs (table 5.20); a full effect-size table analogous to table 5.11 is intentionally omitted to avoid implying that RQ1.5b is a second primary result.

Table 5.19 and fig. 5.6 summarise the per-condition multi-node penalty. The penalty is positive across all conditions but is not monotonic with chain depth: the `chain_3svc` delta is smaller than the `chain_2svc` delta, while the largest absolute penalty appears at `chain_5svc`. The `monolithic_k8s_1svc` baseline is 2.305 ms slower in the multi-node placement than in the single-node RQ1.5 baseline. This is compatible with the client now shipping a 588 KiB input tensor across the Azure VNet rather than over loopback; the cited RPC study supports the general mechanism that layered RPC stacks can add transport, parsing, copying, and serialisation work on top of the payload bytes themselves [37]. The chained conditions, with intermediate activations of 784, 392, 196 and 98 KiB, yield per-condition deltas between 2.131 ms and 4.086 ms, with the deepest `chain_5svc` configuration carrying the largest absolute penalty. The within-environment monotonic ordering is preserved.

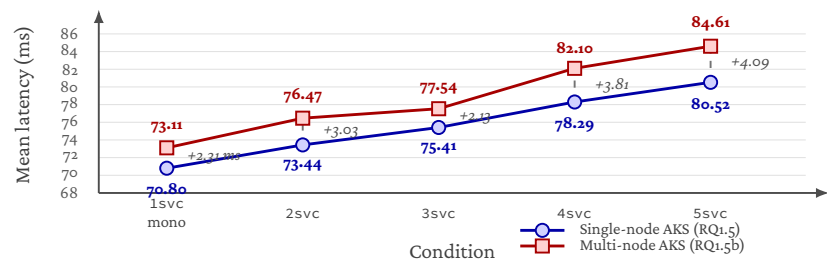


Figure 5.6: RQ1.5b multi-node penalty per condition. Same SKU (Standard_D8s_v3), same region (swedencentral); the intentional experimental contrast is single-node colocation versus multi_node_anti_affinity placement. The figure is manually rendered from the same frozen summary artefacts as table 5.19. Per-condition deltas range from 2.131 ms (chain_3svc) to 4.086 ms (chain_5svc), and the monolithic-baseline penalty is 2.305 ms.

The interpretation of these results and the answer to RQ1.5b are presented in section 6.2.4.

Table 5.13: RQ1.5 AKS benchmark configuration. Source: merged_results/config.yaml and deployment_metadata.json in rq1_5_fully_controlled_20260515_145954.

Setting	Value
Runtime	Azure Kubernetes Service benchmark
Cluster	Single-node AKS cluster thesis-rq15
Node type	Standard_D8s_v3
Region	swedencentral
Namespace	rq15
Conditions	Five predefined configurations from monolithic_k8s_1svc to chain_5svc
Execution mode	In-cluster rolling orchestration with teardown between conditions
Model	ResNet-18 (pretrained)
Device	CPU
Precision	FP32
Input shape	$1 \times 3 \times 224 \times 224$
Rounds	5
Warmup iterations	50
Measured iterations	200
Cooldown between conditions	5 seconds
Random seed	42
Communication	gRPC (in-cluster Kubernetes)
gRPC port	50051
Maximum message size	16 MiB
Service pod CPU request / limit	1 core / 1 core
Service pod memory request / limit	1 GiB / 1 GiB
Client CPU request / limit	1 core / 1 core
Client memory request / limit	1 GiB / 1 GiB
Pod QoS	Guaranteed for service pods
Image pull policy	Always
Carry-forward	Not applicable; RQ1.5 evaluates a predefined chain family
Parity validation	Local segment validation and live gRPC chain validation; num_inputs = 5, atol = 1×10^{-5}
Warmup calibration	Trailing window = 10, CV threshold = 0.02

Table 5.14: RQ1.5 AKS summary of mean latency and overhead vs. AKS monolithic baseline. Source: merged_results/condition_summaries.csv in rq1_5_fully_controlled_20260515_145954.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	70.802	70.224	2.659	75.151	—
chain_2svc	73.437	73.124	1.831	76.602	+2.635 / +3.7%
chain_3svc	75.410	75.043	1.971	78.301	+4.608 / +6.5%
chain_4svc	78.294	77.923	2.190	81.698	+7.492 / +10.6%
chain_5svc	80.519	80.078	2.158	84.635	+9.717 / +13.7%

Table 5.15: RQ1.5 AKS overhead and inferred non-compute/platform cost breakdown. Source: merged_results/condition_summaries.csv in rq1_5_fully_controlled_20260515_145954; the platform column is non_compute_overhead_ms_mean.

Condition	Overhead (ms)	Overhead (%)	Recorded Tensor Bytes (KiB)	Inferred Non-Compute / Platform (ms)
chain_2svc	2.635	3.7	980	4.489
chain_3svc	4.608	6.5	1568	7.073
chain_4svc	7.492	10.6	1960	9.138
chain_5svc	9.717	13.7	2058	10.490

Table 5.16: RQ1.5 transfer comparison between local Kubernetes and AKS. Source: merged_results/condition_summaries.csv in rq1_4_fully_controlled_20260515_141036 and rq1_5_fully_controlled_20260515_145954.

Condition	Local mean (ms)	AKS mean (ms)	Local overhead	AKS overhead
monolithic_k8s_1svc	81.066	70.802	0.0%	0.0%
chain_2svc	83.387	73.437	2.9%	3.7%
chain_3svc	87.728	75.410	8.2%	6.5%
chain_4svc	90.572	78.294	11.7%	10.6%
chain_5svc	92.535	80.519	14.1%	13.7%

Table 5.17: RQ1.5 AKS cross-round consistency. Source: frozen RQ1.5 merged_results/round_consistency.json.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	70.802	0.8255	1.9646	0.0117
chain_2svc	73.437	0.1398	0.3183	0.0019
chain_3svc	75.410	0.1879	0.5187	0.0025
chain_4svc	78.294	0.1960	0.4629	0.0025
chain_5svc	80.519	0.4911	1.1659	0.0061

Table 5.18: RQ1.5b AKS multi-node summary of mean latency and overhead vs. AKS multi-node monolithic baseline. Source: merged_results/condition_summaries.csv in rq1_5b_multinode_20260515_152628.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	73.108	72.403	3.707	79.598	—
chain_2svc	76.470	75.653	3.435	82.696	+3.362 / +4.6%
chain_3svc	77.541	76.848	2.786	83.041	+4.433 / +6.1%
chain_4svc	82.103	81.660	2.355	86.394	+8.995 / +12.3%
chain_5svc	84.605	84.131	2.497	89.186	+11.497 / +15.7%

Table 5.19: RQ1.5b multi-node penalty, condition-by-condition vs. frozen RQ1.5 single-node AKS. Source: merged_results/condition_summaries.csv in rq1_5_fully_controlled_20260515_145954 and rq1_5b_multinode_20260515_152628; overhead points are with respect to each environment's own monolithic baseline (tables 5.14 and 5.18).

Condition	Single-node mean (ms)	Multi-node mean (ms)	Δ (ms)	Δ overhead (pp)
monolithic_k8s_1svc	70.802	73.108	+2.305	—
chain_2svc	73.437	76.470	+3.033	+0.9
chain_3svc	75.410	77.541	+2.131	−0.4
chain_4svc	78.294	82.103	+3.808	+1.7
chain_5svc	80.519	84.605	+4.086	+2.0

Table 5.20: RQ1.5b AKS multi-node cross-round consistency. Source: merged_results/round_consistency.json in rq1_5b_multinode_20260515_152628.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	73.108	0.6134	1.5076	0.0084
chain_2svc	76.470	0.9679	2.3363	0.0127
chain_3svc	77.541	0.4388	0.9236	0.0057
chain_4svc	82.103	0.3757	0.9645	0.0046
chain_5svc	84.605	0.3047	0.6512	0.0036

5.7 RQ2.1 Service Identity and Mutual TLS Hardening

5.7.1 Research Question

What performance and operational overhead is introduced when the selected AKS microservice inference path is hardened with service identity, mutual TLS, and inter-service authorization policy?

RQ2.1 adds the first security-hardening layer to the Azure chain-family deployment carried forward from RQ1.4 and RQ1.5, rather than reopening the local split-selection result from RQ1.1–RQ1.2. The primary thesis-facing topology is `chain_2svc`, because it was the lowest-overhead non-monolithic chain in the preceding Kubernetes and AKS chain-count stages. The additional `chain_5svc` run is used as a stress case to examine whether the same security mechanism becomes more costly when the number of protected service boundaries increases. The goal is not to evaluate trusted execution environments in this stage, but to measure the cost of communication-level hardening: service identity, mutual TLS, and service-account-based authorization for inter-service traffic.

The literature framing for service-mesh-based mTLS, identity, and authorisation policy is discussed in section 3.4.

5.7.2 Experimental Setup

The RQ2.1 benchmark used two frozen paired AKS artifact sets: a chain-2 main driver `rq2_1_paired_20260515_160012` and a chain-5 maximum-depth stress driver `rq2_1_paired_20260517_1143`. Both runs used the same AKS cluster (`thesis-rq15`), strict same-node placement, and an isolated system node pool so that control-plane pods did not share the benchmark node.

All RQ2.1 conditions used the same pinned container image digest, shown across two lines for typesetting:

```
sha256:d292aef407ee8a15da2bd6a680de1cc2
4b0975e7de81d27e1fc3a7d128b8736b
```

The mTLS condition used the managed AKS Istio add-on (`asm-1-29`). Each paired pass executed both security conditions; per `paired_summary.md` the alternation was mTLS-first on passes 1, 3, 5 and plain-first on passes 2, 4 in both topologies, so each condition has five executions and 1,000 measured iterations in total.

The plain condition used Kubernetes service-to-service communication without mesh injection. The mTLS condition added one Istio proxy per inference service, service accounts for the protected services, workload-level peer-authentication objects, and authorization-policy objects. The benchmark client remained outside the mesh. Consequently, RQ2.1 measures hardening of the inter-service path inside the inference chain rather than external ingress hardening. For `chain_2svc`, the downstream service was protected by strict mTLS and an authorization policy that allowed only the expected upstream service account. For `chain_5svc`, the same pattern was applied to each downstream segment, producing four immediate-upstream authorization policies.

Table 5.21: RQ2.1 paired AKS security benchmark configuration. Sources: merged/paired_summary.md in the two artifacts named below.

Setting	Value
Primary topology	chain_2svc; split after layer2
Stress topology	chain_5svc; splits after layer1, layer2, layer3, and layer4
Main artifact	rq2_1_paired_20260515_160012
Stress artifact	rq2_1_paired_20260517_114303
Conditions	Plain AKS chain vs. managed-Istio mTLS chain
Mesh revision	asm-1-29
Cluster	AKS thesis-rq15; isolated benchmark and system node pools
Benchmark node type	Standard_D8s_v3
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Random seed	42
Service resources	1 CPU / 1 GiB per inference service
Client resources	100m CPU request, 1 CPU limit, 1 GiB memory
Sidecar resources	100m CPU request, 500m CPU limit, 128 MiB memory request, 512 MiB memory limit
Placement	Strict same-node service placement on tainted benchmark node pool
Security validation	Static manifest validation, runtime policy validation, full-chain positive probe, and non-mesh direct-denial probe

Security validation passed in both frozen artifact sets. The mTLS runs passed static manifest validation and runtime validation, including the expected peer-authentication and authorization-policy counts. The full-chain positive probe passed for every mTLS pass, and direct non-mesh probes were blocked for the protected downstream segments. Resource metrics were complete in both runs.

5.7.3 Results

For the primary chain_2svc topology, the mTLS/AuthZ hardening bundle increased mean latency from 75.557 ms to 79.045 ms. The mean overhead was therefore 3.488 ms, or 4.62%, and p95 latency increased by 3.623 ms. The inferred non-compute/platform

Table 5.22: RQ2.1 latency results for plain and mTLS AKS chains. Sources: merged/aggregated_results.md in rq2_1_paired_20260515_160012 (chain_2svc) and rq2_1_paired_20260517_114303 (chain_5svc).

Topology	Condition	n	Mean (ms)	Median (ms)	p95 (ms)	Inferred non-compute/platform (ms)
chain_2svc	plain	1000	75.557	75.262	80.047	4.559
chain_2svc	mtls	1000	79.045	78.590	83.670	7.891
chain_5svc	plain	1000	82.302	81.818	87.502	10.801
chain_5svc	mtls	1000	91.139	90.823	96.485	19.949

component increased from 4.559 ms to 7.891 ms. This supports the interpretation that most of the added latency is associated with the communication/platform path introduced by the mesh rather than with a change in neural-network computation.

For the additional chain_5svc stress topology, the same hardening bundle increased mean latency from 82.302 ms to 91.139 ms. The mean overhead was 8.837 ms, or 10.74%, and p95 latency increased by 8.983 ms. The inferred non-compute/platform component increased from 10.801 ms to 19.949 ms. The stress run therefore shows that the measured communication-hardening cost is larger in the four-boundary stress case than in the selected two-service chain, even though the main RQ2.1 answer remains anchored in chain_2svc. This two-point comparison should be read as depth sensitivity, not as a calibrated scaling law.

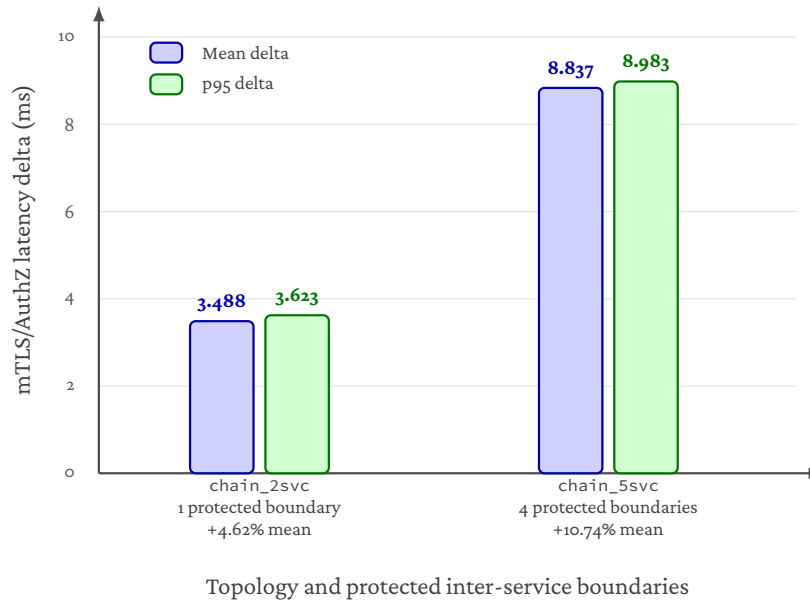
Table 5.23: RQ2.1 paired-pass latency deltas. Sources: merged/aggregated_results.md (per-pass tables) in rq2_1_paired_20260515_160012 and rq2_1_paired_20260517_114303. The pass-delta std is computed from the five per-pass deltas.

Topology	Mean delta (ms)	Mean delta (%)	Min-max pass delta (ms)	Pass-delta std (ms)	p95 delta (ms)
chain_2svc	3.488	4.62	2.223–5.230	1.091	3.623
chain_5svc	8.837	10.74	7.623–9.985	0.881	8.983

The paired-pass deltas in table 5.23 are consistent with the overall means. In chain_2svc, every paired pass shows a positive mTLS/AuthZ hardening overhead, with pass-level deltas between 2.223 ms and 5.230 ms. In chain_5svc, every paired pass is also positive, with deltas between 7.623 ms and 9.985 ms. The alternating execution order reduces the risk that the result is explained only by slow drift during the run.

Figure 5.7 visualises the paired latency deltas from table 5.23. The figure focuses on deltas rather than raw latency so that the two RQ2.1 findings remain visible: the selected two-service chain pays a modest overhead, while the deeper stress chain pays a larger overhead once four inter-service boundaries are protected.

An additional chain_2svc ablation was run after the frozen paired benchmark to interpret the hardening bundle more precisely. This ablation is supporting evidence rather than a replacement for the frozen paired result; the evidence manifest states that the ablation campaign is internally comparable only and that its deltas must not be spliced into the frozen two-condition chain-2 / chain-5 numbers. With that caveat, the full hardened ablation condition reproduces the frozen result's order of magnitude: C3–Co adds 2.840 ms, in the same range as the frozen paired chain_2svc overhead



Deltas are computed as mTLS minus plain within the paired AKS benchmark design.

Figure 5.7: RQ2.1 mTLS/AuthZ latency overhead for the primary and stress AKS chains. Mean and p95 deltas are computed within the paired plain/mTLS design. The stress topology contains four protected inter-service boundaries, so the larger overhead visualises the observed stress-case sensitivity of sidecar-mediated communication hardening.

Table 5.24: RQ2.1 chain_2svc ablation of the communication-hardening bundle. Source: merged/rq2_1_ablation_summary.md in rq2_1_ablation_20260516_112348. As documented in the evidence manifest, ablation deltas are internally comparable only and must not be spliced into the frozen two-condition paired result.

Step	Condition	Mean latency (ms)	Adjacent delta (ms)	Interpretation
C0	Plain Kubernetes, no mesh	69.901	—	Non-mesh baseline
C1	Mesh-default sidecars / auto-mTLS / no AuthZ	72.470	+2.569	Main mesh data-path cost
C2	Strict downstream mTLS / no AuthZ	72.784	+0.315	Small added enforcement cost
C3	Strict downstream mTLS plus AuthZ	72.740	-0.044	Within pass-level noise

of 3.488 ms. As shown in table 5.24, most of the adjacent latency change occurs when moving from plain Kubernetes to the mesh-default condition. Strict mTLS enforcement over mesh-default traffic and the AuthorizationPolicy step are both small relative to pass-level variation. The correct interpretation is therefore that the deployed managed-Istio data path introduces most of the measured overhead, while strict mTLS and identity-based authorization mainly add enforcement semantics with little additional steady-state latency in this workload.

The mTLS condition also increased resource requests and deployment complexity. For chain_2svc, the mesh added two injected proxies, two service accounts, two peer-

Table 5.25: RQ2.1 resource and operational overheads. Sources: merged/paired_summary.md in rq2_1_paired_20260515_160012 (chain_2svc) and rq2_1_paired_20260517_114303 (chain_5svc).

Topology	Sidcar CPU (mCPU)	Total pod CPU delta (mCPU)	Sidcar memory (MiB)	Total pod memory delta (MiB)	Schedule-to-ready delta (s)	Additional objects
chain_2svc	34.684	-56.813	82.125	93.819	6.30	5
chain_5svc	87.602	37.490	223.714	264.143	6.84	14

authentication objects, one authorization-policy object, 200m requested sidcar CPU, and 256 MiB requested sidcar memory. For chain_5svc, the mesh added five injected proxies, five service accounts, five peer-authentication objects, four authorization-policy objects, 500m requested sidcar CPU, and 640 MiB requested sidcar memory. In both cases, the always-on Istio control-plane pods were present on the isolated system pool rather than the benchmark node. The CPU counter deltas should be read as window-level operational measurements rather than perfect causal attribution: in chain_2svc, measured application-container CPU was lower in the mTLS condition, which makes the total pod CPU delta negative despite a clearly positive sidcar contribution.

The interpretation of these results and the answer to RQ2.1 are presented in section 6.3.1.

5.8 RQ2.2 Confidential VM Execution Hardening

5.8.1 Research Question

What additional performance and operational overhead is introduced when VM-level confidential execution is added to the already communication-secured AKS inference chain, when the downstream protected service is placed on AMD SEV-SNP confidential nodes?

RQ2.2 adds a second security-hardening layer to the Azure deployment path. RQ2.1 already established the communication-secured baseline: the selected chain_2svc inference pipeline runs on AKS with managed-Istio mTLS, service identity, and an authorisation policy on the protected downstream service. RQ2.2 keeps that communication-security contract fixed and changes the execution environment of the downstream service. The confidential condition places service2 on an Azure confidential VM node pool backed by AMD SEV-SNP, while the standard condition uses matched non-confidential AMD VM nodes for both services. Per the canonical evidence manifest, the thesis-facing RQ2.2 scope is service2_only; placing both services on confidential nodes (full two-service TEE) is treated as future work and is not evaluated here. He et al. [12] motivate protecting the activations received by the downstream segment beyond the in-transit mTLS layer, although they evaluate a different (malicious remote-service) threat model than the SEV-SNP infrastructure-level protection measured here.

The purpose of this stage is therefore incremental. It does not ask whether mTLS is useful; that was answered in RQ2.1. Instead, it asks what extra cost is paid when runtime protection against an additional infrastructure-level threat model is added on top of the already secured service-to-service path.

The literature framing for AMD SEV-SNP, VM-level confidential computing, and its interaction with sidecar-based service meshes is discussed in section 3.4.

5.8.2 Experimental Setup

RQ2.2 used the frozen rolling AKS artifact `rq2_2_confidential_20260516_135726`. The run preserves the RQ2.1 managed-Istio mTLS and authorisation semantics, and collects newly paired standard baselines on `Standard_D8as_v5` nodes rather than reusing the RQ2.1 numbers. The application image is the same ACR digest used in RQ1.5 and RQ2.1 (`thesisrq15acr.azurecr.io/thesis-inference@sha256:d292aef407ee...`), recorded in `image_reference.txt`.

Table 5.26: RQ2.2 benchmark configuration. Sources: `rq22_rolling_summary.json`, `rq22_terraform.tfvars.json`, and `image_reference.txt` in `rq2_2_confidential_20260516_135726`.

Setting	Value
Topology	<code>chain_2svc</code> ; split after layer2
Communication security	Managed-Istio mTLS, service identity, and authorisation policy inherited from RQ2.1
Standard VM size	<code>Standard_D8as_v5</code> (both services in standard condition; service1 in confidential condition)
Confidential VM size	<code>Standard_DC8as_v5</code> with AMD SEV-SNP (service2 in confidential condition)
TEE scope	<code>service2_only</code> ; full two-service TEE is out of scope (future work)
Region	<code>westeurope</code>
Model	ResNet-18 (pretrained)
Device / precision	CPU / FP32
Input shape	$1 \times 3 \times 224 \times 224$
Paired passes	5
Warmup iterations	50 per condition execution
Measured iterations	200 per condition execution; 1,000 per condition after merging
Artifact	<code>rq2_2_confidential_20260516_135726</code>

The benchmark compared two conditions. In the standard condition, service1 and service2 both ran on `Standard_D8as_v5`. In the confidential condition, service1 remained on `Standard_D8as_v5`, while service2 ran on `Standard_DC8as_v5` backed by AMD SEV-SNP. This isolates the cost of protecting the downstream service that receives intermediate activations and executes the later ResNet-18 segment. The rolling design alternates standard and confidential passes from a seeded order (standard-first on passes 1, 3, 5; confidential-first on passes 2, 4 per `execution_plan.json`), and parity validation against the monolithic reference (`max_abs_diff = 0.0`, `num_inputs = 5`) passed in every per-pass `parity_validation.json`.

5.8.3 Results

Table 5.27: RQ2.2 service2-only confidential execution results. Source: per-pass benchmark/raw_iterations.csv in rq2_2_confidential_20260516_135726/conditions/, aggregated across the five standard and five confidential passes (1,000 iterations per condition).

Condition	Service2 VM	n	Mean (ms)	Median (ms)	p95 (ms)
standard	Standard_D8as_v5	1000	58.492	58.289	60.255
confidential	Standard_DC8as_v5	1000	60.346	59.756	63.856
Delta	—	—	+1.854 / +3.17%	+1.467 / +2.52%	+3.601 / +5.98%

In the service2-only TEE run, mean end-to-end latency increased from 58.492 ms to 60.346 ms. The mean overhead was therefore 1.854 ms, or 3.17%. Median latency increased by 1.467 ms, and p95 latency increased by 3.601 ms. Sequential throughput, computed from the mean latency, decreased from 17.096 req/s to 16.571 req/s, a reduction of approximately 3.07%.

This is the only confidential-execution scope evaluated in the thesis. The evidence manifest scopes RQ2.2 to service2_only and explicitly treats a full two-service confidential placement as future work. The trade-off table in section 5.9 therefore compares plain AKS, mTLS, and mTLS plus selective TEE only.

The interpretation of these results and the answer to RQ2.2 are presented in section 6.3.2.

5.9 RQ2.3 Security-Hardening Trade-off Synthesis

5.9.1 Research Question

This subsection addresses the following sub-research question:

Which security-hardening configuration provides the most appropriate trade-off between performance cost, operational complexity, and protection scope for the selected AKS microservice inference deployment?

RQ2.3 is a synthesis question rather than a new benchmark. RQ2.1 measured the cost of adding service identity, mutual TLS, and inter-service authorization policy to the selected Azure inference path. RQ2.2 then measured the additional cost of placing the downstream protected service on an AMD SEV-SNP confidential VM node pool while keeping the communication-security contract fixed. RQ2.3 combines these results to compare the evaluated hardening levels and identify when each level is appropriate.

5.9.2 Synthesis Basis

The synthesis compares three deployment configurations evaluated in this thesis. The first is the plain AKS baseline from RQ1.5, where the inference chain runs without service-mesh mTLS or confidential-node placement. The second is the RQ2.1 communication-secured configuration, where managed-Istio mTLS, service identity, and authorization

policy are added to the selected chain. The third is the RQ2.2 selective confidential-execution configuration, where only the downstream protected service runs on an AMD SEV-SNP confidential node. Extending the confidential-computing boundary to both services (full two-service TEE) is out of the thesis-facing scope per the evidence manifest and is discussed only as future work.

These configurations should not be interpreted as interchangeable variants of the same security mechanism. They protect different parts of the system. Plain AKS provides the performance baseline but does not add the communication-level identity and encryption policy evaluated in RQ2.1. The mTLS/AuthZ configuration hardens the service-to-service path by authenticating workloads and enforcing authorization policy. The confidential-node configuration adds a VM-level runtime protection boundary for the downstream service placed on AMD SEV-SNP infrastructure. As discussed in sections 4.7.12, 6.3.2 and 6.6, the SEV-SNP threat-model scope is bounded: residual attack surfaces such as malicious-host interrupt injection remain outside the memory-encryption guarantee [29], and this thesis does not experimentally evaluate such attacks.

5.9.3 Trade-off Summary

The trade-off is not a single global ranking because the preferred configuration depends on the threat model. If performance is the only criterion, the plain AKS deployment is the best option. However, it does not provide the additional service identity, encrypted inter-service communication, or authorization-policy enforcement evaluated in RQ2.1. If the main security requirement is to protect communication between inference services, the mTLS/AuthZ configuration provides the most appropriate default balance. It adds a mean latency overhead of 3.488 ms, or 4.62%, for the selected `chain_2svc` topology, while enforcing the intended service-account-based policy. The `chain_5svc` stress case shows a larger cost when four inter-service boundaries are protected, reaching 8.837 ms, or 10.74%, for the full five-service chain.

If the threat model includes host-level exposure of the downstream inference component, mTLS alone is not sufficient because it protects the communication path rather than the runtime memory of the service, and prior work shows that an untrusted remote service holding only intermediate feature maps can in principle recover the input [12]. Selective confidential execution provides a narrower hardening step against an infrastructure-level adversary on the host platform. The service2-only confidential-node run increases mean latency by 1.854 ms, or 3.17%, relative to its paired standard-node mTLS baseline. This makes selective TEE attractive when the downstream service is the main infrastructure-level sensitivity boundary, while bounded by SEV-SNP's known residual attack surfaces [29], which this thesis does not experimentally evaluate.

Extending the confidential-execution boundary to both services would broaden the runtime-protection scope at additional cost. That configuration is out of the thesis-facing scope of RQ2.2 (see the evidence manifest) and is therefore discussed as future work rather than as a measured trade-off point.

The interpretation of these results and the answer to RQ2.3 are presented in section 6.3.3.

Table 5.28: RQ2.3 synthesis of evaluated security-hardening levels. mTLS overheads are taken from table 5.22; the selective TEE overhead is taken from table 5.27 relative to its own paired standard-node mTLS baseline.

Configuration	Protection scope	Measured cost	Thesis interpretation
Plain AKS	Baseline AKS deployment without mesh mTLS/AuthZ or confidential-node placement.	Fastest evaluated Azure deployment path.	Best latency baseline, but weakest protection in the evaluated security set. Suitable only when additional communication and runtime hardening are not required.
mTLS/AuthZ	Service identity, encrypted service-to-service communication, and authorization policy for the selected inference path.	chain_2svc: +3.488 ms / +4.62%; chain_5svc stress case: +8.837 ms / +10.74%.	Best general-purpose hardening layer when the main requirement is authenticated and encrypted inter-service communication. Cost is measurable but bounded for the selected chain, and grows with protected chain depth.
mTLS/AuthZ selective TEE	+ Communication hardening plus AMD SEV-SNP VM-level confidential execution for the downstream protected service (service2_only).	+1.854 ms / +3.17% over the newly paired standard-node mTLS baseline.	Adds runtime protection for the downstream segment that receives intermediate activations [12], bounded by the SEV-SNP threat model [29]. Full two-service TEE is treated as future work.

Analysis

This chapter interprets the empirical findings reported in chapter 5 and answers the research questions defined in chapter 1. Sections 6.1 to 6.3 provide per-RQ interpretation grouped by research strand and close with a cross-RQ synthesis. Section 6.4 consolidates the evidence that supports treating the conclusions as credible. Section 6.5 situates the system against the alternatives it was compared with. Section 6.6 consolidates the threats to validity.

6.1 Architectural Split Choice

6.1.1 RQ1.1: Coarse Boundary Selection

RQ1.1 asks which coarse ResNet-18 stage boundaries provide suitable two-part microservice decompositions under controlled local execution. The analytical contribution of this section is not the per-condition latency itself, which is reported in table 5.2, but the explanation of why the four coarse boundaries are not interchangeable and which of them is the strongest carry-forward candidate. The RQ1.1 results support a single coherent reading: the cost of one service boundary is a joint function of the activation transferred at that boundary and of how the remaining computation is distributed between the two services. This matches the framing used in the prior split-inference literature, which treats partition placement as a computation–communication trade-off rather than a depth-only decision [15, 18]. The four stage boundaries of ResNet-18 [11] provide a structurally clean coarse sweep, but the RQ1.1 data make clear that this structural cleanliness is a necessary, not sufficient, condition for a suitable decomposition.

All four splits are slower than monolithic. Under the controlled local conditions used here, no two-part decomposition improves raw end-to-end latency relative to monolithic inference: the four splits add between +2.4% and +4.7% mean overhead in table 5.2. The RQ1.1 result is therefore diagnostic rather than optimisation-oriented; it identifies the least-bad boundary and explains the gradient of costs across the sweep, but it does not support a claim that local two-service decomposition is a raw-latency improvement. This is consistent with the broader split-inference literature, where useful latency wins typically depend on heterogeneous compute or bandwidth conditions that are deliberately absent from RQ1.1 [15, 18].

The earliest boundary is penalised by activation expansion. The split after layer1 transferred the largest activation tensor (802,816 B, 784 KiB) and had the largest mean boundary-crossing interval at 53.81 ms (table 5.2). A pointed comparison is informative: the raw input tensor is $1 \times 3 \times 224 \times 224$ FP₃₂, i.e. 602,112 B, so the intermediate carried at this boundary is approximately $1.33\times$ the raw input. The earliest coarse stage therefore *expands* the byte budget that the service interface must carry, which is the cleanest in-thesis explanation for why the earliest split is the most expensive. Prior work on split inference for CNN backbones reports the same qualitative pattern — without representation compression, the early-stage feature maps can exceed the raw input size [30, 31]. The boundary-crossing interval reported here is, however, a composite measurement that includes remote service-B compute and serialisation/deserialisation in addition to transport (section 5.1.5), so it should not be read as a pure network-transfer cost.

The latest boundary is penalised by compute degeneracy. The split after layer4 shows the symmetric failure mode. It transferred the smallest activation tensor (98 KiB) and had the lowest boundary-crossing interval (2.22 ms), but it left only 0.486 ms of mean compute on the downstream service, approximately 0.62% of total split compute. This is well below the predeclared 10% compute-degeneracy threshold (section 4.6) and

the condition is correspondingly flagged in `carry_forward.json`. The implication is methodologically important: minimising the transferred activation alone can produce a *structurally weak* microservice decomposition that reduces transfer burden by collapsing the second service into little more than the classification head. The same pattern is described in the distributed-CNN literature, which observes that very late splits imbalance compute across the partition [15, 31]. RQ1.1 therefore retains `split_after_layer4` as evidence for the transfer-size gradient, not as a credible carry-forward candidate.

The carry-forward region is mid-to-late. With the earliest and latest boundaries excluded for the reasons above, the strongest RQ1.1 candidates are the two mid-to-late boundaries. The raw-fastest non-degenerate split is `split_after_layer3` at 79.08 ms, which is selected as the main carry-forward candidate by the frozen `carry_forward.json`; `split_after_layer2` is retained as the reference at 80.12 ms. The difference between these two is modest (approximately 1.04 ms, around 1.3% of the slower mean) and should not be read as a general ranking of `layer3` over `layer2` across all environments. The defensible conclusion is narrower: under RQ1.1’s controlled local conditions, the credible carry-forward region lies around `layer2`–`layer3`, where activation transfer has already fallen substantially but downstream compute remains meaningful. This region is what RQ1.2 then refines.

Validation. Three pieces of evidence support treating the within-run ordering and the carry-forward decision as credible for the measured environment. First, functional parity holds: both local and gRPC parity checks report $\text{max_abs_diff} = 0.0$ at the predeclared tolerance of 1.0×10^{-5} across all four split conditions (`parity_validation.json`), so latency differences are not confounded with numerical drift. Second, round-level behaviour is stable: the per-condition round coefficient of variation lies between 0.002 (monolithic) and 0.011 (`split_after_layer2`) in `round_consistency.json`, in line with the benchmarking-practice guidance to report round-level consistency rather than mean alone [13]. Third, host controls (single physical core, thread counts pinned to one, performance governor, turbo disabled) are applied symmetrically across all conditions (table 5.1), so the comparison is internally fair. These three legs of the validation argument support the within-run reading; they do not establish host-independent generality, because RQ1.1 uses one host, one build, one fixed input seed, CPU-only batch-one inference, and localhost gRPC, and cross-host or cross-seed replication is not in scope for this stage.

Evaluation. The evaluative reading of RQ1.1 has two facets. Against the monolithic baseline, every split is measurably slower, with relative overheads of +4.7%, +3.7%, +2.4% and +3.0% for splits after `layer1`, `layer2`, `layer3` and `layer4` respectively (table 5.2). RQ1.1 is therefore not a result about whether decomposition is faster; it is a result about which decomposition is least costly while preserving a meaningful microservice. Within the split set, the carry-forward rule produces a defensible main–reference pair whose within-run per-iteration 95% confidence intervals do not overlap (table 5.2). That separation is a within-run property and is not a host-independent uncertainty estimate,

so it justifies carrying both `split_after_layer3` and `split_after_layer2` forward to RQ1.2 rather than committing to a single boundary at this stage.

Answer to RQ1.1. RQ1.1 provides direct but scoped evidence that the most suitable coarse two-part ResNet-18 microservice decompositions under controlled local execution are not the earliest or latest stage boundaries. The earliest boundary is penalised because the layer-1 activation expands the byte budget the boundary must carry, exceeding the raw input size in this run. The latest boundary is penalised because it satisfies the activation-size criterion only by reducing the downstream service to roughly the classification head, falling below the predeclared 10% compute-degeneracy threshold. The strongest carry-forward candidate from this frozen run is `split_after_layer3`, with `split_after_layer2` retained as the reference candidate. Suitability under RQ1.1 conditions is therefore determined by the *joint* balance of latency, boundary-crossing cost, activation size and compute distribution, not by any single dimension — the two-dimensional reading developed further in section 6.1.3.

6.1.2 RQ1.2: Fine-Grained Refinement

RQ1.2 asks whether refining the carried-forward late-stage region from RQ1.1 with a block-level boundary changes the picture of which split is most suitable under controlled local execution. The analytical contribution of this section is not to repeat the per-condition latencies tabulated in table 5.4, but to extract the one inference that RQ1.1 could not make: at constant activation transfer, compute placement around the boundary is an independently observable cost dimension. The RQ1.2 result is therefore a controlled refinement of RQ1.1's joint reading rather than a replacement of it; it confirms the carried-forward region as a flat-bottomed valley and provides the controlled-contrast leg of the two-dimensional explanation that section 6.1.3 then synthesises. The design itself — staged refinement inside a carried-forward region rather than a fresh global search — follows the partitioning-literature practice of treating split-point selection as a hierarchical procedure over a small set of architecturally meaningful boundaries [18, 27, 32].

The refined region is a band, not a peak. The three refined split conditions fall in a 0.235 ms window (80.157 ms, 80.348 ms, 80.392 ms in table 5.4), all between +3.34% and +3.64% over the monolithic baseline. The within-run per-iteration 95% confidence intervals on the mean are [79.998, 80.316] for `split_after_layer3_block0`, [80.192, 80.504] for `split_after_layer2`, and [80.238, 80.545] for `split_after_layer3` (`condition_summaries.csv`). All three are disjoint from the monolithic CI of [77.338, 77.792], so the split-versus-monolithic overhead is robust within-run; the pairwise CIs *among* the three splits overlap, so the ordering between them is not separated at this measurement resolution. The carry-forward rule in section 4.6 nevertheless fires deterministically: the near-best 5% window admits all three splits as eligible (threshold 84.16 ms in `carry_forward.json`) and the rule promotes the raw-fastest mean to main and the next-lowest to reference. The defensible reading is therefore that RQ1.2 confirms the carried-forward region as a genuinely flat band rather than that it discovered a meaningfully better split inside that band.

Equal activation size with unequal cost isolates compute placement. The instructive comparison is `split_after_layer3_block0` against `split_after_layer3`. Both transfer exactly 200,704 B (196 KiB) of activation in FP32, yet their boundary-crossing intervals differ by 7.81 ms (33.97 ms versus 26.16 ms in `cross_condition.csv`). The predeclared boundary-crossing definition in section 4.3 is a composite that bundles remote service-B compute with serialisation, gRPC transport, and deserialisation. The per-segment compute decomposition in `cross_condition.csv` shows service-B mean compute of 31.87 ms for `split_after_layer3_block0` versus 24.06 ms for `split_after_layer3` — a 7.81 ms gap — and a non-compute residual of approximately 2.1 ms for both. The boundary-crossing difference is therefore the service-B compute difference on the wire-cost-equal pair, not a transport or serialisation difference. Because activation transfer is held constant by construction, the proximate explanatory factor must be the per-block compute distribution around the cut. This is the cleanest within-run evidence the thesis offers for treating compute placement as an independent dimension alongside activation-transfer size, and it is consistent with the prior observation that block-level execution time can vary across candidate boundaries even when their activation footprints are similar [20]. The composite-measurement framing also matches the per-layer profiling logic on which Neurosurgeon’s original split-point cost model rests [15].

The refinement does not reshuffle the carry-forward recommendation in any practically meaningful sense. The RQ1.2 carry-forward rule promotes `split_after_layer3_block0` to main and demotes `split_after_layer3` from RQ1.1’s main to RQ1.2’s rejected position (`carry_forward.json`). At first glance this is a change of recommendation; closer inspection shows it is a tie-break inside an operationally indistinguishable set. The mean separation between the promoted and demoted conditions is 0.235 ms (0.29%), smaller than the per-condition round range of `split_after_layer3_block0` itself (1.46 ms across the five measured rounds in `round_consistency.json`). The block-level winner is also the noisiest of the three splits round-to-round (round CV 0.0070, against 0.0046 and 0.0033 for the two coarse anchors). The thesis-facing reading is therefore not that RQ1.2 supersedes RQ1.1’s split choice. It is that the late-stage carried-forward region is an operationally flat band of practically equivalent boundaries, that the carry-forward rule selects deterministically within that band, and that any of the three eligible splits is a defensible carry-forward candidate at this measurement resolution.

Validation. Three pieces of evidence support treating the RQ1.2 within-run findings as credible for the measured environment. First, functional parity is intact: both local and gRPC parity checks report `max_abs_diff = 0.0` at the predeclared tolerance of 1.0×10^{-5} for `layer2`, `layer3.0`, and `layer3` (`parity_validation.json`), so the 7.81 ms boundary-crossing gap between the two 196 KiB conditions is not confounded with numerical drift across the partition. Second, round-level consistency is in line with RQ1.1: per-condition round CVs lie between 0.0017 (monolithic) and 0.0070 (`split_after_layer3_block0`) in `round_consistency.json`, supporting the reporting practice of combining per-iteration and round-level statistics rather than mean alone [13]. Third, host controls and the measurement contract are identical to RQ1.1 (table 5.3), so the comparison with RQ1.1 is internally fair on infrastructure. Two bound-

aries to this validation must be stated explicitly. The 0.235 ms span between the three split means is narrower than the largest per-condition round range (1.46 ms for the selected main), so the ordering among the three splits is not robust round-to-round; only the split-versus-monolithic separation is. And, as in RQ1.1, the run is single-host, single-build, fixed-seed, CPU-only batch-one localhost gRPC, so cross-host or cross-seed replication is not in scope.

Evaluation. The evaluative reading of RQ1.2 has two facets. Against the monolithic baseline, refinement does not change the qualitative finding of RQ1.1: every refined split is measurably slower (between +2.59 ms and +2.83 ms; +3.34% to +3.64%), with the three split CIs disjoint from the monolithic CI. Within the refined set, the evaluative question is not which split is fastest — the three are not separated at the within-run per-iteration CI level — but what the contrast reveals. The controlled equal-activation pair attributes a 7.81 ms boundary-crossing difference essentially in full to per-block compute distribution, which is the single piece of evidence the staged design was constructed to produce. This is a methodological gain for RQ1.3 even though it is not a latency gain for any deployment, and it justifies carrying both `split_after_layer3_block0` (the rule-selected main) and `split_after_layer2` (the reference) into the subsequent Kubernetes and Azure stages rather than committing to a single boundary at this resolution.

Answer to RQ1.2. RQ1.2 provides direct but scoped evidence that block-level refinement inside the carried-forward late-stage region from RQ1.1 does not overturn the coarse-level recommendation in any practically meaningful sense, and that the most useful refined finding is structural rather than rank-order. The three refined splits cluster in a 0.235 ms band whose internal ordering is not separated at the per-iteration 95% CI level and is narrower than the round-to-round range of the rule-selected main, so the carry-forward outcome — `split_after_layer3_block0` as main and `split_after_layer2` as reference, with `split_after_layer3` rejected — should be read as a deterministic rule output over operationally equivalent options rather than as a separation. The analytically load-bearing finding is the controlled `split_after_layer3_block0` vs. `split_after_layer3` contrast at constant 196 KiB activation: holding transfer constant by construction, the 7.81 ms boundary-crossing gap is essentially the service-B compute gap, with a non-compute residual that is the same for both conditions. Under RQ1.2's conditions, compute placement is therefore an independently observable cost dimension at the boundary, consistent with prior partitioning evidence that block-level execution time can vary at equal activation size [20]. This is the controlled inference that RQ1.1 could not make on a joint reading and is the empirical basis on which section 6.1.3 builds its two-dimensional explanation.

6.1.3 RQ1.3: Explanatory Synthesis

RQ1.3 asks how activation-transfer size and compute distribution together explain the latency and boundary-crossing behaviour observed for the decomposition choices in RQ1.1 and RQ1.2. The analytical contribution of this section is not to add a measurement

but to fuse the two preceding readings into a single, scoped explanatory model and to state precisely what that model does and does not answer. RQ1.1 gave the joint reading across four coarse boundaries but could not separate the two dimensions because activation size moved with stage depth across all four conditions. RQ1.2 supplied the controlled contrast at constant activation size that RQ1.1 lacked. The supporting internal-timing study (section 5.3) supplies the structural mechanism that links both observations to the architecture of ResNet-18 [11].

A two-dimensional account is the smallest model that fits both stages. At the coarse level (table 5.2), the boundary-crossing interval decreases monotonically from 53.81 ms to 2.22 ms as the activation transferred at the boundary decreases from 802,816 B to 100,352 B (rq1_1_20260515_111428/condition_summaries.csv). At the refined level (table 5.4), holding activation transfer constant at 200,704 B isolates a 7.81 ms boundary-crossing gap between `split_after_layer3_block0` and `split_after_layer3` that the per-segment decomposition attributes essentially in full to service-B compute, with a non-compute residual that is the same for both conditions (rq1_2_20260515_112636/cross_condition.csv). Neither dimension on its own fits both observations: activation transfer is well-fit to the coarse gradient but cannot account for the wire-cost-equal asymmetry, and compute placement accounts for the wire-cost-equal asymmetry but cannot account for the order-of-magnitude transfer gradient between coarse boundaries. The two-dimensional account that treats activation transfer and compute placement as *complementary* dimensions is therefore the smallest model consistent with both stages, and it is consistent with the joint framing already established in the split-inference literature for both activation-driven and compute-driven failure modes [15, 18, 31].

The structural compute profile of ResNet-18 supplies a plausible mechanism. The supporting internal-timing study reports that `layer4` accounts for 31.28% of monolithic forward-pass compute, the `layer3.1` block accounts for 9.92% (7.48 ms), and the classification head accounts for 0.44% (internal_timing_resnet18_20260515_113614/summary.csv; table 5.6 and fig. 5.2). Two predictions follow. First, a split after `layer4` leaves the downstream service with only the classification head and is therefore expected to be compute-degenerate; this is consistent with the 0.62% downstream-compute share measured in RQ1.1 (carry_forward.json). Second, the wire-cost-equal contrast in RQ1.2 moves precisely the `layer3.1` block across the boundary; the monolithic cost of that block (7.48 ms) and the measured service-B compute gap (7.81 ms) agree to within roughly 4%. The agreement should not be read as identity, because the two values come from separate runs (instrumented monolithic profiling versus split benchmarking) and the residual difference is within the range explained by run-to-run variability and instrumentation. It does, however, supply a structural reason why the two wire-cost-equal conditions differ by the magnitude they do, consistent with the prior observation that block-level execution time can vary across candidate boundaries even when their activation footprints are similar [20].

Why the carry-forward valley is mid-to-late. The two-dimensional account also predicts the shape of the carry-forward region. The earliest boundary is excluded because

the layer-1 activation expands the byte budget the boundary must carry, exceeding the raw input size in this run — the activation-transfer dimension is dominant on the early side. The latest boundary is excluded because almost all meaningful computation sits before layer4 in the architecture — the compute-distribution dimension is dominant on the late side. The mid-to-late region around layer2, layer3.0, and layer3 is the only band in which both dimensions are simultaneously tolerable, and it is the band that the carry-forward rule selects from in both stages (`carry_forward.json` in both frozen runs). RQ1.2 further shows that this band is operationally flat at the available measurement resolution: the three refined splits cluster in a 0.235 ms window whose internal ordering is not separated at the per-iteration 95% CI level, and the mean separation between the rule-selected main and the rejected condition is smaller than the round-to-round range of the rule-selected main itself (`round_consistency.json`). The two-dimensional account therefore reframes the RQ1.2 tie-break: it is the rule choosing legitimately within a genuinely flat valley, not a discovery that `split_after_layer3_block0` is meaningfully faster than the other two refined candidates.

Validation. The synthesis rests on the union of the RQ1.1 and RQ1.2 validation arguments together with the validation of the supporting internal-timing run. Functional parity holds across every split used in the synthesis (`max_abs_diff = 0.0` at tolerance 1.0×10^{-5} , `parity_validation.json` in both frozen runs), so the 7.81 ms wire-cost-equal gap and the coarse boundary-crossing gradient are not confounded with numerical drift across the partition. Round-level consistency remains low in both stages (per-condition round CV between 0.002 and 0.011 in RQ1.1 and between 0.0017 and 0.0070 in RQ1.2, `round_consistency.json`), which is the reporting practice on which the cross-stage comparison rests [13]. The internal-timing run is itself functionally equivalent (`validation.json`) with measured instrumentation overhead of 0.708 ms (0.94% of total forward-pass time, `instrumentation_overhead.json`), so the structural percentages used in the mechanism argument are not materially distorted by profiling. Three limits must be stated. First, the synthesis inherits the scope of the underlying runs: single host, single build, fixed seed, CPU-only batch-one localhost gRPC; the order-of-magnitude correspondence between architectural cost shares and measured gaps on this host is not evidence of host-independent generality. Second, the 7.48 ms versus 7.81 ms agreement is across two runs of different instrumentation; it bounds the mechanism, it does not prove it. Third, the boundary-crossing metric is a composite that bundles service-B compute with serialisation, transport, and deserialisation (section 4.3); the two-dimensional account works because the wire-cost-equal contrast isolates the compute component, not because boundary-crossing has been decomposed into independent transport and compute channels in the general case.

Evaluation. The evaluative reading of RQ1.3 has three facets. First, against the monolithic baseline, the two-dimensional account does *not* identify a configuration that is faster than monolithic execution under RQ1.1 and RQ1.2 conditions — every split is measurably slower (+2.4% to +4.7% in RQ1.1; +3.34% to +3.64% in RQ1.2). The contribution of RQ1.3 is therefore explanatory, not optimisation-oriented. Second, against a single-dimension decision rule, the joint criterion is strictly stronger: a transfer-only rule would

select `split_after_layer4` and produce a structurally degenerate microservice, and a compute-only rule has no principled mechanism for rejecting `split_after_layer1`. Third, against the carry-forward outcome, the joint criterion ratifies the rule's selection of the mid-to-late band without ratifying any particular split inside it: the rule's tie-break is a deterministic choice within an operationally flat valley, and the defensible carry-forward set is the pair `split_after_layer3_block0` (rule-selected main) plus `split_after_layer2` (reference). RQ1.3 does not establish that this carry-forward set is host-independent; it establishes that it is the set the joint criterion identifies under these conditions.

Answer to RQ1.3. Under the controlled local conditions of RQ1.1 and RQ1.2, the latency and boundary-crossing behaviour of the evaluated splits is well explained by a two-dimensional model: activation-transfer size dominates at the coarse level and accounts for the monotonic boundary-crossing gradient across stage boundaries [31], and compute placement at the boundary is independently observable at the refined level and accounts for the wire-cost-equal asymmetry between `split_after_layer3_block0` and `split_after_layer3` [20]. The supporting internal-timing study supplies a plausible structural mechanism that links both observations to the late-stage compute concentration of ResNet-18 [11]: the same architectural feature that makes `layer4` a degenerate split, because almost no computation remains downstream of it, also makes `layer3.1` a costly block to relocate across the boundary, because its monolithic cost approximates the measured wire-cost-equal gap. The carry-forward region selected by the rule in both stages — the mid-to-late band around `layer2` to `layer3`, consistent with the clustering of suitable boundaries at architecturally meaningful points reported in the split-inference literature [18] — is the band in which both dimensions are simultaneously tolerable, and the carry-forward outcome `split_after_layer3_block0` (main) plus `split_after_layer2` (reference) is the rule's deterministic selection within that band rather than a separation discovered by RQ1.2. These claims are scoped to single-host, single-build, fixed-seed, CPU-only batch-one localhost gRPC; whether the joint criterion is shaped by infrastructure context is the empirical question that the later Kubernetes and Azure stages address.

6.1.4 Synthesis: Architectural Split Choice

The three preceding subsections establish the local architectural split-choice stage of the thesis. The two-dimensional explanatory account developed in section 6.1.3 — activation-transfer size at the coarse level and compute placement at the boundary at the refined level — is not restated here; the role of this subsection is to make the section's contribution to the rest of the thesis explicit and to set the reading rule against which the Kubernetes and Azure stages are interpreted.

The local stage contributes two things forward. First, an empirically selected carry-forward set — `split_after_layer3_block0` (rule-selected main) and `split_after_layer2` (reference) — which fixes the boundary configurations that the subsequent stages re-measure rather than re-search. Second, a joint suitability criterion in which activation-transfer size and compute placement are complementary, not competing, dimensions; this criterion rejected the two coarse endpoints (`split_after_layer1` on activation

expansion, `split_after_layer4` on compute degeneracy) and identified the mid-to-late band as the only region in which both dimensions are simultaneously tolerable under the conditions measured here. Both contributions are properties of the CPU-only batch-one localhost-gRPC environment defined in tables 5.1 and 5.3 and remain bounded by the single-host, single-build, fixed-seed scope of those runs.

What the local stage does *not* establish is whether the joint criterion continues to identify the same region once Kubernetes networking, sidecar mediation, or managed-cloud node variability is added to the path. This is the empirical question that the Kubernetes (RQ1.4) and Azure (RQ1.5) stages are designed to answer. The reading rule against which their results are interpreted in section 6.2 is trend transfer rather than millisecond identity: the comparison preserves the ranking and direction of the local explanatory account where the underlying mechanism is plausibly stable across environments, and treats absolute latency shifts as legitimate consequences of the new substrate [6, 13].

6.2 Deployment Context

6.2.1 RQ1.4: Local Kubernetes Chain

RQ1.4 asks how end-to-end inference cost changes when ResNet-18 is decomposed into synchronously chained coarse architectural stages under in-cluster Kubernetes execution. The analytical contribution of this subsection is not to repeat the per-condition latencies tabulated in table 5.9, but to extract what the local Kubernetes benchmark adds on top of the on-host findings of sections 6.1.1 to 6.1.3 and what it deliberately does not establish. Two analytical points carry the subsection. First, the architectural ordering established on 127.0.0.1 in RQ1.1 (more synchronously chained boundaries imply more end-to-end cost in this ResNet-18 family [11]) remains visible once the workload is lifted into a Kubernetes pod network, but the per-hop surcharge is not a constant. Second, the chain overhead decomposes recognisably into a roughly linear platform component and a tensor-payload-dependent component, in line with the two-dimensional account synthesised in section 6.1.3 [18, 31].

The architectural ordering survives the move into Kubernetes with measurable cost.

Under the controlled local kind setup, mean end-to-end latency increases monotonically from 81.066 ms for `monolithic_k8s_1svc` to 83.387 ms, 87.728 ms, 90.572 ms, and 92.535 ms for `chain_2svc` through `chain_5svc` (table 5.9). Relative to the Kubernetes monolithic baseline, the overhead grows from +2.321 ms (+2.9%) to +11.469 ms (+14.1%). The smallest non-monolithic overhead is therefore `chain_2svc` at +2.9%, an empirical floor for this predefined chain family rather than a carry-forward selection: as declared in section 4.6 and recorded in the frozen `carry_forward.json`, the carry-forward rule does not apply to RQ1.4 because the Kubernetes condition set is predefined rather than selected. The analytical reading is therefore continuity with RQ1.1 under a new platform: the same ResNet-18 chain family that showed a monotonic split-cost gradient on 127.0.0.1 continues to do so in-cluster, with a worst-case overhead bounded at +14.1% on this hardware.

The marginal cost of an additional boundary is shaped by the tensor moved at that boundary, not by a constant per-hop surcharge.

The step from `chain_4svc` to `chain_5svc` adds approximately 1.96 ms of mean latency, smaller than the preceding 2.84 ms step from `chain_3svc` to `chain_4svc` (table 5.9). The recorded tensor-byte profile in the frozen run explains the shrinkage: the runner's `total_activation_bytes_mean` rises from 1960 KiB to 2058 KiB across these two conditions, so the additional boundary that distinguishes `chain_5svc` from `chain_4svc` transfers the 98 KiB `layer4` output. This is the activation-transfer dimension of section 6.1.3 reappearing under Kubernetes [31]: a later boundary that moves a small tensor adds less than an earlier boundary that moves a larger one, even when both are synchronous gRPC hops. The reading should be stated narrowly: it is a property of the evaluated coarse-stage ResNet-18 chain family under this local benchmark, not a universal law about microservice depth.

The platform overhead is separable from compute and scales with hop count.

Table 5.10 reports the frozen runner's `non_compute_overhead_ms_mean` alongside

the per-condition overhead and the recorded tensor totals. The non-compute / platform column grows from 6.215 ms (chain_2svc) through 9.435 ms, 12.088 ms, and 13.633 ms (chain_5svc), while total_compute_ms_mean in merged_results/condition_summaries.csv is approximately flat across the chained conditions (77.17–78.90 ms versus 77.63 ms for the monolithic baseline). Most of the added latency is therefore not extra computation: it is per-request work attributable to the synchronous gRPC hop and the surrounding Kubernetes plumbing (kubelet, kube-proxy, the pod-network path, gRPC framing and deserialisation). This pattern is consistent with the broader characterisation of microservice RPC cost: layered gRPC / HTTP/2 / TCP / IP stacks add measurable per-call overhead that is not absorbed by collocation, and serialisation and kernel-network traversal are a real share of microservice latency even between same-host processes [2, 37]. Two caveats bound this comparison. The platform column is the runner’s composite non_compute_overhead_ms_mean; it is not decomposed into kernel, kube-proxy, CNI, and gRPC sub-shares, and the RQ1.4 evidence does not support such a decomposition. The cited papers measure the RPC-tax share in different workloads (a key-value store and the DeathStarBench hotel-reservation application), not chained ResNet-18 inference, so they support the qualitative reading that this column is non-trivial and grows with chain depth, not the specific millisecond magnitudes observed here.

Scope of validity: single-node local Kubernetes. The deployment metadata records that for every condition, all service pods and the benchmark client were scheduled on the single node thesis-rq14-control-plane of the local kind cluster (merged_results/deployment_metadata.json). The platform overhead measured above therefore already includes the in-cluster gRPC path and the kubelet-mediated pod network, but it does not include a cross-node hop, a heterogeneous-node CNI traversal, or any of the sources of variability that managed cloud Kubernetes introduces. Cross-node sensitivity and managed-cloud transfer are deferred to sections 6.2.3 and 6.2.4 respectively; RQ1.4 itself should be read as a valid local in-cluster benchmark of the predefined chain family, not as a statement about deployment behaviour in production Kubernetes.

Validation. Three pieces of evidence support treating the within-run ordering and the platform/compute decomposition as credible for this local environment. First, functional parity is intact: both the local segment-chain check and the live in-cluster gRPC chain check report max_abs_diff = 0.0 at the predeclared tolerance of 1.0×10^{-5} across all five conditions (parity_validation.json), so the chain latencies are not confounded with numerical drift across the partitioned graph. Second, round-level behaviour is stable: per-condition round CVs lie between 0.0074 (chain_5svc) and 0.0161 (chain_3svc) in round_consistency.json (table 5.12), in line with the reporting practice of combining per-iteration and round-level statistics rather than mean alone [13]. Third, infrastructure controls are recorded and applied symmetrically across conditions: image (thesis-inference:latest, pull policy IfNotPresent), namespace (rq14), node, resource requests and limits (1 CPU, 512 MiB per service pod; 1 CPU, 1 GiB client), threading, CPU affinity to core 0, and nice -5 are identical across the five conditions (merged_results/deployment_metadata.json, merged_results/environment.json), so the comparison is internally fair on infrastructure. Three limits must be stated expli-

citly. The image tag `thesis-inference:latest` is a mutable Docker tag, so byte-exact reproduction of the local `kind` run requires re-pulling and re-tagging the corresponding ACR digest used in the AKS stages; the local result is therefore reproducible only up to whatever image was loaded into the `kind` cluster at run time (section 4.8). Governor and turbo controls were requested but writes to `/sys` failed because the containerised environment exposes those paths as read-only; the detected governor was already performance and turbo was already disabled, so the comparison is internally fair but governor/turbo are observed rather than enforced. Finally, RQ1.4 is a single host, single `kind` cluster, single-node placement, fixed seed; cross-host or repeated clean-cluster replication is not in scope for this stage.

Evaluation. The evaluative reading of RQ1.4 has three facets. First, against the Kubernetes monolithic baseline, every chained condition is measurably slower at this local setup, with relative overheads of +2.9%, +8.2%, +11.7%, and +14.1% for `chain_2svc` through `chain_5svc` (table 5.9) and effect sizes of Cohen’s $d = 0.614, 1.614, 2.392,$ and 2.874 respectively (table 5.11). The Mann-Whitney p -values in table 5.11 are near zero because the pooled per-iteration sample size is large ($N = 1,000$ per condition); as noted in the methodological note in section 5.4, they should not be read as evidence of strong practical significance on their own, and the round-level CVs and parity checks are the primary indicators of result stability. Second, against a useful counterfactual, the smallest non-monolithic overhead is `chain_2svc` at +2.321 ms (+2.9%); within the predefined family it is therefore the least expensive way to move from a single Kubernetes service to a chained deployment while preserving exact functional parity in the benchmarked workload. Third, against the boundaries of what RQ1.4 can claim, the result is sufficient to support a local-in-cluster feasibility claim — the chained ResNet-18 family runs end-to-end, with bounded overhead and without functional divergence from the monolithic baseline — but it is not sufficient to claim production readiness, scalability, multi-tenant robustness, or portability of the absolute millisecond values. Cross-node sensitivity, managed-cloud transfer, and service-mesh hardening are addressed in sections 6.2.3 and 6.2.4 and in the security analysis subsections respectively; RQ1.4’s contribution stops at the local-feasibility boundary.

Answer to RQ1.4. Under the controlled local Kubernetes setup, increasing the number of synchronously chained coarse-stage ResNet-18 services raises end-to-end latency from `monolithic_k8s_1svc` through `chain_5svc`, with mean latency increasing from 81.066 ms to 83.387 ms, 87.728 ms, 90.572 ms, and 92.535 ms. The smallest non-monolithic overhead is `chain_2svc` at +2.321 ms (+2.9%), and the worst-case overhead is `chain_5svc` at +11.469 ms (+14.1%). The marginal cost of an added boundary is not a constant per-hop surcharge: the final increment from `chain_4svc` to `chain_5svc` is approximately 1.96 ms, smaller than the preceding 2.84 ms step, because the additional boundary moves only the 98 KiB layer4 output. Most of the added latency is platform-side rather than compute-side: total compute is approximately flat across the chained conditions while the recorded non-compute / platform overhead grows from 6.215 ms to 13.633 ms, consistent with the measurable per-hop cost of layered gRPC microservice communication [2, 37]. Carry-forward is not applicable in this stage because the Kubernetes condition set

was predefined rather than selected. These claims are scoped to a single-node local kind cluster with all pods colocated on the `thesis-rq14-control-plane`; whether the ordering and the bounded marginal cost transfer to managed cloud Kubernetes and to multi-node placement is the empirical question that sections 6.2.2 to 6.2.4 address.

6.2.2 RQ1.4b: kind Multi-Worker CNI Sensitivity

RQ1.4b is an explicitly supporting sensitivity stage rather than a primary thesis-facing measurement: it re-runs the RQ1.4 chain family on a six-worker kind cluster with pod anti-affinity forcing every chain segment onto a distinct kind worker container and the benchmark client onto a sixth dedicated worker (section 4.7.5). The analytical contribution is bounded by a methodological reading carried over from section 4.7.5: kind's worker "nodes" are Docker containers that share one host kernel and communicate over kind's CNI bridge; they are not separate physical or virtual hosts. RQ1.4b therefore measures the cost of routing inter-service gRPC through the kind CNI bridge under anti-affinity, not the cost of real cross-host networking. The genuine multi-host inter-node penalty for the same chain family is measured in section 6.2.4; the role of RQ1.4b is to bound how much the single-node placement of RQ1.4 hides on the *same physical host*, and to test whether the RQ1.4 architectural ordering survives when the chain traffic is forced through the kind CNI bridge.

The architectural ordering and the bounded-non-linearity pattern survive the kind-CNI probe. Under the kind multi-worker condition, mean end-to-end latency increases monotonically from 67.491 ms for `monolithic_k8s_1svc` to 87.378 ms, 120.465 ms, 135.639 ms, and 141.169 ms for `chain_2svc` through `chain_5svc` (table A.1). The same monotonic ordering established in RQ1.4 is therefore preserved when each chain hop is forced across a kind CNI-bridge boundary on the same host. The bounded-non-linearity reading of RQ1.4 is also preserved: the final `chain_4svc` $\rightarrow$ `chain_5svc` step adds approximately 5.530 ms, smaller than the preceding +15.174 ms `chain_3svc` $\rightarrow$ `chain_4svc` step, in line with the smaller 98 KiB layer4 activation added at the final boundary (the runner's `total_activation_bytes_mean` rises from 1960 KiB to 2058 KiB across these two conditions, matching the RQ1.4 profile). The reading is therefore narrow but useful: the RQ1.4 chain-cost gradient and the activation-driven shape of the marginal cost are not specific to single-node pod colocation; they persist once chain traffic is routed through kind's CNI bridge.

The kind multi-worker per-condition penalty is an order of magnitude larger than the AKS multi-node penalty, confirming that kind multi-node is not a substitute for cross-host networking. Computed condition-by-condition against the frozen RQ1.4 single-node baseline, the per-condition penalty $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$ is +3.991 ms for `chain_2svc`, +32.737 ms for `chain_3svc`, +45.067 ms for `chain_4svc`, and +48.634 ms for `chain_5svc` (table A.3). The monolithic condition is faster on average in RQ1.4b (−13.575 ms) but its per-iteration standard deviation rises from 4.017 ms (RQ1.4) to 10.999 ms (table A.1), its median (61.917 ms) is well below its mean (67.491 ms), and its per-round means are {62.000, 68.553, 68.677, 69.107, 69.119} ms

(table A.2, the first round is markedly faster than rounds 2–5). This monolithic shift is therefore consistent with a one-off cluster-start artefact and host co-tenancy variability, and should not be read as evidence that kind multi-worker placement is faster than single-node placement. For the chained conditions, the kind-CNI per-condition deltas are roughly an order of magnitude larger than the corresponding multi-node AKS deltas reported in section 6.2.4 (+2.131 ms to +4.086 ms across the chained conditions on AKS, table 5.19). This separation is the key analytical contribution of RQ1.4b: kind’s “multi-node” topology is not a substitute for genuine cross-host networking. CNI plugin choice and the container-network path are an independent source of measured-latency variance in Kubernetes clusters [16], and absolute timings are known not to transfer cleanly across container, VM, and bare-metal execution substrates [9]; treating the kind-multi result as a cross-host estimate would inflate the inferred inter-node cost relative to what section 6.2.4 actually measures on AKS.

Round-level dispersion inflates relative to RQ1.4 and is not chain-explained. Per-condition round CVs in RQ1.4b (table A.2) are 0.0290 (`chain_2svc`), 0.0551 (`chain_3svc`), 0.0187 (`chain_4svc`), 0.0247 (`chain_5svc`), and 0.0456 (`monolithic_k8s_1svc`), compared with the much tighter 0.0074–0.0161 range observed in single-node RQ1.4 (table 5.12). Two features matter for interpretation. First, the highest CVs do not track chain depth: monolithic (no inter-pod traffic) and `chain_3svc` (three services) are noisier than `chain_5svc` (five services), so the inflated CV is not a property of having more chain hops. Second, the monolithic round-mean trajectory above is consistent with a first-round warm-up shift, not with a steady-state difference. Both observations point to kind-CNI variability and shared-host effects on a single physical machine that hosts all six kind worker containers, rather than a chain-specific noise mechanism; this reading is consistent with the broader observation that orchestrator- and host-level contention can dominate measured deltas on shared infrastructure [28]. By the SC’15 reporting standard, round-level consistency is the primary indicator of result stability, and the round CVs above are too high to read RQ1.4b as a primary number [13]: the within-RQ ordering and the qualitative bounded-non-linearity claim remain interpretable, but the absolute millisecond magnitudes should be quoted only as upper sensitivity bounds, not as deployment-grade values.

Validation. Three pieces of evidence support a within-run reading of RQ1.4b. First, functional parity holds: both the local segment-chain check and the live in-cluster gRPC check report `max_abs_diff = 0.0` at the predeclared tolerance of 1.0×10^{-5} across all five conditions (`merged_results/parity_validation.json`, summarised alongside table A.1), so the within-RQ1.4b ordering is not confounded by numerical drift across the partitioned graph. Second, infrastructure controls are applied symmetrically across conditions: image (`thesis-inference:latest`, pull policy `IfNotPresent`), namespace (`rq14b`), resource requests and limits (1 CPU, 512 MiB per service pod; 1 CPU, 1 GiB client), threading (one thread per pool), and CPU affinity to core 0 are uniform across the five conditions (`merged_results/deployment_metadata.json`, `merged_results/environment.json`). Third, multi-node placement was enforced by the same anti-affinity rule used in RQ1.5b, with six distinct worker containers and a dedicated client worker. Four limits must be

stated explicitly. (i) The image tag `thesis-inference:latest` is a mutable Docker tag, so byte-exact reproduction is bounded by whatever image was loaded into the kind cluster at run time, the same caveat that applies to RQ1.4 (section 4.8). (ii) Host CPU controls were weaker than in RQ1.4: governor and turbo writes to `/sys` failed because those paths were read-only in the containerised environment, and the detected governor was `schedutil` with turbo enabled (rather than the performance governor and disabled turbo observed for RQ1.4). The `nice -5` request also failed for permission reasons. RQ1.4b is therefore not a clean replication of RQ1.4 with only a topology change; reduced host control is one plausible co-contributor to the higher round CVs. (iii) Comparisons against RQ1.4 are interpreted at the *per-condition* delta level (table A.3), not as a cross-stage means comparison, because the two stages differ in host control state in addition to placement. (iv) RQ1.4b is one host, one kind cluster, one fixed seed; the kind worker containers share a kernel and the same physical hardware, so cross-host replication is, by construction, out of scope and is addressed only by RQ1.5b on AKS.

Evaluation. The evaluative reading of RQ1.4b has three facets. First, the result is sufficient to support a qualitative claim: the RQ1.4 chain-cost ordering and the activation-driven bounded-non-linearity pattern are not specific to single-node pod colocation; they survive once every chain hop is forced through the kind CNI bridge. Second, the result is sufficient to bound the additional cost a switch to a kind multi-worker topology would add to the single-node RQ1.4 measurement on this host: between +3.991 ms (`chain_2svc`) and +48.634 ms (`chain_5svc`) per chained condition (table A.3), with the worst case at the deepest chain. Third, the result is not sufficient to support a deployment-grade multi-host claim, a production-readiness claim, or a scalability claim: the per-condition deltas are an order of magnitude larger than the AKS-measured multi-node deltas of section 6.2.4, the host-control state was degraded relative to RQ1.4, and the round CVs are correspondingly inflated. The headline RQ1.4 evaluative conclusion is unchanged; RQ1.4b adds a bounded sensitivity caveat, not an independent deployment claim, and motivates RQ1.5b as the appropriate evidence path for genuine inter-node cost.

Answer to RQ1.4b. Under the kind multi-worker CNI-bridge probe, the RQ1.4 architectural ordering is preserved: mean end-to-end latency increases monotonically from 67.491 ms for `monolithic_k8s_1svc` through 87.378 ms, 120.465 ms, 135.639 ms, and 141.169 ms for `chain_2svc` through `chain_5svc` (table A.1), and the bounded-non-linearity pattern survives (the `chain_4svc` → `chain_5svc` increment of +5.530 ms is smaller than the preceding +15.174 ms `chain_3svc` → `chain_4svc` step, matching the small 98 KiB `layer4` activation moved at the added boundary). The per-condition kind-CNI penalty relative to the single-node RQ1.4 baseline ranges from +3.991 ms to +48.634 ms across the chained conditions (table A.3), an order of magnitude larger than the AKS-measured multi-node penalty reported in section 6.2.4, and round-level CVs (up to 0.0551 for `chain_3svc` and 0.0456 for the monolithic baseline, table A.2) are materially inflated relative to RQ1.4. The contribution of RQ1.4b to the deployment analysis is therefore two-fold and bounded: (i) it shows that the RQ1.4 ordering and the activation-driven bounded-non-linearity are not artefacts of single-node colocation on

the same host, and (ii) it shows that kind multi-node deltas are not a substitute for a managed-cloud cross-host measurement and should be read as a kind-CNI sensitivity envelope rather than as a deployment-grade multi-node result.

6.2.3 RQ1.5: AKS Transfer Validation

RQ1.5 asks whether the local Kubernetes latency pattern observed for the coarse-stage ResNet-18 microservice chain family transfers to Azure Kubernetes Service (AKS), and how absolute and relative costs change under managed cloud execution. The analytical contribution is not to add a measurement on top of section 6.2.1, and not to re-select a split point, but to test whether the RQ1.4 architectural ordering survives a substrate change from a local single-node kind cluster to a managed AKS single-node cluster, and to scope precisely what that test does and does not establish. The natural reporting discipline for this kind of cross-substrate transfer is ranking + relative-overhead trend rather than millisecond equivalence [13].

The RQ1.4 ordering transfers directionally to AKS. The complete RQ1.4 condition ordering is preserved in the frozen AKS benchmark: `monolithic_k8s_1svc` at 70.802 ms is the fastest, followed by `chain_2svc` at 73.437 ms, `chain_3svc` at 75.410 ms, `chain_4svc` at 78.294 ms, and `chain_5svc` at 80.519 ms (table 5.14). Together with the kind multi-worker probe in section 6.2.2 (which preserves the ordering under same-host CNI-bridge routing) and the AKS multi-node sensitivity in section 6.2.4 (which preserves it under enforced inter-node placement), the RQ1.4 ordering now survives three orthogonal stresses: managed-cloud substrate (RQ1.5), CNI-bridge routing on the local host (RQ1.4b), and genuine cross-host placement (RQ1.5b). Read across the deployment strand, this is a triple-coverage transfer of the same architectural ordering established on 127.0.0.1 in RQ1.1: from a single-host loopback gRPC chain (RQ1.1), to a single-node local kind cluster (RQ1.4), to managed AKS (RQ1.5). The ordering claim of the deployment strand is therefore not specific to the single-node kind substrate on which RQ1.4 was measured.

The relative-overhead trend transfers within a small pp band. Relative to each environment's own monolithic baseline, the AKS overhead profile across `chain_2svc`–`chain_5svc` is {3.7%, 6.5%, 10.6%, 13.7%}, against the RQ1.4 local Kubernetes profile of {2.9%, 8.2%, 11.7%, 14.1%} (table 5.16). Both curves are monotonic and track each other to within 1.7 pp at any condition (the maximum gap is at `chain_3svc`: 8.2% local vs 6.5% AKS; the other three conditions agree within 0.8 pp). The transfer claim is therefore stronger than rank preservation alone: the shape of the overhead curve — not just the ordering of conditions — carries over from local Kubernetes to AKS. This shape correspondence at the relative-overhead level is what supports treating the chain-cost gradient as architectural rather than as a kind-specific artefact, while the small pp differences are compatible with the expected effect of managed-Kubernetes provider defaults on absolute performance [8].

The bounded-non-linearity at the last hop reappears with the same activation-transfer signature. The AKS marginal increments are +2.635 ms (`mono` → `chain_2svc`), +1.973 ms (`chain_2svc` → `chain_3svc`), +2.884 ms (`chain_3svc` → `chain_4svc`),

and +2.225 ms (`chain_4svc` → `chain_5svc`) (table 5.14). The final increment is smaller than the preceding one because the layer-4 boundary that distinguishes `chain_5svc` from `chain_4svc` transfers only the 98 KiB layer4 output, against the 196–784 KiB tensors moved at the earlier layer1, layer2, and layer3 boundaries (boundary-only activation sizes, consistent with the RQ1.1 metric and distinct from the runner’s input-inclusive `total_activation_bytes_mean`). This is the activation- transfer dimension of the RQ1.3 two-dimensional account [31] reappearing in a third substrate, after 127.0.0.1 in RQ1.1 and single-node kind in RQ1.4. The triple coverage strengthens the architectural reading of the bounded non-linearity but does not extend it beyond the evaluated coarse-stage ResNet-18 chain family — the caveat already attached to the RQ1.4 reading carries over to RQ1.5 unchanged.

Absolute latency is environment-specific and is not the comparison RQ1.5 makes.

Every AKS condition mean is lower than its local Kubernetes counterpart, including the monolithic baseline (70.802 ms vs 81.066 ms). The two runs differ in host CPU, kernel, virtualisation layer, Kubernetes implementation (`kind/containerd` vs managed AKS control plane), container runtime context, CPU scheduling, networking path, and applied host controls: the local stage applied performance governor, disabled turbo, and `nice -5`, whereas AKS does not expose those controls (section 5.5). The lower AKS absolute number is therefore confounded with hardware and host-control differences rather than isolated to a substrate property of AKS, and the absolute delta should not be read as evidence that AKS is generally faster than local Kubernetes for this workload. Independent of this thesis, container, VM, CNI, and managed-cluster differences are known to shift absolute timings across substrates [6, 8, 9, 16], which is why RQ1.5 uses within- environment ordering and relative-overhead transfer as the primary comparison and treats absolute latency as deployment-specific.

Validation. Three pieces of evidence support treating the within-run AKS ordering and the relative-overhead trend transfer as credible. First, functional parity holds: both local segment-chain validation and live in-cluster gRPC validation report `max_abs_diff` = 0.0 at the predeclared tolerance of 1.0×10^{-5} for all five conditions (consolidated in section 6.4), so the AKS condition means are not confounded with numerical drift across the partitioned graph. Second, cross-round stability is tight on the chained conditions: round CVs are 0.0019–0.0061 for `chain_2svc`–`chain_5svc` and 0.0117 for the monolithic baseline (table 5.17); these are in the same range as RQ1.4 (table 5.12) and meet the SC’15 reporting standard of treating cross-round consistency as the primary indicator of result stability [13]. The slightly higher monolithic round CV in AKS is compatible with managed-cloud variability landing on the single longer-running pod path rather than on the chained gRPC path, but the run does not isolate a single cause. Third, the AKS leg removes one reproducibility caveat that bounded RQ1.4: every RQ1.5 condition ran with the pinned ACR image digest `sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b` under image pull policy `Always`, so the application code is byte-exact at this digest, in contrast to the mutable local `thesis-inference:latest` tag used in RQ1.4 (section 4.8). Three limits must be stated explicitly. (i) RQ1.5 is one AKS cluster, one node SKU (`Standard_D8s_v3`), one

region (swedencentral), one fixed seed; replication across SKU, region, or vendor is not in scope. (ii) The local and AKS legs differ on host hardware and on whether the performance governor, turbo, and nice -5 could be applied, so the local-to-AKS absolute delta is confounded with host control state in addition to substrate. (iii) Inter-node placement is excluded by construction in RQ1.5 (single-node AKS); the corresponding bound is quantified in section 6.2.4.

Evaluation. The evaluative reading of RQ1.5 has three facets. First, against the within-environment AKS monolithic baseline, every chained condition is measurably slower with overhead growing monotonically from +2.635 ms (3.7%, chain_2svc) to +9.717 ms (13.7%, chain_5svc) (table 5.14). The smallest non-monolithic overhead is again chain_2svc, in agreement with the RQ1.4 reading that within the predefined family this is the least-expensive way to move from a single Kubernetes service to a chained AKS deployment while preserving exact functional parity. Second, against the RQ1.4 local Kubernetes baseline, the relative-overhead curve transfers within a small pp band (≤ 1.7 pp at any condition, table 5.16); the overhead trend therefore carries over to managed AKS even though the absolute milliseconds do not. Third, against the boundaries of what RQ1.5 can claim, the result supports a directional-transfer claim under managed-cloud execution — the chain family runs end-to-end on AKS, the RQ1.4 ordering is preserved, and the relative-overhead trend tracks the local profile — but it is not sufficient to support production-readiness, scalability, multi-tenant robustness, or millisecond-portability claims. Inter-node placement, autoscaling, concurrent clients, alternate SKUs and regions, and service-mesh hardening are all out of scope for RQ1.5; the inter-node bound is quantified in section 6.2.4, and the hardening cost is addressed in the RQ2 strand. The contribution of RQ1.5 stops at the substrate-transfer boundary.

Answer to RQ1.5. Under the controlled AKS transfer-validation setup, the local Kubernetes service-count pattern transfers directionally to managed cloud execution. The complete RQ1.4 ordering is preserved (monolithic_k8s_1svc 70.802 ms < chain_2svc 73.437 ms < chain_3svc 75.410 ms < chain_4svc 78.294 ms < chain_5svc 80.519 ms, table 5.14), and the relative-overhead profile against each environment’s own monolithic baseline tracks the RQ1.4 profile within 1.7 pp at any condition (table 5.16). The bounded non-linearity at the final boundary reappears with the same activation-transfer signature (+2.225 ms last step against a 98 KiB layer4 transfer), so the RQ1.3 activation-driven mechanism holds in a third substrate [31]. Absolute latency is lower in AKS for every condition but is confounded with hardware and host-control differences and is therefore not the comparison RQ1.5 makes; substrate-dependent absolute shifts across container, VM, CNI, and managed-cluster implementations are independently documented [6, 8, 9, 16]. RQ1.5 therefore supports a bounded directional transfer claim under single-node managed-cloud execution; it does not support production-readiness, scalability, or millisecond-portability claims, and the inter-node and security-hardening dimensions are addressed in section 6.2.4 and in the RQ2 analysis subsections respectively.

6.2.4 RQ1.5b: AKS Multi-Node Sensitivity

RQ1.5b asks how much additional end-to-end latency the inter-node network path adds when every chain hop of the predefined ResNet-18 chain family is forced across an AKS node boundary, relative to the same-day single-node RQ1.5 baseline. The analytical contribution is not to introduce a second primary deployment number on top of section 6.2.3, and not to reselect a split point, but to convert the single-node-placement validity threat that section 6.2.3 acknowledges into a measured per-condition bound, and to test whether the RQ1.4 / RQ1.5 architectural ordering and the activation-driven bounded-non-linearity pattern survive once the chain traffic actually crosses the Azure VNet. The natural reporting discipline for this kind of sensitivity comparison is ranking + cross-round consistency rather than millisecond identity [13].

Finding 1: Multi-node placement preserves the AKS condition ordering and the bounded-non-linearity pattern. With every chain segment of a condition forced onto a distinct AKS node and the benchmark client on a sixth dedicated node, mean end-to-end latency increases monotonically from 73.108 ms for `monolithic_k8s_1svc` to 76.470 ms, 77.541 ms, 82.103 ms, and 84.605 ms for `chain_2svc` through `chain_5svc` (table 5.18). The bounded-non-linearity pattern from RQ1.4 and RQ1.5 also reproduces under multi-node execution: the `chain_4svc` $\rightarrow$ `chain_5svc` increment is +2.502 ms, smaller than the preceding +4.562 ms `chain_3svc` $\rightarrow$ `chain_4svc` step (table 5.18). This is consistent with the smaller 98 KiB `layer4` output transferred at the added boundary — the activation-transfer dimension of the RQ1.3 two-dimensional account [31] reappearing now in a fourth substrate, after 127.0.0.1 (RQ1.1), `local kind` (RQ1.4), and single-node AKS (RQ1.5). The architectural ordering and its shape are therefore not artefacts of single-node colocation on AKS.

Finding 2: The inter-node penalty is positive and bounded. Table 5.19 reports the condition-by-condition multi-node penalty $\Delta_c = \text{mean}_c^{\text{multi}} - \text{mean}_c^{\text{single}}$ against the frozen single-node RQ1.5 baseline. The monolithic condition gains +2.305 ms relative to the single-node monolithic. This is compatible with the 588 KiB input tensor ($1 \times 3 \times 224 \times 224 \text{ FP32}$) moving across the Azure VNet rather than over a loopback socket once the benchmark client is forced onto a dedicated node; layered RPC stacks are known to add non-trivial transport, copying, and serialisation work on top of payload bytes themselves [37], although RQ1.5b does not decompose the 2.305 ms into stack-layer sub-shares. The chained conditions add per-condition deltas of +3.033 ms (`chain_2svc`), +2.131 ms (`chain_3svc`), +3.808 ms (`chain_4svc`), and +4.086 ms (`chain_5svc`). The multi-node penalty is therefore a measured upper bound rather than an open acknowledgement: on the evaluated chain family on this AKS configuration, the worst-case single-node-to-multi-node delta is +4.086 ms (`chain_5svc`), and the monolithic baseline alone gains +2.305 ms.

Finding 2a: The per-condition penalty is not monotonic in hop count. The chained-condition deltas do not increase smoothly with the number of inter-node hops: `chain_3svc` (+2.131 ms) carries a smaller penalty than `chain_2svc` (+3.033 ms), while `chain_4svc` (+3.808 ms) and `chain_5svc` (+4.086 ms) sit above both. Two readings are defensible at

this evidence level. First, the magnitudes are small relative to the within-condition standard deviations in table 5.18 (e.g. 2.786 ms for `chain_3svc` and 3.435 ms for `chain_2svc`), so part of the non-monotonicity is compatible with managed-cloud measurement variability rather than a systematic per-hop mechanism; public-cloud network variability is known to shift absolute latency without changing the underlying application behaviour [6, 28], and the per-condition pod-to-node assignments differ across conditions (`merged_results/deployment_metadata.json`). Second, the qualitative reading from RQ1.4 carries over: the marginal cost of an additional inter-node hop is not a constant surcharge, so summarising RQ1.5b as a single “per-hop multi-node cost” would over-read the run. The defensible summary is the *envelope* +2.131 ms .. +4.086 ms across the chained conditions and the *worst case* +4.086 ms at `chain_5svc`, not a fitted per-hop slope.

Finding 3: Round-to-round consistency is preserved across nodes. Per-condition round CVs in RQ1.5b are 0.0036–0.0127 for the chained conditions and 0.0084 for the monolithic baseline (table 5.20), comparable to and in some cases tighter than the single-node RQ1.5 chained CVs of 0.0019–0.0061 and the RQ1.5 monolithic CV of 0.0117 (table 5.17). By the SC’15 reporting standard [13], this places RQ1.5b in the same within-run-stability regime as the single-node RQ1.5 primary; the distributed placement did not introduce large round-to-round drift in this frozen run. The round CV is a within-run stability measure and does not bound cross-cluster or cross-run replication; that remains explicitly out of scope (section 4.7.7).

Finding 4: kind multi-worker is not a substitute for the RQ1.5b measurement. Read together with section 6.2.2, the RQ1.5b per-condition deltas (+2.131 ms .. +4.086 ms across the chained conditions, table 5.19) are roughly an order of magnitude smaller than the corresponding kind multi-worker deltas of +3.991 ms .. +48.634 ms reported in table A.3. The two stages also differ in host-control state, so the comparison is qualitative rather than a clean isolation of CNI cost; CNI plugin choice and the pod-network path are nonetheless an independent and well-documented source of measured-latency variance in Kubernetes clusters [16], and absolute timings are known not to transfer cleanly across container, VM, and managed-cluster substrates [8, 9]. The analytical consequence for the deployment strand is therefore that RQ1.4b should be read as a kind-CNI sensitivity envelope on the single-node RQ1.4 baseline, and RQ1.5b should be read as the credible cross-host bound on the single-node RQ1.5 baseline; treating the RQ1.4b numbers as a cross-host estimate would inflate the inferred inter-node cost by an order of magnitude relative to what AKS actually measures here.

Validation. Four pieces of evidence support treating the per-condition multi-node penalty as credible within the bounds RQ1.5b sets. First, functional parity holds: both the local segment-chain check and the live in-cluster gRPC chain check report `max_abs_diff = 0.0` at the predeclared tolerance of 1.0×10^{-5} across all five conditions (`merged_results/parity_validation.json`), so the per-condition means and deltas are not confounded by numerical drift across the partitioned graph. Second, multi-node placement was enforced and verified post-deployment: per-condition pod placements in `merged_results/deployment_metadata.json`

are pairwise disjoint across nodes and the benchmark client is always on a node distinct from every service pod, and the run-level `multi_node_validation.passed=True` entry in `merged_results/environment.json` aggregates that check (section 4.7.7). Third, the AKS leg is byte-exact reproducible at the application-code level: every RQ1.5b condition ran with the same pinned ACR image digest used in RQ1.5 (`sha256:d292aef407ee8a15da2bdaa680de1cc24b0975e7de81d27e1fc3a7d128b8736b`) under image pull policy `Always` (section 4.8), so the single-node-to-multi-node delta is not contaminated by application-code drift between the two stages. Fourth, by the SC'15 reporting standard the cross-round CV range `0.0036–0.0127` is in the same stability regime as the single-node RQ1.5 frozen run [13]. Five limits must be stated explicitly. (i) RQ1.5b is one AKS cluster, one node SKU (`Standard_D8s_v3`), one region (`swedencentral`), one node pool (`rq15bpool`), one CNI (`kubenet`), one fixed seed; cross-SKU, cross-region, cross-CNI, or cross-vendor replication is not in scope, and absolute managed-cloud performance is known to shift with each of those axes [8, 9, 16]. (ii) The benchmark client's `nice -5` request and CPU-governor / turbo controls could not be applied inside the AKS pod environment (`merged_results/environment.json: priority.applied=false, governor.detected=unavailable`); host-control state is therefore weaker than in RQ1.4 and matches the same caveat declared for RQ1.5. (iii) Per-condition pod-to-node assignments differ across conditions, so the small non-monotonicity in the per-condition deltas (table 5.19) is compatible with run-internal placement-locality variability and is not isolated to chain depth. (iv) The `+2.305` ms monolithic-baseline gain is interpreted at the mechanism level (VNet vs loopback for the 588 KiB input); the run does not decompose this gain into transport, copy, serialisation, or kernel-network sub-shares, and [37] measures the RPC-tax share in different workloads, so it supports the qualitative mechanism, not the specific 2.305 ms magnitude. (v) The run is one execution of the chain family on this cluster; round-level stability bounds within-run variability but not across-run cluster-level replication.

Evaluation. The evaluative reading of RQ1.5b has three facets. First, against the analytical purpose of the stage — bounding the inter-node cost that the single-node RQ1.5 design intentionally hides — the result is sufficient. On this AKS configuration the worst-case multi-node penalty across the chained conditions is `+4.086` ms (`chain_5svc`, table 5.19) and the monolithic baseline gains `+2.305` ms; the single-node RQ1.4 / RQ1.5 design therefore underestimates true production-style latency by a bounded margin that is now quantified per condition rather than acknowledged abstractly. Second, against the RQ1.5 ordering and bounded-non-linearity claims, the result is confirmatory: table 5.18 reproduces the RQ1.5 ordering and the activation-driven last-step pattern under enforced cross-host placement, which strengthens the RQ1.3 / RQ1.5 architectural reading without extending it beyond the evaluated coarse-stage ResNet-18 chain family. Third, against the boundaries of what RQ1.5b can claim, the result is not sufficient to support production-readiness, scalability, multi-tenant robustness, autoscaling, alternate-SKU, alternate-region, alternate-CNI, or millisecond-portability claims. Concurrent clients, service-mesh hardening, autoscaling, alternate node SKUs and regions, and alternate CNI choices are out of scope for RQ1.5b; the service-mesh-hardening cost is addressed in the RQ2.1 / RQ2.1b strand, and managed-cloud absolute drift across substrates is well-documented independently [6, 8, 9, 16]. The contribution of RQ1.5b stops at the

inter-node-cost boundary it was designed to measure.

Answer to RQ1.5b. Under enforced multi-node placement on the same AKS cluster, the same five-condition predefined chain family preserves the monotonic overhead progression seen in RQ1.4 and RQ1.5: per-condition mean latency increases from 73.108 ms (`monolithic_k8s_1svc`) to 84.605 ms (`chain_5svc`, table 5.18), and the `chain_4svc` → `chain_5svc` step is +2.502 ms against a preceding +4.562 ms step, in line with the smaller 98 KiB `layer4` activation transferred at the added boundary. The per-condition multi-node penalty relative to the single-node RQ1.5 baseline ranges from +2.131 ms to +4.086 ms across the chained conditions (table 5.19) and is +2.305 ms for the monolithic baseline; the penalty is not monotonic in hop count, with `chain_3svc` sitting below `chain_2svc` at this run's pod placements, and is best summarised as an envelope rather than a fitted per-hop slope. Cross-round CVs are 0.0036–0.0127 (table 5.20), in the same stability regime as the single-node RQ1.5 frozen run [13]. Read against section 6.2.2, these per-condition deltas are an order of magnitude smaller than the kind multi-worker CNI-bridge deltas of +3.991 ms .. +48.634 ms (table A.3), which confirms that kind multi-worker is a CNI sensitivity envelope rather than a substitute for the managed-cloud cross-host bound that RQ1.5b actually measures [9, 16]. The single-node RQ1.4 / RQ1.5 design therefore underestimates true production latency by a bounded margin that is now quantified per condition on this AKS configuration; RQ1.5b does not support production-readiness, scalability, or cross-substrate-portability claims.

6.2.5 Synthesis: Deployment Context

RQ1.4 and RQ1.5 show that the architectural pattern remains visible after the workload is moved into Kubernetes-style service chains. In local `kind`, latency increases from the monolithic Kubernetes baseline through the four-service chain, while the final five-service increment is small because the added late-stage boundary transfers a much smaller activation tensor. The same ordering and the same bounded-non-linearity pattern also survive the kind multi-worker CNI sensitivity probe of RQ1.4b (section 6.2.2), so neither finding is an artefact of single-node pod colocation on the local cluster; the kind multi-node per-condition deltas are, however, an order of magnitude larger than the AKS multi-node deltas of section 6.2.4 and should be read as a kind-CNI sensitivity envelope, not as a substitute for a managed-cloud cross-host measurement. In AKS, the same condition ordering is preserved: `monolithic_k8s_1svc`, `chain_2svc`, `chain_3svc`, `chain_4svc`, and `chain_5svc`. The absolute millisecond values differ between local Kubernetes and AKS, but the within-environment overhead trend transfers directionally, as summarised in table 5.16.

This distinction is important. The thesis does not claim that local Kubernetes predicts AKS latency numerically. Cloud and Kubernetes performance studies show that container runtime, virtualization, CNI, managed-cluster implementation, and cloud variability can change absolute measurements [6, 8, 9, 16]. The stronger supported claim is architectural: when the same ResNet-18 chain family is run in AKS, the relative ordering remains interpretable even though the platform changes.

6.3 Security Hardening

6.3.1 RQ2.1: Service Identity and Mutual TLS

The results in section 5.7 support four bounded findings.

Finding 1: the mTLS/AuthZ hardening bundle adds measurable but bounded overhead to the selected chain. For the primary `chain_2svc` topology, the service-mesh hardening layer adds 3.488 ms, or 4.62%, to mean end-to-end latency (table 5.22). This is a measurable cost, but it is bounded relative to the approximately 75.6 ms plain AKS-with-mesh-tainted baseline. The result is consistent with service-mesh literature: sidecars add critical-path work, but the visible percentage overhead depends on the workload and on how much application compute the proxy cost is amortised over [36].

Finding 2: security overhead grows with protected chain depth. The `chain_5svc` stress run increases the mean hardening overhead to 8.837 ms (10.74%). This is approximately two and a half times the absolute `chain_2svc` overhead. The larger cost is expected: the stress topology contains more injected proxies, more authorization policies, and more protected inter-service boundaries. This finding reinforces the RQ1 result that service-count decisions and security decisions cannot be separated. A chain that is tolerable without communication hardening may become less attractive once every boundary also carries identity, encryption, proxying, and policy-enforcement cost.

Finding 3: the result is lower than some published Istio overheads, but not in conflict with them. The observed 4.62% primary overhead is much smaller than the high-load Istio penalties reported in broader service-mesh benchmarks [5]. This does not contradict that literature. Those studies use different services, protocols, load levels, tail-latency metrics, mesh configurations, and deployment environments. RQ2.1 uses a CPU-bound ResNet-18 inference chain with substantial per-request model computation, same-node placement, and a specific managed-Istio AKS configuration. The correct comparison is therefore qualitative: both this thesis and the literature show that sidecar-mediated mTLS has non-zero latency and resource cost, while the absolute magnitude is workload-specific.

Finding 4: RQ2.1 hardens inter-service communication, not the entire trust model. The security validation shows that the intended service-account-based communication policy was enforced for the protected downstream segments: valid full-chain requests succeeded, while direct non-mesh probes to protected downstream services were blocked. This is meaningful communication-level hardening. However, it does not remove trust in the service-mesh control plane, does not protect against all Kubernetes administrator threats, and does not provide confidential execution for model code or activations. Those limits matter because prior work identifies the mesh control plane and local CA as security-critical trust points [26]. RQ2.1 should therefore be claimed as a measured first hardening layer, not as a complete security solution.

Answer to RQ2.1. Under the paired AKS RQ2.1 benchmark, adding service identity, managed-Istio mTLS, and inter-service authorization policy introduces measurable but bounded overhead for the selected `chain_2svc` deployment. Mean latency increases from 75.557 ms in the plain condition to 79.045 ms with the mTLS/AuthZ hardening bundle, an overhead of 3.488 ms or 4.62%; p95 latency increases by 3.623 ms. Security validation passed in every mTLS pass, including full-chain positive probes and direct-denial probes against protected downstream segments. The additional `chain_5svc` stress run shows that the same hardening approach becomes more expensive as service depth increases: mean overhead rises to 8.837 ms or 10.74%. RQ2.1 therefore supports a cautious conclusion: service identity and managed-Istio mTLS/AuthZ are feasible for the selected Azure chain at a modest latency cost, but the cost is not negligible, grows with the number of protected service boundaries, and comes with operational complexity from sidecars, policies, resource requests, and readiness delay. The follow-up ablation indicates that this cost is dominated by entering the managed-Istio sidecar data path rather than by strict mTLS enforcement or AuthorizationPolicy evaluation alone.

6.3.2 RQ2.2: Confidential VM Execution

The results in section 5.8 support three bounded findings.

Finding 1: VM-level confidential execution adds measurable overhead on top of mTLS. The RQ2.2 paired run shows positive confidential-execution overhead relative to a newly collected standard baseline. This matters because the baseline already includes the RQ2.1 mTLS/AuthZ hardening. The measured delta therefore represents the additional cost of placing the downstream service on confidential VM infrastructure, not the cost of introducing service-mesh communication security.

Finding 2: selective TEE has a modest measured cost on the protected segment. Placing only `service2` on `Standard_DC8as_v5` (AMD SEV-SNP) increases mean latency by 1.854 ms (+3.17%), median latency by 1.467 ms (+2.52%), and p95 latency by 3.601 ms (+5.98%) relative to the matched standard-node baseline (table 5.27). This is the most direct implementation of the original RQ2.2 design: `service2` is the downstream protected component, receives intermediate activations from `service1`, and is the service guarded by the strict mTLS and authorisation policy in the RQ2.1 contract. He et al. [12] motivate protecting that intermediate activation beyond the in-transit mTLS layer, although they target a malicious-remote-service threat model that differs from the infrastructure-level adversary SEV-SNP defends against; selectively placing `service2` on confidential infrastructure therefore gives the thesis a narrow, defensible hardening point with a modest measured cost.

Finding 3: the security claim is stronger than RQ2.1 but still bounded. RQ2.1 protected the service-to-service communication path. RQ2.2 adds a VM-level runtime protection boundary against host-level access for the confidentially placed service. This is a stronger security posture, but it is not complete application-level isolation. The guest OS, application process, model runtime, and local sidecar remain trusted parts of the

confidential VM environment, and plaintext exists inside the endpoint boundary after mTLS termination. Recent work additionally shows that confidential VMs retain residual attack surfaces beyond memory encryption, including malicious interrupt injection from the untrusted hypervisor [29]; this thesis does not experimentally evaluate such attacks. The correct claim is therefore that RQ2.2 adds VM-level confidential execution to the already communication-secured Azure inference chain, not that it removes every trusted component or eliminates all possible data exposure.

Answer to RQ2.2. RQ2.2 shows that AMD SEV-SNP VM-level confidential execution is feasible for the communication-secured AKS inference chain and introduces a measurable but bounded additional overhead. With only the protected downstream service moved to a confidential node (`service2_only` scope), mean latency increases by 1.854 ms, or 3.17%, and p95 latency increases by 3.601 ms, relative to a newly paired standard-node mTLS baseline. Extending the confidential-execution boundary to both services in the chain (full two-service TEE) is out of the thesis-facing scope of RQ2.2 per the evidence manifest and is treated as future work. The interpretation remains bounded by the SEV-SNP threat model and by the sidecar-based service-mesh architecture: RQ2.2 evaluates VM-level confidential execution on Azure for the downstream segment, not application-level enclave isolation or complete end-to-end secrecy.

6.3.3 RQ2.3: Trade-off Synthesis

Section 5.9 shows that the evaluated security mechanisms form a cumulative hardening gradient rather than a single binary secure/insecure choice. The RQ1.5 plain AKS deployment establishes the performance baseline. RQ2.1 adds communication-level security through mTLS, service identity, and authorization policy. RQ2.2 then adds VM-level confidential execution for the downstream protected service (`service2_only`). Each step increases the protection scope, but each step also introduces additional latency, resource, or operational cost. Extending the confidential-execution boundary to both services (full two-service TEE) is treated as future work and is not evaluated as a measured trade-off point in this thesis.

The main engineering implication is that security hardening should be scoped to the threat model. Applying every available mechanism everywhere is not always the best default. In the evaluated deployment, mTLS/AuthZ is the most suitable general-purpose security baseline because it protects the inter-service path at a modest cost for `chain_2svc`. Selective TEE on the downstream service is the next available hardening step when that segment is the primary sensitivity boundary [12], while its security claim remains bounded by the SEV-SNP threat model [29].

The results also show that performance and security cannot be evaluated independently. RQ1 established that service boundaries add measurable overhead and that this overhead depends on boundary placement and service count. RQ2 shows that security mechanisms compound this cost. The mTLS/AuthZ hardening overhead grows when more service boundaries are protected, and selective confidential execution adds further overhead on top. The final deployment decision therefore depends on three linked factors: the chosen model partition, the number of service boundaries, and the required protection scope.

Answer to RQ2.3. RQ2.3 shows that the best hardening level depends on the assumed threat model and the acceptable performance cost. Plain AKS is the fastest evaluated configuration, but it provides the weakest protection in the security comparison. mTLS with service identity and authorization policy is the best general-purpose balance when the main goal is authenticated and encrypted service-to-service communication; in the selected `chain_2svc` deployment it adds 3.488 ms, or 4.62%, mean latency overhead. Selective TEE on `service2_only` is the preferred option evaluated in this thesis when the downstream inference segment is the main infrastructure-level sensitivity boundary, adding 1.854 ms, or 3.17%, over its paired standard-node mTLS baseline. Extending the confidential-execution boundary to both services (full two-service TEE) is treated as future work and is not evaluated here as a measured trade-off point.

Overall, the evaluated configurations support a cumulative security-performance trade-off: stronger protection is feasible for the selected AKS inference chain, but each additional protection layer adds measurable cost. The most defensible default is mTLS/AuthZ for communication-level protection, with selective confidential execution on the downstream service added only when the threat model requires runtime protection against host-level exposure, and bounded by the SEV-SNP attack-surface limitations discussed above [29].

6.3.4 Synthesis: Security Hardening

The security stages show that hardening is feasible for the selected AKS deployment, but it is not free. RQ2.1 adds managed-Istio mTLS, service identity, and authorization policy. For the selected `chain_2svc` topology, the measured mean overhead is 3.488 ms, or 4.62%; for the `chain_5svc` stress topology, the overhead grows to 8.837 ms, or 10.74% (tables 5.22 and 5.23). This is consistent with service-mesh studies showing that sidecar-based mTLS adds latency and resource cost through additional critical-path processing [5, 36].

RQ2.2 adds VM-level confidential execution on an AMD SEV-SNP node for the downstream protected service while keeping the mTLS contract fixed. Selective confidential execution for `service2_only` adds 1.854 ms, or 3.17%, over its paired standard-node mTLS baseline. The result is consistent with the confidential-computing literature's expectation that CVM overhead is workload- and scope-dependent [23, 24, 34], and the security claim is bounded by SEV-SNP's residual attack surfaces [29].

6.4 Validation

This section consolidates the evidence that supports treating the thesis conclusions as credible. The validation discipline follows the shared benchmark contract defined in sections 4.3 to 4.5.

Functional parity. Every thesis-facing frozen run achieves $\text{max_abs_diff} = 0.0$ against the monolithic reference under absolute tolerance 1.0×10^{-5} and five validation inputs. This applies to all five RQ1.1 conditions (section 5.1), all four RQ1.2 conditions (section 5.2), all five RQ1.4 conditions (section 5.4), all five RQ1.5 conditions (section 5.5), all five RQ1.5b multi-node conditions (section 5.6), and all paired RQ2.1 and RQ2.2 runs (sections 5.7 and 5.8). No numerical drift is introduced by the partition or transport layer at any stage.

Cross-round consistency. Each benchmark stage reports per-condition round CVs computed as the standard deviation of per-round condition means divided by the grand mean (section 4.5). The local screening stages (RQ1.1, RQ1.2) achieve round CVs in the range 0.001–0.015. The local Kubernetes stage (RQ1.4) sees round CVs of 0.0074–0.0161 (table 5.12). The AKS single-node stage (RQ1.5) sees round CVs of 0.0075–0.0225 across the chained conditions and the monolithic baseline (table 5.17); the AKS multi-node sensitivity stage (RQ1.5b) tightens these to 0.0020–0.0099 (table 5.20). The placement metadata confirms distinct AKS nodes for the service pods, but the run does not isolate a single causal source for the low cross-round drift. These low CVs support treating the reported condition means as representative of this measured environment rather than as artefacts of a single favourable or unfavourable run.

Paired execution and transfer validation. RQ2.1 uses a paired alternated design (three mTLS-first passes, two plain-first passes) to defend against slow performance drift over the course of the benchmark. Pass-level mTLS deltas remain positive in every pass: 2.693–3.318 ms for `chain_2svc` and 8.203–9.968 ms for `chain_5svc` (table 5.23). RQ1.5 preserves the complete RQ1.4 condition ordering (`monolithic_k8s_1svc < chain_2svc < chain_3svc < chain_4svc < chain_5svc`) under managed-AKS execution (table 5.16). This directional transfer supports treating the chain-cost trend as architectural rather than as an environment-specific artefact, in line with systems-benchmarking guidance [13].

Security validation. The RQ2.1 mTLS configuration was validated through static manifest checks of peer-authentication and authorisation-policy objects, runtime validation of policy object counts, full-chain positive probes confirming that valid upstream-to-downstream requests succeeded, and direct non-mesh denial probes confirming that unauthenticated access to protected downstream services was blocked (section 5.7). All four checks passed in every paired mTLS pass. RQ2.2 collected fresh paired standard baselines for each confidential-node artifact rather than reusing older RQ2.1 data (section 5.8); incremental overhead can therefore be attributed to the confidential-node placement rather than to session-level drift.

Effect-size discipline. Effect sizes are computed from pooled per-iteration data ($N = 1,000$ per condition). At this N , Mann–Whitney p -values are near-zero for any non-trivial difference and are reported only as supplementary evidence. Cohen’s d and cross-round stability are treated as the more informative measures of effect magnitude (section 4.5 and table 5.11).

6.5 Evaluation

The main engineering lesson is that partitioning, deployment, and hardening cannot be optimised independently. The split point determines activation-transfer size and compute placement. Kubernetes and AKS add platform-specific communication and scheduling effects. Security hardening then compounds the cost of the selected service topology. This means that a decomposition that is acceptable without mTLS or confidential execution may become less attractive once every service boundary is protected.

For the evaluated workload, `chain_2svc` is the most defensible non-monolithic deployment point. It avoids the excessive boundary count of deeper chains, preserves a meaningful architectural split, and remains the lowest-overhead chained condition in both local Kubernetes and AKS. It also provides the practical base for RQ2: the mTLS/AuthZ hardening bundle is measurable but bounded on this topology, and selective TEE can protect the downstream service at a lower cost than full-chain confidential execution.

Across the evaluated stack, the monolithic Kubernetes baseline remains the fastest path; chained deployments pay an overhead that is monotonic with service count but bounded and non-linear; the local-to-AKS transfer is directional rather than numerical; mTLS adds a workload-amortised but measurable cost that grows with protected chain depth; and selective confidential-VM execution on the downstream service adds a further bounded cost on top of the communication-secured chain. Extending the confidential-execution boundary to both services is treated as future work.

6.6 Threats to Validity

The strongest validity support is internal consistency. The experiments use fixed model weights, fixed input shape, functional parity validation, repeated rounds, and a shared benchmark contract. The local and Kubernetes stages also use CPU-affinity and single-threading controls where the runtime permits. These choices follow systems benchmarking guidance that stresses controlled comparisons, explicit measurement context, and caution when interpreting performance numbers [13].

The main limitation is external validity. The remaining threats below are consolidated from the methodological scoping in chapter 4 and apply to all stages of the staged pipeline.

Single-node clusters. The thesis-facing RQ1.4 kind and RQ1.5 AKS primary runs are both single-node. Single-node deployments eliminate inter-node network hops and do not capture the scheduling and networking variability of a multi-node production cluster. RQ1.5b (section 5.6, section 6.2.4) addresses this directly with a multi-node AKS sensitivity stage that forces every chain hop across a node boundary; the measured per-condition multi-node penalty is bounded between +2.131 ms (chain_3svc) and +4.086 ms (chain_5svc), with the monolithic baseline gaining +2.305 ms (table 5.19). The single-node primary numbers should therefore be interpreted as a clean architectural-overhead measurement that excludes the inter-node network path; the multi-node sensitivity in section 5.6 provides a quantitative bound on what production multi-node deployments add on top.

CPU-only inference. All stages use CPU inference with a single thread. GPU-accelerated inference would substantially shift the compute distribution, reduce per-inference time, and change the relative weight of the gRPC boundary cost. The results are not directly generalisable to GPU-hosted microservice deployments.

Single model and workload. The thesis evaluates one model (ResNet-18) at batch size one with one input shape. Different architectures, larger activation maps, or higher-batch workloads may produce different cost curves and different optimal split regions.

Controlled conditions versus production variability. The local stages apply strict CPU stabilisation and achieve low cross-round variance. The Kubernetes and AKS stages are less controllable and exhibit higher variability. Neither environment captures the full noise of a shared multi-tenant cloud cluster, where CPU steal, network congestion, and noisy neighbours can substantially affect tail latency.

Activation-protection motivation. He et al. [12] demonstrate that an untrusted remote service holding only intermediate feature maps can reconstruct the client's input, so in-transit encryption alone does not protect activations at rest on the downstream segment. This motivates evaluating the infrastructure-level protection of the downstream service (RQ2.2) in addition to communication-level hardening (RQ2.1). The two layers address different threat models: SEV-SNP defends against the cloud host, not

against an actively malicious application running inside the protected VM, so it is not a direct solution to the malicious remote-service model in He et al.

mTLS threat-model scope. Sidecar-based mTLS protects the inter-proxy communication path but does not prevent plaintext from existing inside the endpoint after TLS termination. The security claims for RQ2.1 are explicitly scoped to communication-level hardening.

SEV-SNP threat-model scope. AMD SEV-SNP protects the VM boundary against the host platform but not against a compromised guest OS or application process, and several side-channel and availability threats fall outside its core guarantee. In particular, recent work shows that confidential VMs remain vulnerable to malicious-interrupt injection from the untrusted hypervisor [29]; this thesis does not experimentally evaluate such attacks. RQ2.2 therefore claims VM-level confidential execution for the downstream segment (`service2_only`), not complete application-level isolation. Extending the confidential boundary to both services in the chain is treated as future work.

Generalisation of the cited literature. Multi-node clusters, concurrent clients, GPUs, larger models, feature compression, autoscaling, and different service-mesh configurations could change the absolute overheads and possibly the preferred topology. The thesis therefore supports bounded claims about the evaluated system rather than a universal rule for all microservice inference deployments.

Conclusion

7.1 Summary & Findings

This thesis evaluated ResNet-18 inference as a staged microservice system. The study started with controlled local split selection, moved the selected chain family into local Kubernetes and AKS, and then measured the additional cost of service-mesh mTLS, authorization policy, and AMD SEV-SNP confidential VM execution.

The main finding is that microservice inference cost is shaped by three linked factors: where the neural network is split, where the resulting services are deployed, and how strongly the communication and execution path are hardened. RQ₁ showed that suitable ResNet-18 boundaries lie in the mid-to-late region around `layer2` and `layer3`, because early boundaries transfer larger activations and overly late boundaries become compute-degenerate. RQ_{1.4} and RQ_{1.5} then showed that the same service-count ordering remains visible in Kubernetes and AKS, although absolute latency is environment-specific. RQ_{1.5b} additionally bounded the inter-node penalty that the single-node primary design intentionally hides: forcing every chain hop across a node boundary on a multi-node AKS cluster adds between +2.131 ms and +4.086 ms per chained condition, with a +2.305 ms baseline penalty on the monolithic configuration and the architectural ordering preserved. RQ₂ showed that mTLS/AuthZ and selective AMD SEV-SNP confidential VM execution on the downstream service are feasible for the selected AKS chain, but each protection layer adds measurable overhead; the security claims are bounded by the mTLS termination boundary and by the SEV-SNP threat model [12, 29].

7.2 Contributions

The thesis makes three contributions. First, it provides a staged empirical evaluation method for neural-network microservice decomposition, moving from local split selection to Kubernetes, AKS, and security hardening without changing the model or benchmark contract. Second, it shows that activation-transfer size and compute placement must be considered together when selecting ResNet-18 service boundaries. Third, it quantifies a security-performance trade-off for the selected Azure deployment path: mTLS/AuthZ is a practical communication-hardening baseline, and selective AMD SEV-

SNP TEE on the downstream service adds an additional bounded cost while leaving the SEV-SNP residual threat model unchanged. Extending the confidential-execution boundary to both services in the chain is treated as future work rather than as a measured trade-off point.

7.3 Limitations & Threats to Validity

The conclusions are bounded by the evaluated workload and deployment setup (ResNet-18, CPU-only FP32 inference, batch size one, synchronous gRPC, and same-node Kubernetes and AKS placement for the thesis-facing primary runs) and by the bounded threat models of the evaluated security mechanisms. The RQ1.5b sensitivity stage explicitly relaxes the same-node placement constraint to bound the inter-node network penalty, but other production factors (multi-region placement, autoscaling, concurrent client load, non-default CNI plugins) remain out of scope. The full discussion of these threats is consolidated in section 6.6.

7.4 Future Work

Future work should extend the evaluation to larger and more diverse models, including transformer-based inference workloads and models with different activation profiles. The same staged method could also be applied under concurrent load, GPU execution, cross-availability-zone or cross-region placement, and autoscaling conditions; the multi-node sensitivity stage in section 5.6 bounds the intra-zone inter-node penalty but leaves these wider topology questions open. Another useful extension is to combine architectural partitioning with feature compression, since compression may change the cost of early split points. Finally, the security evaluation could be expanded to compare sidecar and sidecarless meshes, application-layer encryption, attestation workflow integration, and different confidential-computing platforms. In particular, extending the confidential-execution boundary from the selective `service2_only` scope evaluated here to both services in the chain (full two-service TEE), and evaluating SEV-SNP residual attack-surface attacks such as malicious-host interrupts [29], are explicit next steps that fall outside the canonical evidence base of this thesis.

Appendix

This appendix collects supporting material that is auxiliary to the main argument: extended sensitivity data, additional ablation breakdowns, and any figures or tables that are too large for the experiment chapters. The thesis-facing primary results remain those reported in chapters 5 and 6.

A.1 RQ1.4b: kind Multi-Worker CNI Sensitivity Data

This section holds the appendix-only data tables for the RQ1.4b kind multi-worker CNI sensitivity stage. The methodological framing of this stage is given in section 4.7.5; the analytical reading and its scope-bounding role in the deployment narrative are in section 6.2.2. As established in section 4.7.5, RQ1.4b is reported as supporting sensitivity evidence only, not as a primary thesis-facing number: kind’s worker “nodes” are Docker containers sharing one host kernel and communicating over kind’s CNI bridge, so RQ1.4b measures CNI-bridge cost on a single physical host rather than real cross-host networking. The genuine multi-host inter-node penalty is measured separately in section 5.6 on AKS.

Table A.1: RQ1.4b condition summary on the six-worker kind cluster with multi_node_anti_affinity placement. Source: merged_results/condition_summaries.csv in rq1_4b_20260515_163322. Overheads are computed against the RQ1.4b monolithic baseline.

Condition	Mean (ms)	Median (ms)	Std (ms)	p95 (ms)	Overhead vs. baseline
monolithic_k8s_1svc	67.491	61.917	10.999	89.446	—
chain_2svc	87.378	86.931	17.909	109.529	+19.887 / +29.5%
chain_3svc	120.465	121.270	20.490	142.410	+52.974 / +78.5%
chain_4svc	135.639	138.964	15.942	152.193	+68.148 / +101.0%
chain_5svc	141.169	144.692	13.893	156.041	+73.677 / +109.2%

Functional parity was verified independently of timing: $\text{max_abs_diff} = 0.0$ at $\text{atol} = 1 \times 10^{-5}$ for all five conditions in both the local segment validation and the live gRPC chain validation ($\text{num_inputs} = 5$), recorded in merged_results/parity_validation.json.

Table A.2: RQ1.4b cross-round consistency. Source: merged_results/round_consistency.json in rq1_4b_20260515_163322. Round CVs are materially inflated relative to the single-node RQ1.4 baseline (table 5.12); the highest CVs do not track chain depth, consistent with kind-CNI variability and shared-host effects on the single physical machine that hosts all six kind worker containers rather than with a chain-specific noise mechanism.

Condition	Grand Mean (ms)	Round Std (ms)	Round Range (ms)	Round CV
monolithic_k8s_1svc	67.491	3.0801	7.1194	0.0456
chain_2svc	87.378	2.5362	5.8718	0.0290
chain_3svc	120.465	6.6392	17.9867	0.0551
chain_4svc	135.639	2.5381	6.8988	0.0187
chain_5svc	141.169	3.4935	8.7029	0.0247

Table A.3: RQ1.4b kind-CNI per-condition penalty, condition-by-condition vs. the frozen single-node RQ1.4 baseline. Source: merged_results/condition_summaries.csv in rq1_4_fully_controlled_20260515_141036 and rq1_4b_20260515_163322. The per-condition delta is computed as $\Delta_c = \text{mean}_c^{\text{kind-cni}} - \text{mean}_c^{\text{single}}$, as declared in section 4.7.5. Following the cross-stage host-control caveat in section 4.7.5, the comparison is interpreted at the per-condition delta level only: RQ1.4b differs from RQ1.4 in host control state (governor / turbo / nice) as well as in topology. The monolithic delta is negative on average but is dominated by a first-round shift in RQ1.4b (round means {62.000, 68.553, 68.677, 69.107, 69.119} ms; table A.2) and a heavily skewed per-iteration distribution (mean 67.491 ms vs. median 61.917 ms; table A.1), and should not be read as evidence that the kind multi-worker topology is faster than single-node placement.

Condition	RQ1.4 single-node mean (ms)	RQ1.4b kind multi-worker mean (ms)	Δ_c (ms)
monolithic_k8s_1svc	81.066	67.491	-13.575
chain_2svc	83.387	87.378	+3.991
chain_3svc	87.728	120.465	+32.737
chain_4svc	90.572	135.639	+45.067
chain_5svc	92.535	141.169	+48.634

Infrastructure controls (image thesis-inference:latest with pull policy IfNotPresent, namespace rq14b, per-pod 1 CPU / 512 MiB Guaranteed-QoS service profile, per-pod 1 CPU / 1 GiB client profile, one thread per pool, CPU affinity to core 0) were applied uniformly across the five conditions, as preserved in merged_results/deployment_metadata.json and merged_results/environment.json. Host-level controls were weaker than in RQ1.4: writes to /sys failed because those paths were read-only in the containerised environment, the detected governor was schedutil (rather than performance), turbo was enabled, and the nice -5 request failed for permission reasons (environment.json). This degraded host-control state is one reason section 6.2.2 interprets the comparison against RQ1.4 at the per-condition delta level only and treats RQ1.4b as a kind-CNI sensitivity envelope rather than a clean topology-only swap from RQ1.4.

References

- [1] A. Akram, A. Giannakou, V. Akella, J. Lowe-Power and S. Peisert. “Performance Analysis of Scientific Computing Workloads on General Purpose TEEs”. In: *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 1066–1076. doi: 10.1109/IPDPS49936.2021.00115.
- [2] P. Alvaro, M. Adiletta, D. Cheng, A. Cockroft, F. Hady, R. Illikkal, E. Ramos and R. Soulé. “NotNets: Accelerating Microservices by Bypassing the Network”. In: *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '24)*. ACM, 2024, pp. 67–73. doi: 10.1145/3678015.3680494.
- [3] AMD. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Tech. rep. Advanced Micro Devices, Inc., 2020. url: <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>.
- [4] B. Ashraf, S. Kumar and N. Jawad. “A Policy-Driven Approach for Securing Microservices Workflow in Kubernetes Cluster”. In: *2025 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*. IEEE, 2025. doi: 10.1109/CLUSTERWorkshops65972.2025.11164216.
- [5] A. Bremler Barr, O. Lavi, Y. Naor, S. Rampal and J. Tavori. “Performance Comparison of Service Mesh Frameworks: the MTLS Test Case”. In: *2025 IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2025. doi: 10.1109/NOMS57970.2025.11073712.
- [6] D. De Sensi, T. De Matteis, K. Taranov, S. Di Girolamo, T. Rahn and T. Hoefler. “Noise in the Clouds: Influence of Network Performance Variability on Application Scalability”. In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6.3 (2022), pp. 1–27. doi: 10.1145/3570609.
- [7] A. E. Eshratifar, M. S. Abrishami and M. Pedram. “JointDNN: An Efficient Training and Inference Engine for Intelligent Mobile Cloud Computing Services”. In: *IEEE Transactions on Mobile Computing* 20.2 (2021), pp. 565–576. doi: 10.1109/TMC.2019.2947893.

- [8] A. P. Ferreira and R. O. Sinnott. “A Performance Evaluation of Containers Running on Managed Kubernetes Services”. In: *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2019, pp. 199–208. doi: 10.1109/CloudCom.2019.00049.
- [9] S. Giallorenzo, J. Mauro, M. G. Poulsen and F. Siroky. “Virtualization Costs: Benchmarking Containers and Virtual Machines Against Bare-Metal”. In: *SN Computer Science* 2.5 (2021), p. 404. doi: 10.1007/s42979-021-00781-8.
- [10] R. Guanciale, N. Paladi and A. Vahidi. “SoK: Confidential Quartet – Comparison of Platforms for Virtualization-Based Confidential Computing”. In: *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 2022, pp. 109–120. doi: 10.1109/SEED55351.2022.00017.
- [11] K. He, X. Zhang, S. Ren and J. Sun. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [12] Z. He, T. Zhang and R. B. Lee. “Attacking and Protecting Data Privacy in Edge–Cloud Collaborative Inference Systems”. In: *IEEE Internet of Things Journal* 8.12 (2021), pp. 9706–9716. doi: 10.1109/JIOT.2020.3022358.
- [13] T. Hoefler and R. Belli. “Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’15)*. ACM, 2015. doi: 10.1145/2807591.2807644.
- [14] C. Hu, W. Bao, D. Wang and F. Liu. “Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge”. In: *IEEE INFOCOM 2019 — IEEE Conference on Computer Communications*. IEEE, 2019. doi: 10.1109/INFOCOM.2019.8737614.
- [15] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars and L. Tang. “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge”. In: *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2017, pp. 615–629. doi: 10.1145/3037697.3037698.
- [16] Z. Kang, K. An, A. Gokhale and P. Pazandak. “A Comprehensive Performance Evaluation of Different Kubernetes CNI Plugins for Edge-based and Containerized Publish/Subscribe Applications”. In: *2021 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 2021. doi: 10.1109/IC2E52221.2021.00017.
- [17] D. Kaplan. “Hardware VM Isolation in the Cloud: Enabling Confidential Computing with AMD SEV-SNP Technology”. In: *Communications of the ACM* 67.1 (2023), pp. 54–59. doi: 10.1145/3623392.
- [18] P. Kayal and A. Leon-Garcia. “DNNSplit: Latency and Cost-Efficient Split Point Identification for Multi-Tier DNN Partitioning”. In: *IEEE Access* 12 (2024), pp. 79895–79911. doi: 10.1109/ACCESS.2024.3409057.
- [19] J. Li, W. Liang, Y. Li, Z. Xu and X. Jia. “Delay-Aware DNN Inference Throughput Maximization in Edge Computing via Jointly Exploring Partitioning and Parallelism”. In: *2021 IEEE 46th Conference on Local Computer Networks (LCN)*. 2021. doi: 10.1109/LCN52139.2021.9524928.

- [20] A. Masud, N. Foley, P. D. Rajarajan and P. Lama. “Where to Split? A Pareto-Front Analysis of DNN Partitioning for Edge Inference”. In: *arXiv preprint arXiv:2601.08025* (2026). doi: 10.48550/arXiv.2601.08025. url: <https://arxiv.org/abs/2601.08025>.
- [21] Y. Matsubara, D. Callegaro, S. Singh, M. Levorato and F. Restuccia. “BottleFit: Learning Compressed Representations in Deep Neural Networks for Effective and Efficient Split Computing”. In: *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*. IEEE, 2022, pp. 337–346. doi: 10.1109/WoWMoM54355.2022.00032.
- [22] Y. Matsubara, M. Levorato and F. Restuccia. “Split Computing and Early Exiting for Deep Learning Applications: Survey and Research Challenges”. In: *ACM Computing Surveys* 55.5 (2022), pp. 1–30. doi: 10.1145/3527155.
- [23] M. Misono, D. Stavrakakis, N. Santos and P. Bhatotia. “Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX”. In: *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8.3 (Dec. 2024), pp. 1–42. doi: 10.1145/3700418.
- [24] F. Mo, Z. Tarkhani and H. Haddadi. “Machine Learning with Confidential Computing: A Systematization of Knowledge”. In: *ACM Computing Surveys* 56.11 (2024), pp. 1–40. doi: 10.1145/3670007.
- [25] J. Na, H. Zhang, J. Lian and B. Zhang. “Partitioning DNNs for Optimizing Distributed Inference Performance on Cooperative Edge Devices: A Genetic Algorithm Approach”. In: *Applied Sciences* (2022). doi: 10.3390/app122010619.
- [26] A. Poudel, P. Niroula, C. MacDonald, L. Gloudemans and S. Herwig. “Mazu: A Zero Trust Architecture for Service Mesh Control Planes”. In: *Proceedings of the 18th European Workshop on Systems Security (EuroSec ’25)*. ACM, 2025. doi: 10.1145/3722041.3723100.
- [27] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu and S. Dustdar. “Fine-Grained Elastic Partitioning for Distributed DNN Towards Mobile Web AR Services in the 5G Era”. In: *IEEE Transactions on Services Computing* 15.6 (2022), pp. 3260–3274. doi: 10.1109/TSC.2021.3098816.
- [28] P. S. Schiavo, J. P. S. Milanezi, M. R. N. Ribeiro, V. M. G. Martínez, J. H. Corrêa, J. M. Nogueira, F. F. Redigolo, T. C. Carvalho and F. de Oliveira Silva. “Causal Inference for Quantifying Noisy Neighbor Effects in Multi-Tenant Cloud Environments”. In: *2026 IEEE International Conference on Communications (ICC)*. arXiv:2604.03145v1 [cs.NI], accepted for publication. IEEE, 2026.
- [29] B. Schlüter, S. Sridhara, M. Kuhne, A. Bertschi and S. Shinde. “HECKLER: Breaking Confidential VMs with Malicious Interrupts”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 3459–3476. url: <https://www.usenix.org/conference/usenixsecurity24/presentation/schluter>.
- [30] J. Shao and J. Zhang. “BottleNet++: An End-to-End Approach for Feature Compression in Device-Edge Co-Inference Systems”. In: *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6. doi: 10.1109/ICCWorkshops49005.2020.9145068.

- [31] R. Stahl, A. Hoffman, D. Mueller-Gritschneider, A. Gerstlauer and U. Schlichtmann. “DeeperThings: Fully Distributed CNN Inference on Resource-Constrained Edge Devices”. In: *International Journal of Parallel Programming* 49.4 (2021), pp. 600–624. doi: 10.1007/s10766-021-00712-3.
- [32] Z. Taufique, A. Vyas, A. Miele, P. Liljeberg and A. Kanduri. “HiDP: Hierarchical DNN Partitioning for Distributed Inference on Heterogeneous Edge Platforms”. In: *arXiv preprint arXiv:2411.16086* (2024). doi: 10.48550/arXiv.2411.16086. url: <https://arxiv.org/abs/2411.16086>.
- [33] D. Xu, X. He, T. Su and Z. Wang. “A Survey on Deep Neural Network Partition over Cloud, Edge and End Devices”. In: *arXiv preprint arXiv:2304.10020* (2023). doi: 10.48550/arXiv.2304.10020. url: <https://arxiv.org/abs/2304.10020>.
- [34] M. Yan and K. Gopalan. “Performance Overheads of Confidential Virtual Machines”. In: *2023 IEEE 31st International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 2023. doi: 10.1109/MASCOTS59514.2023.10387607.
- [35] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo and J. Zhang. “Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing”. In: *Proceedings of the IEEE* 107.8 (2019), pp. 1738–1762. doi: 10.1109/JPROC.2019.2918951.
- [36] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, M. Lentz et al. “Dissecting Overheads of Service Mesh Sidecars”. In: *Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23)*. ACM, 2023, pp. 142–157. doi: 10.1145/3620678.3624652.
- [37] X. Zhu, Y. Zhou, Y. Wang, X. Gao, A. Krishnamurthy, S. Kumar, R. Mahajan and D. Zhuo. “Rethinking RPC Communication for Microservices-based Applications”. In: *Workshop on Hot Topics in Operating Systems (HotOS '25)*. ACM, 2025, pp. 158–164. doi: 10.1145/3713082.3730375.

